



microBee

RV32IMC Soft-Core Microcontroller IP Family

EHB326E-RISCV-MCU-Book

AUTHORS

Süleyman Emre Önsay^{1,2}, Fatih Abdullahcan Güler²,
Alp Eren Kıyak¹, Müştak Erhan Yalçın¹

¹Istanbul Technical University, Faculty of Electrical and Electronics Engineering

²TÜBİTAK — Integrated Circuit Design and Education Laboratory (TÜTEL)

onsay20@itu.edu.tr, fatih.guler@bursiyer.tubitak.gov.tr,
kiyak20@itu.edu.tr, mustak.yalcin@itu.edu.tr

Document number:	microBee-DS-001
Revision:	P1.0
Product type:	Synthesizable SystemVerilog IP
Processor architecture:	32-bit RISC-V RV32IMC
Reference profile:	microBee-REF32
Publication date:	9 September 2026

Product scope. microBee is not a packaged semiconductor; it is a microcontroller IP added to an FPGA design. This system reference defines processor behavior, logical ports, the bus protocol, memory and register maps, the interrupt model, and configuration rules. Voltage, temperature, I/O standard, power, and maximum clock frequency are verified separately according to the selected FPGA, board, constraints, and implementation result.

Document Usage Rules

This document is a technical narrative and course material intended to support communication between the microBee RTL designer and the student working on this example.

Table 1: Data classes

Class	Meaning
Fixed	A property that does not change across all compatible microBee instances.
Configurable	Selected during synthesis or SoC integration and fixed for the bitstream.
REF32 profile	An example value fixed in the ready-to-use microBee-REF32 profile.
Target-dependent	Determined by the FPGA, board, memory macro, tool version, or timing constraint.
Reserved	Reserved for future use; unless otherwise specified, software must write zero.

Normative source order.

The microBee Requirements and Architecture Decision Matrix is the product's top-level architectural contract. For ISA and machine-mode behavior, RISC-V International's ratified specifications apply [1, 5]. This datasheet defines microBee-specific port, register, and integration details. A profile manifest may only freeze values that this document leaves configurable.

Contents

Document Usage Rules	i
1 Design Summary	1
1.1 Description	1
2 Introduction	2
2.1 How to Use It	2
2.1.1 Block Design	2
2.2 Xilinx Vivado	3
2.2.1 What Is It?	3
2.2.2 What Is Its Purpose?	5
2.3 What Is an IP Block?	5
2.3.1 Adding the IP Repository to the IP Catalog	6
2.4 Packaging a Prepared Block as an IP	8
2.5 Creating the Block Design and Adding microBee	8
2.5.1 Empty Block Design	8
2.5.2 Adding the microBee SoC to the Block Design	9
2.5.3 Adding AXI UART Lite	10
2.6 Connecting the Blocks with Connection Automation	12
2.6.1 First Stage: AXI and External Port Connections of UART Lite	12
2.6.2 Second Stage: Clock and Reset External Inputs	14
2.7 Forming the External Interrupt Vector	15
2.7.1 Purpose of the Module	16
2.7.2 Adding an RTL Module to the Block Design	17
2.8 IMEM / DMEM Initialization Files	20
2.9 HDL Wrapper	24
2.10 Simulation System	26
2.10.1 What Is It?	26
2.10.2 Adding the Testbench	26
2.10.3 Setting the Simulation Top	29
2.10.4 Limitations	31
2.11 Monitoring Simulation Outputs in Vivado	31
2.11.1 Waveform	31
2.11.2 Debug	33
2.12 Short Checklist	33
3 Architecture	34

3.1	Computer Architecture	34
3.1.1	What Is Architecture?	34
3.1.2	What Does Architecture Affect?	34
3.1.3	Why Should You Know Architecture?	34
3.2	Overall Architecture	35
3.2.1	Top	35
3.2.2	I/O	36
3.2.3	Pipeline	38
3.3	Register File	39
3.3.1	Features	40
3.3.2	Usage Example	40
3.3.3	Debugging	40
3.4	The Instruction Concept	40
3.4.1	What Is It?	40
3.4.2	Where Are Instructions Stored?	41
3.4.3	How Are Instructions Generated?	41
3.4.4	Who Determines It	41
3.5	Arithmetic Logic Unit (ALU)	41
3.6	MUL/DIV	42
3.6.1	MUL	42
3.6.2	DIV	42
3.6.3	Why Separate Units and Pipeline Are Used	42
3.7	Branching (Branch)	42
3.7.1	What Is It?	42
3.7.2	How Is It Expressed?	43
3.7.3	How Is It Resolved?	43
3.8	Stall and Flush Concepts	44
3.8.1	What Is Stall?	44
3.8.2	Situations That Produce Stall in microBee	44
3.8.3	What Is Flush?	45
3.8.4	Branch Flush in microBee	45
3.8.5	The Difference Between Stall and Flush	45
3.8.6	The Bubble Concept	46
3.9	Branch Prediction	46
3.9.1	What Is It?	46
3.9.2	Why Is It Necessary?	46
3.9.3	Taken and Not-Taken	46
3.9.4	Bimodal Branch Prediction	46
3.9.5	What Happens If Prediction Is Correct?	47
3.9.6	What Happens If Prediction Is Wrong?	47
3.9.7	Why Is a Simple Predictor Used in Embedded Systems?	47
3.9.8	Branch Prediction Performance	48
3.9.9	Branch Prediction from the microBee Perspective	48
3.9.10	Example	48
3.10	DMEM	49
3.10.1	What Is It?	49

3.10.2	Load and Store	49
3.10.3	How Is the Address Computed?	49
3.10.4	Data Sizes	50
3.10.5	Byte Enable	50
3.10.6	Memory Stall	51
3.10.7	DMEM and Pipeline	51
3.10.8	Load-Use Hazard	51
3.10.9	DMEM from the Embedded System Perspective	52
4	Instruction Set	53
4.1	What Is the ISA?	53
4.1.1	Definition	53
4.1.2	Who Defines It?	53
4.1.3	Purpose of the ISA	54
4.1.4	Why a Fixed ISA?	54
4.2	I Set	55
4.2.1	R-Type Format	55
4.2.2	I-Type Arithmetic Instructions	57
4.2.3	Shift Immediate Instructions	58
4.2.4	Load Instructions	58
4.2.5	Store Instructions	59
4.2.6	Branch Instructions	60
4.2.7	Jump Instructions	60
4.2.8	Upper Immediate Instructions	61
4.2.9	FENCE	61
4.2.10	Environment Instructions	61
4.3	M Set	62
4.3.1	MUL	62
4.3.2	MULH	62
4.3.3	MULHSU	62
4.3.4	MULHU	62
4.3.5	DIV	62
4.3.6	DIVU	63
4.3.7	REM	63
4.3.8	REMU	63
4.4	C Set	63
4.4.1	C.ADDI4SPN	64
4.4.2	C.LW	64
4.4.3	C.SW	64
4.4.4	C.NOP	64
4.4.5	C.ADDI	64
4.4.6	C.JAL	64
4.4.7	C.LI	65
4.4.8	C.ADDI16SP	65
4.4.9	C.LUI	65
4.4.10	C.SRLI	65

4.4.11	C.SRAI	65
4.4.12	C.ANDI	65
4.4.13	C.SUB	65
4.4.14	C.XOR	65
4.4.15	C.OR	65
4.4.16	C.AND	66
4.4.17	C.J	66
4.4.18	C.BEQZ	66
4.4.19	C.BNEZ	66
4.4.20	C.SLLI	66
4.4.21	C.LWSP	66
4.4.22	C.JR	66
4.4.23	C.MV	67
4.4.24	C.EBREAK	67
4.4.25	C.JALR	67
4.4.26	C.ADD	67
4.4.27	C.SWSP	67
4.5	NOP	67
4.6	ZicSr	68
4.6.1	CSRrW	68
4.6.2	CSRrS	69
4.6.3	CSRrC	69
4.6.4	CSRrWI	69
4.6.5	CSRrSI	69
4.6.6	CSRrCI	69
4.7	Zifencei	70
4.7.1	FENCE.I	70
5	Interrupts	71
5.1	What Is It?	71
5.2	What Is It For?	71
5.3	Interrupt Sources in microBee	72
5.4	Interrupt Acceptance Condition	72
5.4.1	mip Behavior in microBee	72
5.5	Interrupt Priority	73
5.6	Precise Interrupt and Pipeline Drain	73
5.7	mepc During an Interrupt	73
5.8	mstatus on Trap Entry	74
5.9	mret	74
5.10	Nested Interrupt	74
5.11	mtvec	75
5.12	CLINT	75
5.12.1	Software Interrupt	75
5.12.2	Timer Interrupt	76
5.13	PLIC	76
5.13.1	Interrupt ID	77

5.13.2	Priority and Enable	78
5.13.3	Trigger Modes	78
5.13.4	Claim and Complete	79
5.14	Shadow Register Bank	79
5.15	Interrupt Resolution Mechanism	79
5.16	Interrupt and Exception Difference	80
5.17	Student Setup Example (PLIC source 1)	81
5.18	Software Interrupt Example	82
5.19	Hardware Interrupt Flow (Summary)	83
5.20	Summary	83
6	General-Purpose Input/Output Ports (GPIO) and Peripherals	84
6.1	GPIO Design	84
6.1.1	GPIO Ports	84
6.1.2	Input Synchronization	85
6.1.3	Reset Behavior	85
6.1.4	Bus Access	85
6.2	GPIO Functions	86
6.3	GPIO Address Map	86
6.3.1	IN	86
6.3.2	OUT	87
6.3.3	OUT_SET	87
6.3.4	OUT_CLR	87
6.3.5	OUT_TGL	87
6.3.6	OE	88
6.3.7	VALID	88
6.3.8	Multiple GPIO Banks	88
6.4	GPIO_ENABLE = 0 Behavior	88
6.5	Register Update Time	89
6.6	How Is GPIO Used?	89
6.6.1	Example: LED Control with GPIO Bit 0	89
6.6.2	Example: GPIO Bit 1 Input Read	90
6.7	GPIO Verification Hints	90
6.8	Other Native Peripherals	90
6.9	Native Timer	90
6.10	Native UART	91
6.11	AXI UART Lite	92
6.11.1	AXI UART Lite register summary (PG142)	92
6.11.2	Application: setup from scratch, PLIC, AXI write/read	93
7	Configurations	96
7.1	Fixed Configurations	96
7.1.1	RV32C	96
7.1.2	Register file and LUT/FF	97
7.2	Configurable Configurations	97
7.2.1	REF32 summary	97

7.2.2	MUL/DIV	97
7.2.3	Branch predictor	97
7.2.4	Shadow bank	98
7.2.5	IMEM / DMEM size and preload	98
7.2.6	PLIC	98
7.2.7	GPIO	98
7.2.8	AXI	99
7.2.9	CLINT, timer, native UART, monitoring	99
7.2.10	TRACE_ENABLE and DEBUG_ENABLE	99
7.2.11	Reset vector and address windows	99
7.2.12	Single-parameter change principle	99
7.3	Fixed Features and Frequently Confused Points	100
8	Monitoring and Performance Counters	101
8.1	What Is It?	101
8.2	Counter Model	101
8.3	MON_CTRL	102
8.3.1	ENABLE	102
8.3.2	FREEZE	102
8.3.3	CLEAR	102
8.3.4	SNAPSHOT	103
8.4	Control Priority	103
8.5	Monitoring Register Map	103
8.6	Meaning of the Basic Counters	104
8.6.1	CYCLES	104
8.6.2	RETIRED	105
8.6.3	STALL	105
8.6.4	FLUSH	105
8.6.5	BUBBLE	105
8.6.6	HAZARD and LOAD_USE	105
8.7	Branch Counters	106
8.7.1	BRANCH	106
8.7.2	BRANCH_TAKEN	106
8.7.3	MISPREDICT	106
8.7.4	PREDICT_CORRECT	106
8.7.5	SPEC_REDIRECT	106
8.7.6	BRANCH_STALL	107
8.8	Memory Wait Counters	107
8.8.1	IMEM_WAIT	107
8.8.2	DMEM_WAIT	107
8.8.3	MEM_WAIT_TOTAL	107
8.8.4	AXI_WAIT	107
8.9	BUS_ERROR and IRQ_TAKEN	107
8.9.1	BUS_ERROR	107
8.9.2	IRQ_TAKEN	108
8.10	Execution Unit Usage	108

8.11	Load and Store Counters	108
8.12	The CAPABILITY Register	108
8.13	Performance Measurements	108
8.13.1	IPC	109
8.13.2	CPI	109
8.13.3	Stall Rate	109
8.13.4	Branch Prediction Accuracy	110
8.14	How Are Counters Interpreted Together?	110
8.15	Points to Watch While Measuring	110
8.16	Interpreting the CPI Target	111
8.17	Example Measurement Flow	111
8.17.1	Example: writing the CYCLES counter to AXI UART Lite	111
9	Preparing the Performance Report	114
9.1	How Is the System Observed?	114
9.1.1	Waveform and Debug Signals	115
9.1.2	Monitoring Counters	115
9.1.3	Simulation Log and Trace	116
9.2	Which Method Should Be Used When?	116
9.3	System Observation Tools	116
9.3.1	AXI UART Lite and Host Terminal	116
9.3.2	Vivado Waveform	117
9.3.3	Computation	117
9.4	How Is the Performance Report Prepared?	117
9.4.1	Identity Information	118
9.4.2	Raw Counters	118
9.4.3	Derived Metrics	119
9.4.4	How Are CPI and IPC Interpreted?	119
9.5	How Are Counters Interpreted Together?	120
9.6	Example Performance Report	120
9.6.1	Host Output	120
9.7	Configuration Comparison	121
9.7.1	Why Is Only One Parameter Changed?	121
9.7.2	Experiment 1: predictor capacity (GShare off)	121
9.7.3	Experiment 2: GShare, BP_ENTRIES=128	122
9.7.4	Summary of the Experiment Result	122
9.8	Common Mistakes	123
9.8.1	MONITOR_ENABLE = 0	123
9.8.2	Counter Saturation	123
9.8.3	Not Using Freeze or Snapshot	123
9.8.4	Including UART Output in the Measurement	123
9.8.5	Mistaking Different Binaries for a Core Comparison	123
9.8.6	Taking the Simulation Cycle to Be Real Time	124
9.8.7	Looking at a Single Metric	124
9.8.8	Mixing Live and Shadow Counters	124
9.9	Minimum Valid Performance Report	124

10 Debug Ports	125
10.1 Structure of the Debug Ports	125
10.1.1 Pipeline PC observation	125
10.1.2 Register file write port	126
10.1.3 CSR snapshot	126
10.1.4 Optional trace (TRACE_ENABLE)	127
10.2 Use of the Debug Ports	127
10.2.1 When to Use dbg_* and When to Use Monitoring	127
10.2.2 Vivado Block Design simulation	127
10.2.3 Verilator / tmcu_top	128
10.2.4 Reading order	128
10.3 Examples of the Debug Ports	128
10.3.1 First fetch after reset	128
10.3.2 An ALU write	128
10.3.3 Load	128
10.3.4 Branch	129
10.3.5 Trap	129
10.3.6 A character written to UART	129
10.4 Limitations of the Debug Ports	129
11 rv32 Emulator and Assembly Development Environment	131
11.1 Introduction	131
11.1.1 What Is It?	131
11.1.2 Purpose	132
11.1.3 Usage	132
11.1.4 Limitations	132
11.2 Installation and First Launch	132
11.2.1 Windows	132
11.2.2 Linux	132
11.2.3 Layout of the Working Files	132
11.2.4 Troubleshooting Startup Problems	133
11.3 Interface and Run Controls	133
11.4 First Program: Arithmetic, Store, and Load	134
11.4.1 Usage Example	134
11.4.2 How to Run It	134
11.4.3 Student Exercise	134
11.5 Register, Memory, and Instruction Observation	135
11.5.1 Reading the Register File	135
11.5.2 Computing the DMEM Address	135
11.5.3 Encoding and Symbols	135
11.6 Pipeline and Debugging	135
11.6.1 Reading the Pipeline	135
11.6.2 Meaning of One Step	136
11.6.3 Difference from the Hardware Pipeline	136
11.7 Breakpoint, Step Back, and Diagnostic Messages	136
11.7.1 Examining a Loop	136

11.7.2 Using Code Completion	137
11.8 Peripheral and Interrupt Studies	137
11.8.1 UART Usage Example	137
11.8.2 Loading the Configuration	137
11.8.3 What Should Be Considered When Handling Interrupts?	137
11.9 Producing a Memory Image	138
11.9.1 Using Generate	138
11.9.2 Reading the File	138
11.9.3 Requirements for Using the Output in Vivado	138
11.10 Use Together with microBee-REF32	138
11.10.1 Does a Board File Make rv32 REF32-Compatible?	139
11.10.2 Preparing the Port	139
11.11 Command Line and Repeatable Experiment	139
11.11.1 Basic Commands	139
11.11.2 Using the Debugger	139
11.12 Student Exercises and Result Record	140
11.12.1 Exercise Order	140
11.12.2 Reporting the Result	140
11.12.3 Review Questions	141
11.13 Sources and Relation Across Chapters	141
A Address Map and System Capacity	143
A.1 REF32 Capacity Summary	143
A.2 Global Memory Map	144
A.3 DMEM — 0x20000000	144
A.4 CLINT — 0x40000000	145
A.5 PLIC — 0x40001000	145
A.6 GPIO — 0x40002000	145
A.7 TIMER — 0x40003000	146
A.8 Native UART — 0x40004000	146
A.9 MONITOR — 0x40005000	147
A.10 AXI UART Lite — 0x60000000	147
A.11 IMEM — 0x80000000	148
A.11.1 Alignment	148
A.12 CSR Numbers	148
A.12.1 mcause Codes	149

List of Figures

2.1	Final state of the sample laboratory Block Design: <code>tmcu_soc_0</code> , AXI Interconnect, AXI UART Lite, Clocking Wizard, Processor System Reset and <code>interrupt_order_manager</code> .	3
2.2	Vivado Project Summary screen: target FPGA, top module and synthesis/implementation status.	4
2.3	Project Manager, IP Integrator, Simulation and subsequent FPGA design steps within the Flow Navigator.	5
2.4	IP Catalog under Project Manager.	6
2.5	Add Repository selection on the IP Catalog.	7
2.6	Selecting the <code>tmcu_soc</code> IP repository folder.	7
2.7	Create Block Design window.	8
2.8	Empty Block Design workspace.	9
2.9	Add IP selection within the Block Design.	9
2.10	microBee SoC RV32IMC in the Add IP search.	10
2.11	Only <code>tmcu_soc_0</code> within the Block Design.	10
2.12	Adding the AXI UART Lite IP to the Block Design.	11
2.13	Before microBee and AXI UART Lite are connected to each other.	11
2.14	Run Connection Automation through Designer Assistance.	12
2.15	Selecting the AXI UART Lite <code>S_AXI</code> and UART connections in the first Connection Automation window.	13
2.16	System infrastructure formed after the first Connection Automation.	14
2.17	Second Connection Automation: clock and reset external connections.	14
2.18	Block Design after clock and reset automation is completed.	15
2.19	<code>interrupt_order_manager</code> : combining 16 single-bit interrupt inputs into a 16-bit external interrupt vector.	16
2.20	Add Module selection on the Block Design.	17
2.21	Selecting the <code>interrupt_order_manager</code> module in the Add Module window.	18
2.22	Adding the <code>interrupt_order_manager</code> module to the Block Design.	18
2.23	Connecting the <code>interrupt_array[15:0]</code> output to microBee's <code>ext_irq_i[15:0]</code> input.	19
2.24	Connecting the AXI UART Lite <code>interrupt</code> output to the <code>in_0</code> input.	20
2.25	Memory and peripheral parameters together with the Imem Init File field in the Customize IP window.	21
2.26	Entering the Imem Init File path.	22
2.27	Dmem Init File and AXI address parameters in the Customize IP window.	23
2.28	Dmem Init File example: <code>soc_integration_dmem_poll.hex</code> .	24
2.29	Create HDL Wrapper operation for <code>design_1.bd</code> .	25
2.30	The <code>design_1_wrapper</code> module under Design Sources.	25
2.31	Simulation source selection within Add Sources.	27
2.32	Adding the <code>tb_design_1_hello.sv</code> file as a simulation source.	27

2.33	Creating the testbench module on the Define Module screen.	28
2.34	Defining clock, reset, UART and the expected “Hello” message in the testbench source.	28
2.35	Set as Top selection within Simulation Sources.	29
2.36	Selecting the <code>tb_design_1_hello</code> module as the simulation top.	30
2.37	Run Simulation within the Flow Navigator.	30
2.38	Receiving “Hello” characters over UART on the behavioral simulation waveform.	32
2.39	Tcl Console: UART “Hello” output and the PASS: <code>tb_design_1_hello</code> message.	32
3.1	microBee SoC high-level architecture (<code>tmcu_soc</code> , REF32).	36
3.2	microBee SoC external I/O ports and functional groups (<code>tmcu_soc</code>).	38
3.3	ARM architecture ABI map [6].	40
3.4	Transformation of C source code into RISC-V assembly output by the compiler.	41
11.1	When the first instruction reaches the write-back stage, register a0 becomes 5. The Encoding tab shows the operation that occurred.	133
11.2	After the first program completes, result = 8 is observed in data memory. The row address and the column offset are read together.	135
11.3	Pipeline view in the <code>sum_loop.s</code> program. The last cycles can be followed on an instruction basis.	136

List of Tables

1	Data classes	i
3.1	RISC-V integer registers [1]	39
3.2	Register file debug signals and descriptions	40
5.1	microBee interrupt sources and mcause values.	72
5.2	CLINT timer register addresses.	76
5.3	PLIC configuration addresses and access states.	77
6.1	GPIO register address map and access types.	86
6.2	AXI UART Lite register addresses and fundamental roles.	92
6.3	AXI UART Lite STAT and CTRL bits (PG142).	92
7.1	Fixed architecture independent of synthesis.	96
7.2	Units enabled in REF32 and core switches. tmcu_soc default unless otherwise stated.	97
8.1	Monitoring registers	103
8.2	Derived performance measurements	109
9.1	microBee performance observation methods.	114
9.2	Identity information that should be kept in the performance report.	118
9.3	Recommended basic monitoring counters in the performance report.	118
9.4	Basic metrics used in the performance report.	119
9.5	Decimal equivalents of the example monitoring output.	120
9.6	Embench weighted results, GShare off, only BP_ENTRIES changes.	121
9.7	Embench weighted results, BP_ENTRIES =128, GShare scan.	122
9.8	Short summary of the predictor experiments.	122
10.1	Debug PC ports and RTL source.	125
10.2	Debug valid bits.	126
10.3	Register file debug write port.	126
10.4	Debug CSR ports.	127
11.1	Information classes used in this chapter	131
11.2	Situations frequently seen during installation	133
11.3	Basic shortcuts	133
11.4	Expected values at the end of the program	134
11.5	Common errors and recommended fixes	137
11.6	rv32 default peripheral addresses	137
11.7	Scope of the export operations	138

11.8 Differences affecting porting between REF32 and rv32	139
11.9 Terminal debugger commands	140
11.10 Starter exercises	140
11.11 Functional experiment record template	140
11.12 Sources used	141
A.1 REF32 system capacity.	143
A.2 REF32 memory windows, in address order.	144
A.3 CLINT register map.	145
A.4 PLIC register map.	145
A.5 GPIO register map.	146
A.6 Native Timer register map.	146
A.7 Native UART register map.	146
A.8 Monitoring register map.	147
A.9 AXI UART Lite register map.	148
A.10 microBee machine-mode CSR set, in number order [5].	148
A.11 Exception cause codes, bit31=0.	149
A.12 Interrupt cause codes, bit31=1.	149

DRAFT

Chapter 1

Design Summary

1.1 Description

microBee is a 32-bit RISC-V soft-core microcontroller developed in partnership between the TÜBİTAK Integrated Circuit Design and Education Laboratory (TÜTEL) bursary laboratory and the Istanbul Technical University Embedded Systems Design Laboratory (GSTL). It is designed for embedded systems education, computer architecture laboratories, FPGA-based control systems, and small real-time applications. A single IP family includes RV32I, multiply and divide units that can be enabled independently of each other, an RV32C predecoder, a machine-mode trap system, local interrupt control, memory-mapped peripherals, performance monitoring, and AXI4-Lite connectivity.

The core is single-issue and in-order. The main application organization is a five-stage IF-ID-EX-MEM-WB pipeline. Instruction and data accesses are performed over separate Harvard instruction/data paths; both paths use the same valid/ready request-response interface protocol. This structure supports small systems connected directly to FPGA block RAMs and broader SoC integrations over AXI4-Lite with the same processor core.

Chapter 2

Introduction

This chapter describes microBee’s general usage structure and how it is run within a student laboratory on Xilinx Vivado.

The goal is not to examine the core RTL line by line; it is to place the packaged `tmcu_soc` IP into a Block Design, communicate with AXI UART Lite, and verify the “Hello” output on behavioral simulation.

microBee is an RV32IMC-based soft-core design. It is packaged as IP for use on FPGA. Interfaces such as clock, reset, AXI master, GPIO, Native UART and external interrupt appear on the Block Design.

The configuration of the pipeline, memory and peripheral structures inside the core is determined by the parameters given through Customize IP.

Note

The screenshots in this chapter were taken from a sample Vivado project (`tmcu_ip_deneme_1`, Artix-7 `xc7a100tcsg324-1`).

The IP repository path may vary depending on the computer used and the working directory. The microBee IP packaged in the repo is typically found under:

```
work/fpga_synth/ip/tmcu_soc
```

In the laboratory example used in this chapter, the Block Design and testbench were prepared with a 100 MHz system clock and a 9600 baud UART setting. The 50 MHz / 115200 configuration used in the REF32 profile is a separate FPGA configuration. Clock and baud values must be changed together.

2.1 How to Use It

The laboratory flow generally consists of the following steps:

1. The Vivado project is opened and the microBee IP repository is added to the IP Catalog.
2. An empty Block Design is created. Then the `tmcu_soc` and AXI UART Lite IPs are added to the design.
3. AXI, clock and reset connections are created using *Connection Automation*.
4. Optionally, the helper RTL module named `interrupt_order_manager` is added in order to place external interrupt sources onto the `ext_irq_i[15:0]` line.
5. IMEM and DMEM initialization files are defined through Customize IP.
6. The HDL wrapper is created. The testbench is added under Simulation Sources and is selected as the simulation top.
7. Behavioral simulation is run. The expected text sent over UART is verified on the waveform and in the simulation console.

2.1.1 Block Design

The target system is a small SoC structure in which microBee operates as an AXI master.

While the Clocking Wizard produces the system clock, Processor System Reset generates the required reset signals. microBee accesses the AXI UART Lite peripheral through the AXI Interconnect.

The AXI UART Lite interrupt output can be connected to microBee’s `ext_irq_i[15:0]` external interrupt

vector through the helper `interrupt_order_manager` module.

The basic connections of the Block Design with the outside world in the sample project are:

- `clk_100MHz`,
- `reset_rtl_0`,
- `uart_rtl_0`

`reset_rtl_0` is used as an active-high reset at the external input level. microBee's active-low `rst_ni` input is produced through the required reset infrastructure.

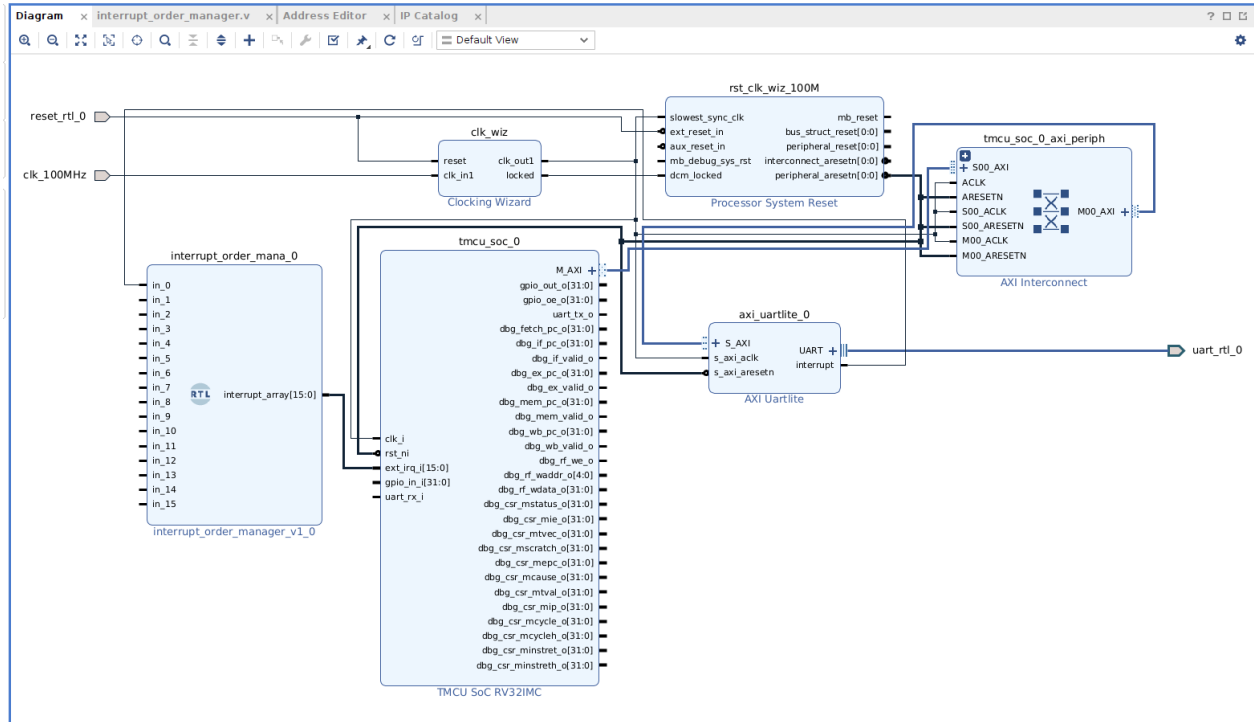


Figure 2.1: Final state of the sample laboratory Block Design: `tmcu_soc_0`, AXI Interconnect, AXI UART Lite, Clocking Wizard, Processor System Reset and `interrupt_order_manager`.

The following sections will describe how this Block Design is created step by step.

2.2 Xilinx Vivado

2.2.1 What Is It?

Vivado is the integrated FPGA design environment used for AMD/Xilinx FPGAs.

Within a single project:

- RTL sources,
- IP Catalog,
- IP Integrator and Block Design,
- behavioral simulation,
- synthesis,
- placement and routing,
- bitstream generation

operations can be managed.

In a new or empty project the main view is generally the *Project Summary* screen.

The *Flow Navigator* on the left is used to access the project's different design stages.

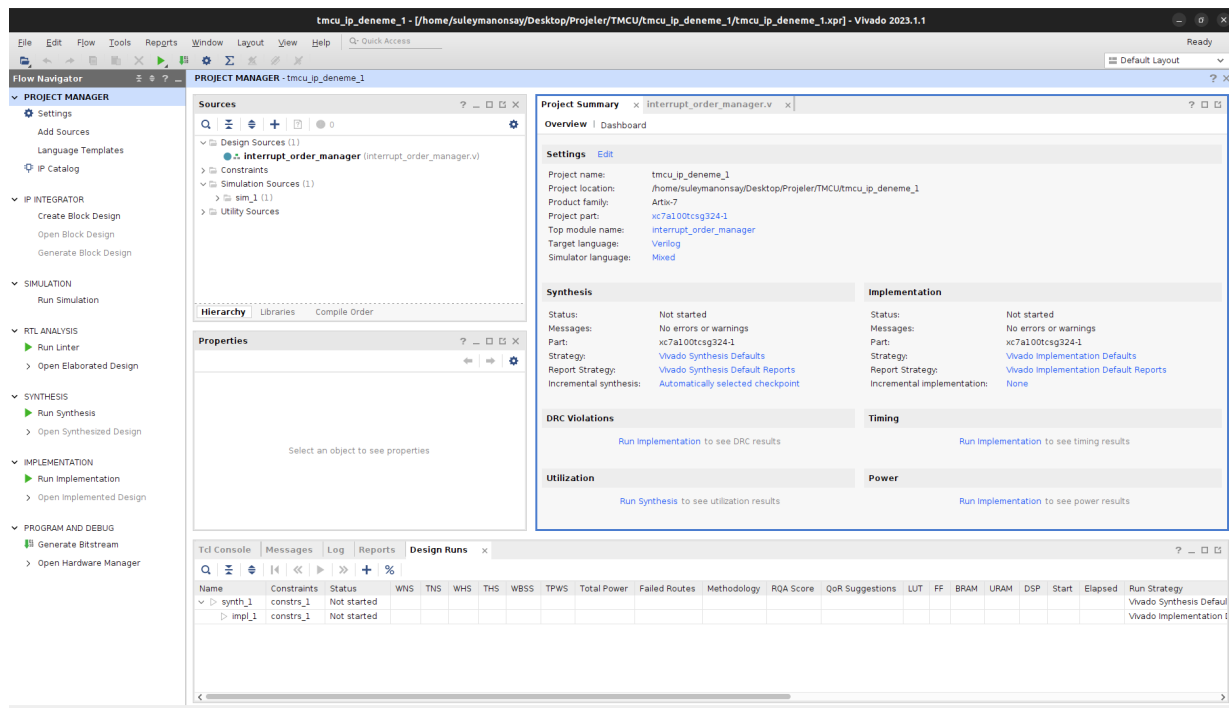


Figure 2.2: Vivado Project Summary screen: target FPGA, top module and synthesis/implementation status.

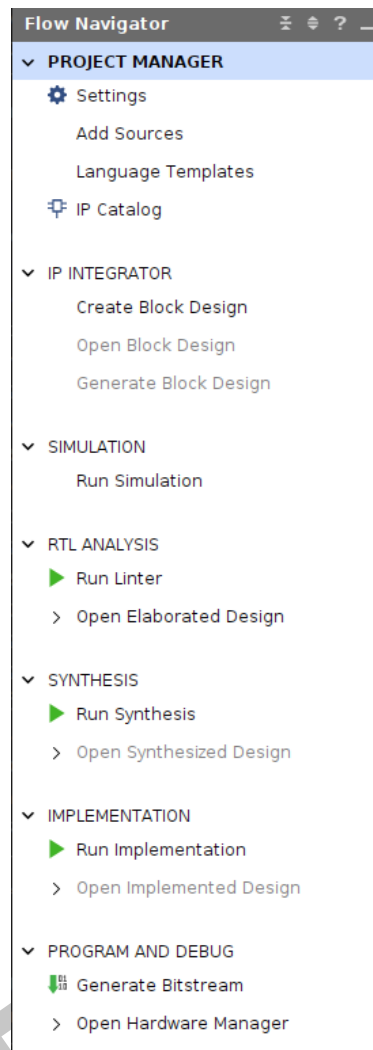


Figure 2.3: Project Manager, IP Integrator, Simulation and subsequent FPGA design steps within the Flow Navigator.

2.2.2 What Is Its Purpose?

In this laboratory, three main parts of Vivado will be used:

- **IP Catalog and Repository:** Used to introduce ready-made or user-packaged IPs to the project. The repository is the file location of the IP; the IP Catalog is the interface in which Vivado lists these IPs.
- **IP Integrator / Block Design:** Used to visually connect microBee, clock, reset, AXI peripheral and helper RTL blocks to one another.
- **Simulation:** Used to verify the functional behavior of the system through the HDL wrapper and testbench. UART and, when necessary, `dbg_*` signals can be observed on the waveform.

Bitstream generation and loading onto the FPGA are later stages of the same design flow.

In this chapter the primary goal is to show that the system operates correctly in behavioral simulation before moving to the physical FPGA.

2.3 What Is an IP Block?

IP, that is *Intellectual Property*, is a reusable hardware block.

A Vivado IP generally has:

- specific input/output ports,
- configuration parameters,
- RTL sources,

- IP-XACT definitions such as `component.xml`,
- XGUI configuration files

AMD/Xilinx AXI UART Lite is an IP. The microBee SoC similarly appears in the Vivado IP Catalog as a packaged user IP.

The microBee IP is introduced to the project as an IP repository rather than by adding RTL files one by one. Thus, within the Block Design, with the

Add IP

command:

```
microBee SoC RV32IMC
```

can be searched and added into the Block Design as an IP instance.

2.3.1 Adding the IP Repository to the IP Catalog

1. **IP Catalog** is opened within the Flow Navigator (Figure 2.4).
2. An empty area within the IP Catalog is right-clicked and **Add Repository...** is selected (Figure 2.5).
3. The packaged microBee IP folder is selected:

```
.../work/fpga_synth/ip/tmcu_soc
```

(Figure 2.6).

The `src`, `sim`, `xgui` and related package files are expected to be present inside the IP folder.

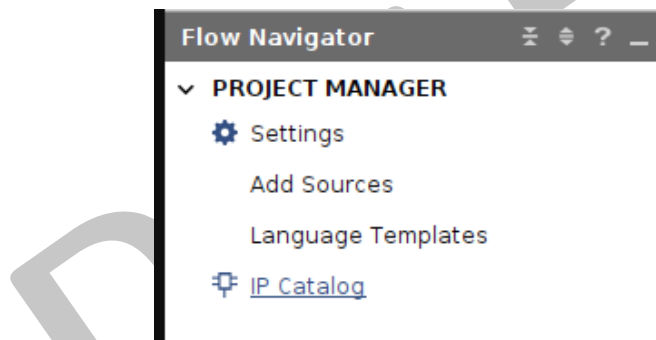


Figure 2.4: IP Catalog under Project Manager.

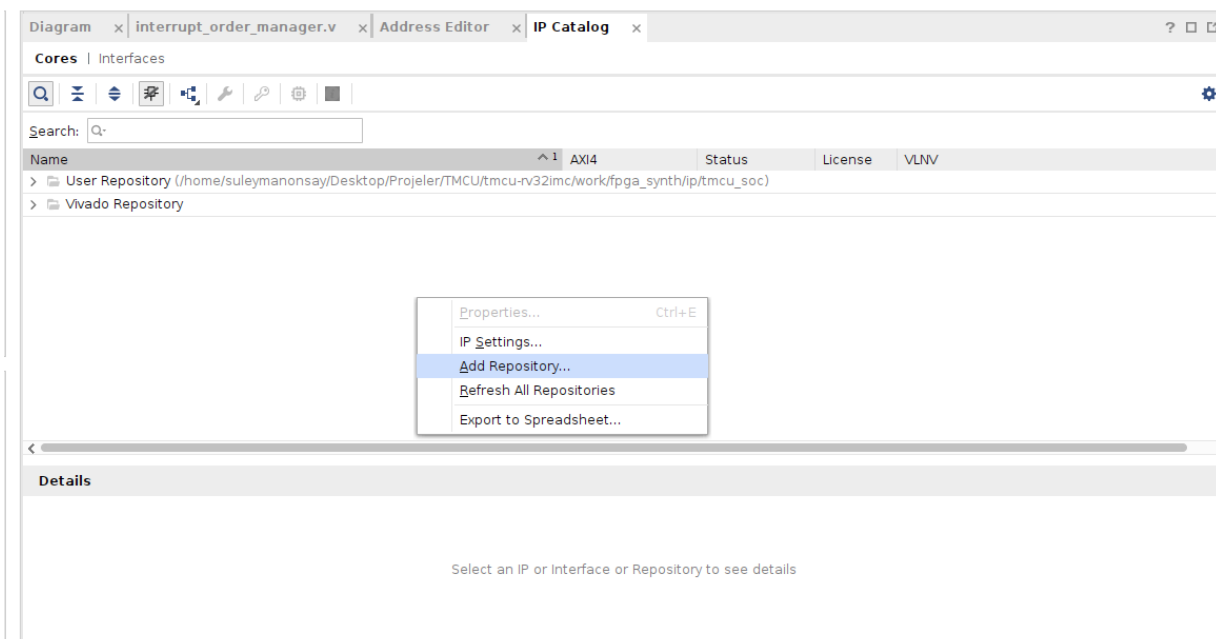


Figure 2.5: Add Repository selection on the IP Catalog.

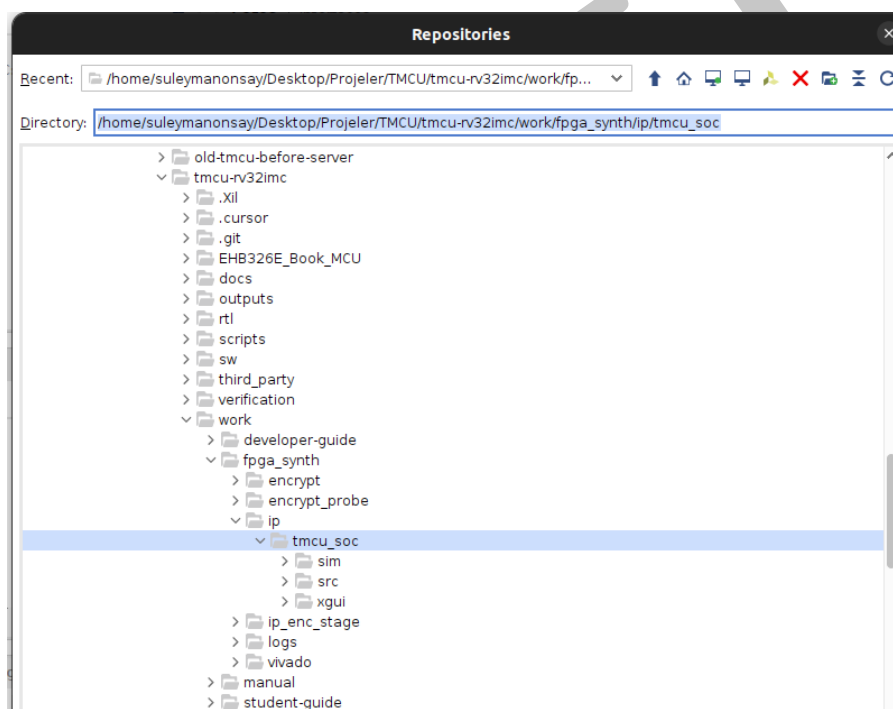


Figure 2.6: Selecting the tmcu_soc IP repository folder.

Note

The repository path should point to the `tmcu_soc` folder where the packaged IP is located. If the IP was re-packaged, the following operation can be applied within Vivado:

Refresh All Repositories

If an older microBee IP instance is present in the project, **Upgrade IP** may additionally be required.

2.4 Packaging a Prepared Block as an IP

In the student laboratory, a previously packaged microBee IP is mostly used.

This IP is the form of the RTL sources in the repository converted into an IP-XACT package that Vivado can recognize.

Typical packaging options within the repository:

- **Unencrypted IP:**

```
scripts/package_tmcu_ip.tcl → work/fpga_synth/ip/tmcu_soc
```

- **Optional encrypted IP:**

```
make ip-encrypted → work/fpga_synth/ip_enc/tmcu_soc
```

Within the packaged IP, the `tmcu_soc` top module and the parameters exposed to the user appear in the Customize IP window.

In this way the Block Design user does not need to add all RTL files belonging to the core to the project one by one.

The remainder of this chapter will focus on the use of the IP rather than its packaging.

2.5 Creating the Block Design and Adding microBee

2.5.1 Empty Block Design

Within the Flow Navigator:

IP Integrator → Create Block Design

is selected.

For the laboratory the Block Design name can be left as:

`design_1`

(Figure 2.7).

When the empty Block Design opens, Vivado shows the warning:

Press the + button to add IP

(Figure 2.8).

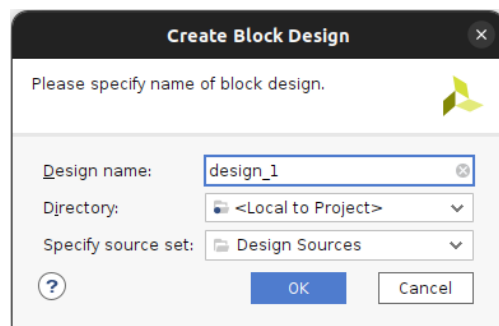


Figure 2.7: Create Block Design window.

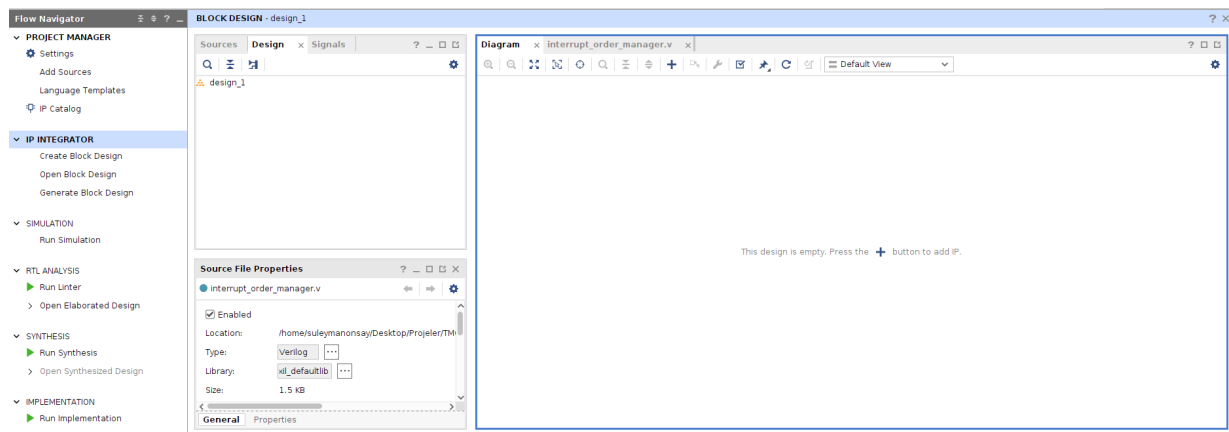


Figure 2.8: Empty Block Design workspace.

2.5.2 Adding the microBee SoC to the Block Design

A right-click is performed within the Block Design and:

Add IP...

is selected.

Ctrl+I can be used as a keyboard shortcut (Figure 2.9).

The following is typed into the search box:

microBee

and:

microBee SoC RV32IMC

is selected (Figure 2.10).

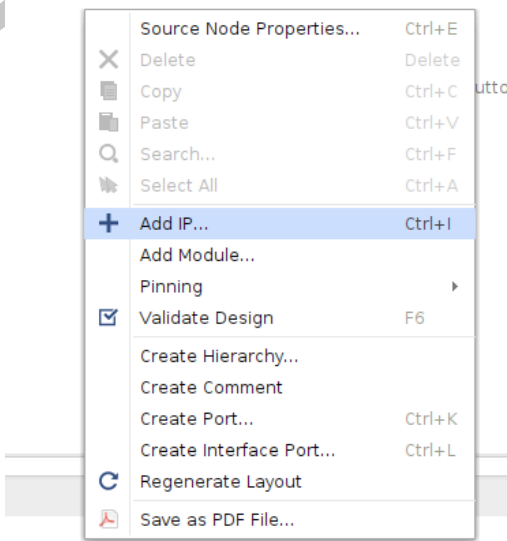


Figure 2.9: Add IP selection within the Block Design.

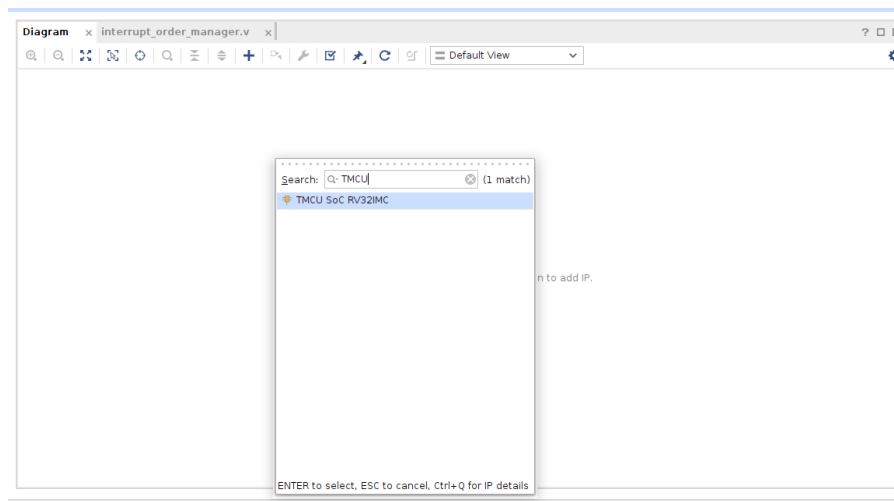


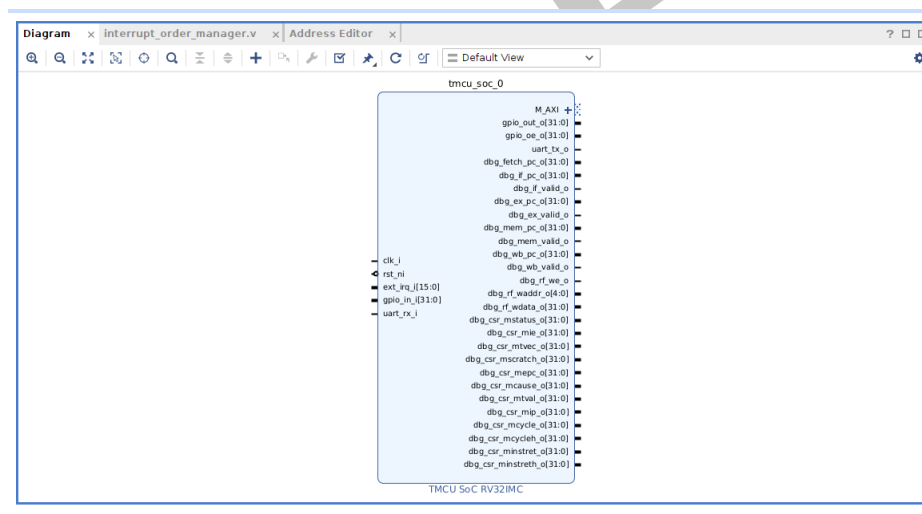
Figure 2.10: microBee SoC RV32IMC in the Add IP search.

The first microBee instance added into the Block Design generally takes the name:

`tmcu_soc_0`

Signals such as clock, reset, interrupt, GPIO and Native UART, together with the M_AXI master interface, appear on the IP.

When `DEBUG_ENABLE=1`, the `dbg_*` debug ports can also be used at the Block Design level (Figure 2.11).

Figure 2.11: Only `tmcu_soc_0` within the Block Design.

2.5.3 Adding AXI UART Lite

To add a UART that microBee can access over AXI, **Add IP** is opened again.

In the search box:

`UARTlite`

is searched and:

AXI UART Lite

is selected (Figure 2.12).

At this stage microBee and AXI UART Lite are not yet connected to each other.

When Vivado detects interfaces that can be connected, it may show the following warning on the Block Design:

Designer Assistance available. Run Connection Automation

(Figure 2.13).

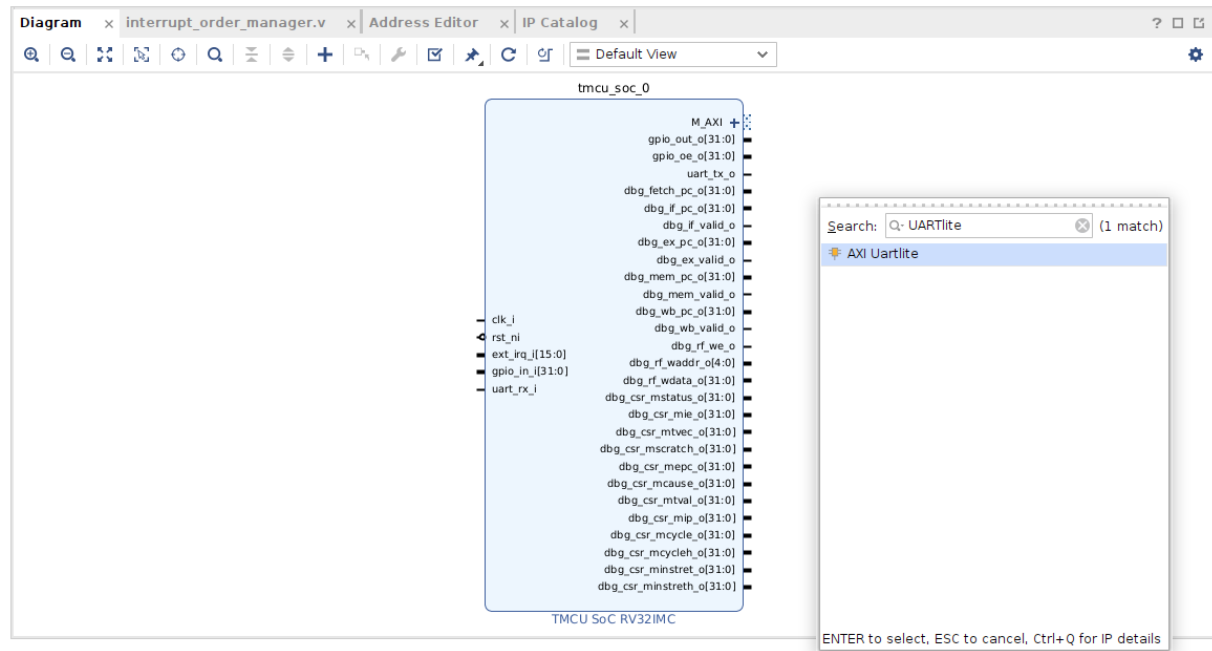


Figure 2.12: Adding the AXI UART Lite IP to the Block Design.

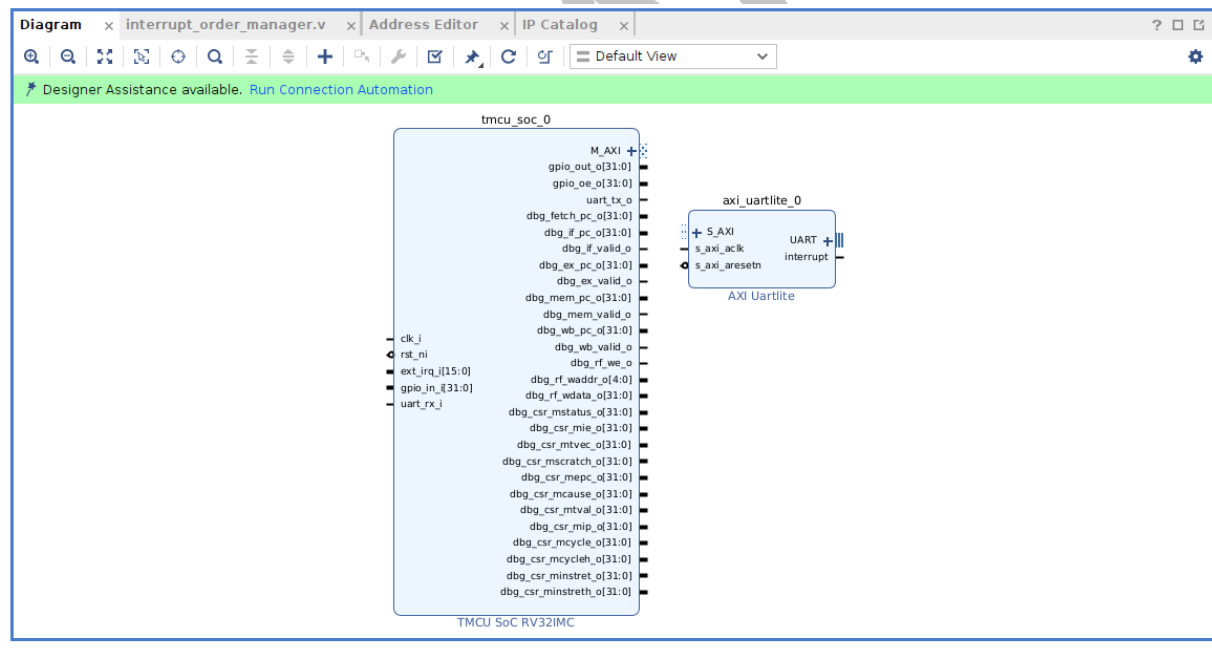


Figure 2.13: Before microBee and AXI UART Lite are connected to each other.

Note

microBee's own Native UART unit and AXI UART Lite are two independent peripherals. Native UART uses the ports:

`uart_rx_i / uart_tx_o`

In this laboratory, AXI UART Lite in the AXI address window is used primarily for serial communication. A sample base address for AXI UART Lite is chosen as:

`0x60000000`

The external RX/TX connection is brought out at the Block Design level through the `uart_rtl_0` interface.

2.6 Connecting the Blocks with Connection Automation

Connection Automation enables Vivado to create a significant portion of the AXI, clock and reset connections automatically.

In this laboratory design, Connection Automation is used in two stages.

2.6.1 First Stage: AXI and External Port Connections of UART Lite

The option that appears on the Block Design:

Run Connection Automation

is run (Figure 2.14).

In the window that opens, the following connections are selected for `axi_uartlite_0`:

- S_AXI,
- UART

(Figure 2.15).

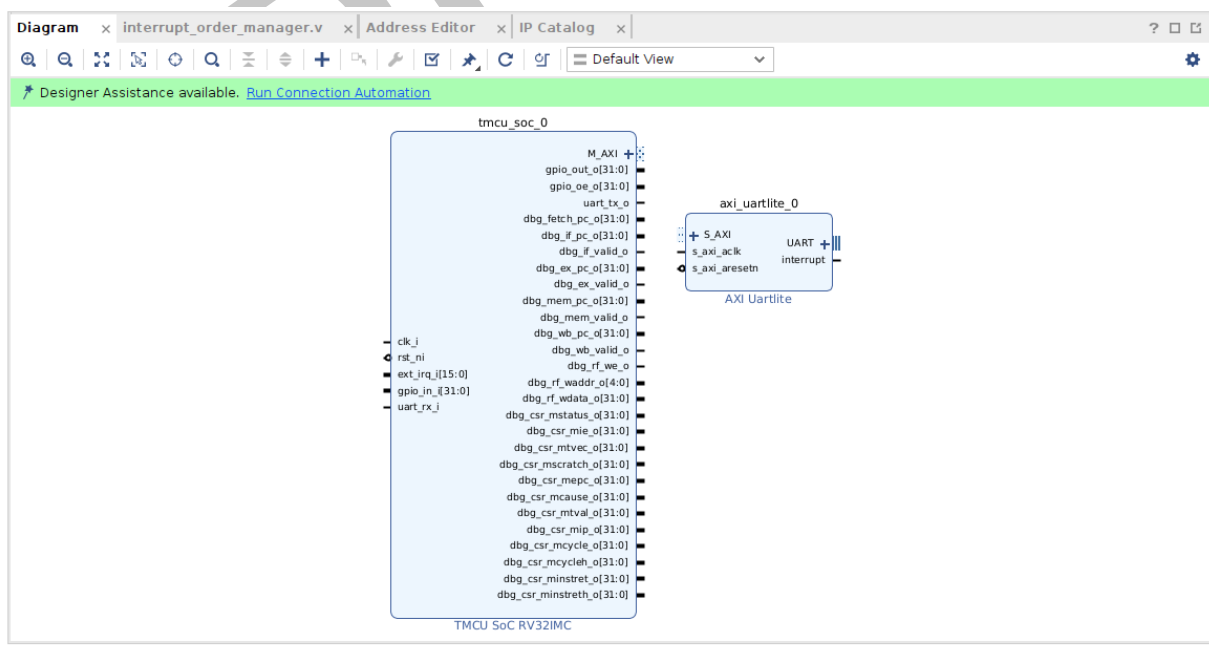


Figure 2.14: Run Connection Automation through Designer Assistance.

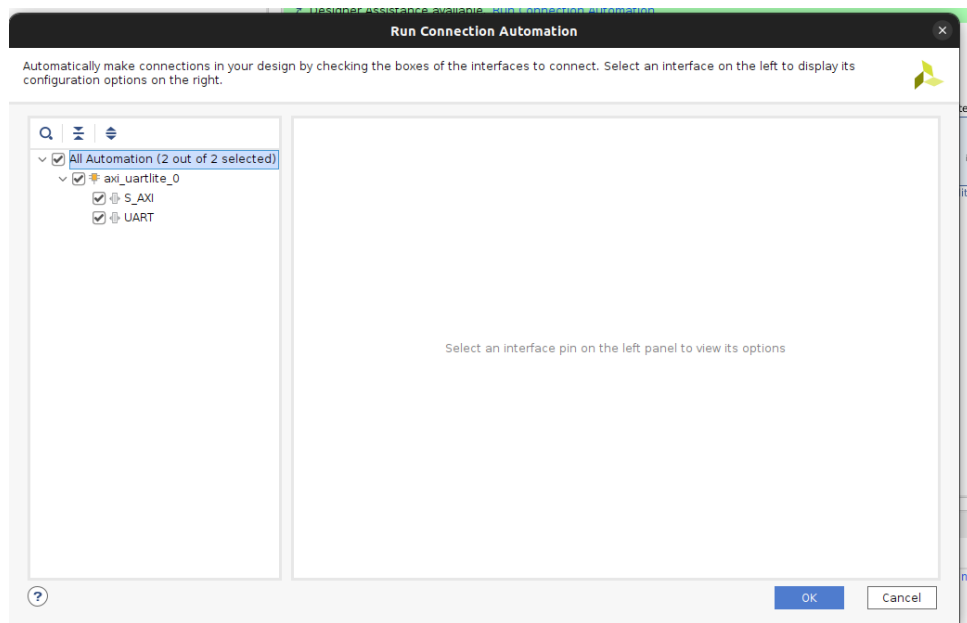


Figure 2.15: Selecting the AXI UART Lite **S_AXI** and **UART** connections in the first Connection Automation window.

After the first automation, Vivado may add the required AXI and clock/reset infrastructure blocks to the design. For example:

- AXI Interconnect,
- Clocking Wizard,
- Processor System Reset

blocks are created.

microBee's **M_AXI** master interface is connected to AXI UART Lite through the AXI Interconnect.

The UART interface can be created as an external port with the name:

`uart_rtl_0`

(Figure 2.16).

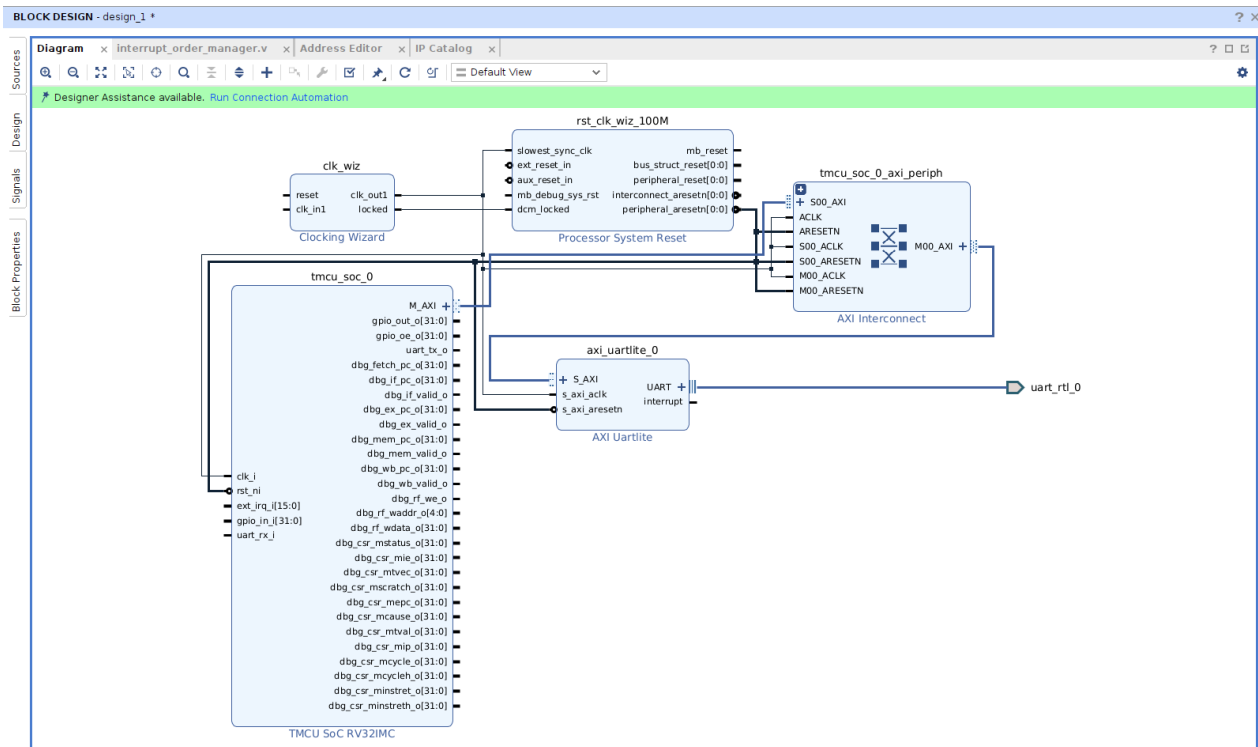


Figure 2.16: System infrastructure formed after the first Connection Automation.

2.6.2 Second Stage: Clock and Reset External Inputs

After the first automation, Vivado may again suggest Connection Automation for the external ports of the Clcking Wizard and the reset infrastructure.

Run Connection Automation is run again.

In the sample design:

- `clk_in1` and `reset` for `clk_wiz`,
- `ext_reset_in` for `rst_clk_wiz_100M`

are selected (Figure 2.17).

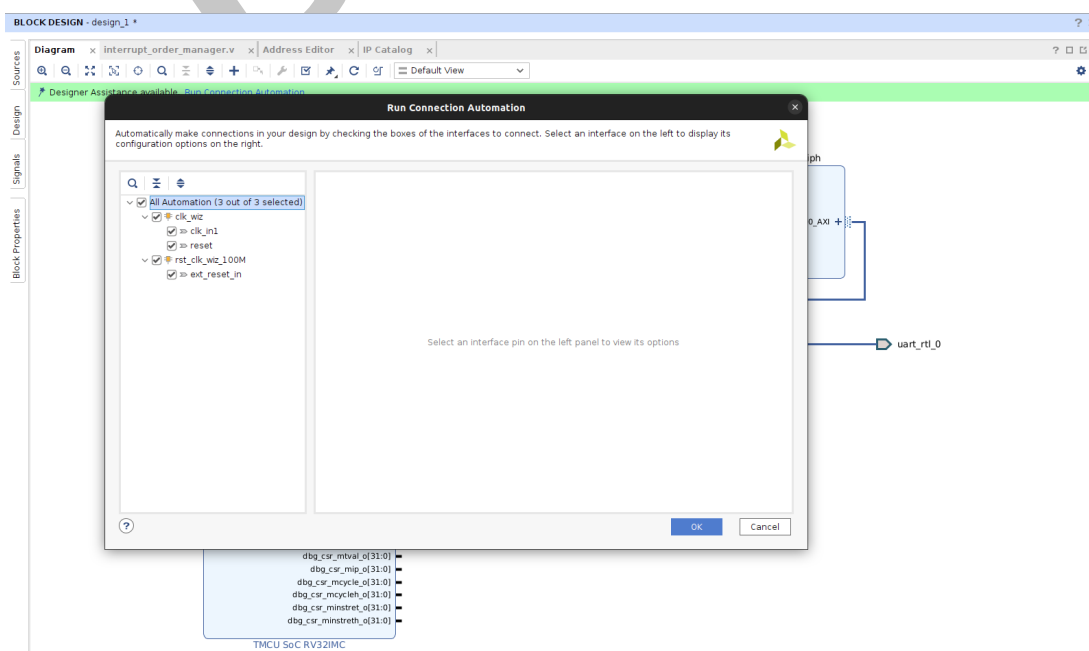


Figure 2.17: Second Connection Automation: clock and reset external connections.

After this stage, the following ports are seen at the top level, for example:

- `clk_100MHz`,
- `reset_rtl_0`

microBee's `clk_i` and `rst_ni` inputs are driven through this infrastructure (Figure 2.18).

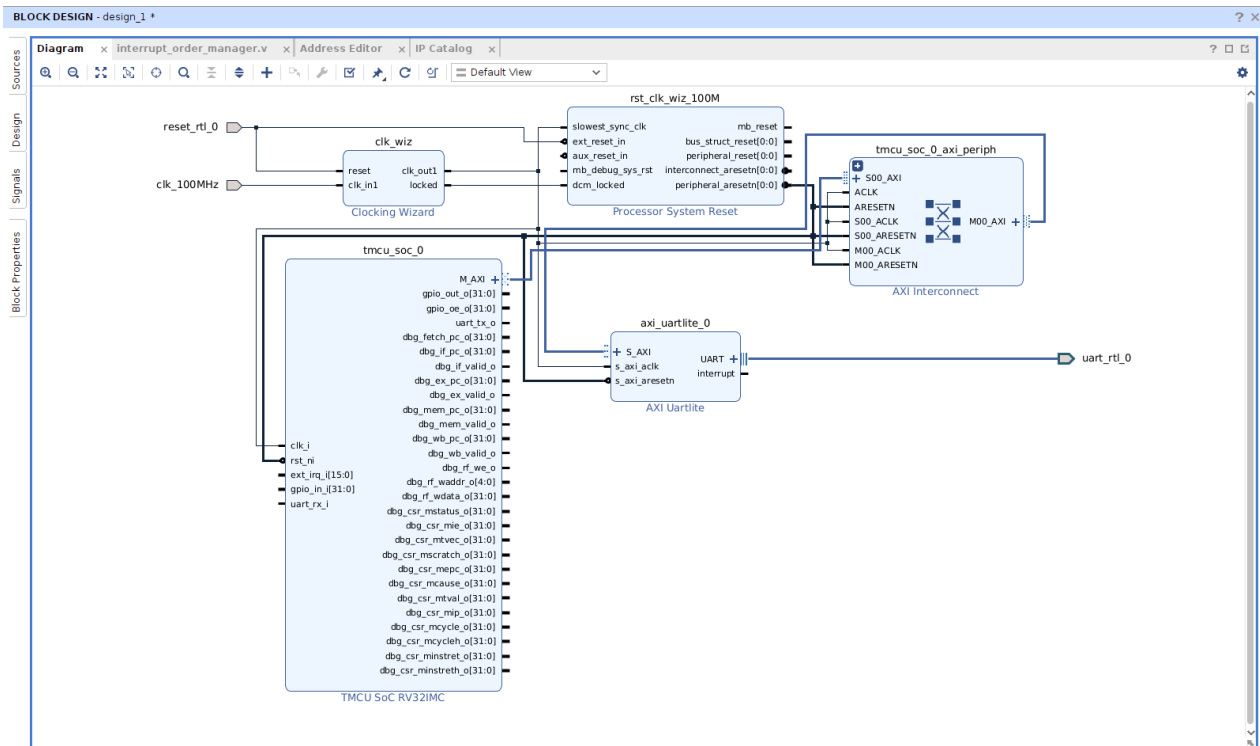


Figure 2.18: Block Design after clock and reset automation is completed.

In the Address Editor, the base address of AXI UART Lite must be selected within microBee's AXI address window.

In this laboratory example:

`0x60000000`

can be used.

The UART baud setting and the system clock must be compatible with each other.

The sample testbench used in this chapter was prepared according to the configuration:

`100 MHz / 9600 baud`

If a different clock or baud value is used, both the AXI UART Lite IP setting and the testbench must be updated according to the same value.

2.7 Forming the External Interrupt Vector

microBee expects a 16-bit vector at the external interrupt input:

`ext_irq_i[15:0]`

AXI UART Lite produces a single-bit:

`interrupt`

output.

To place this and similar single-bit interrupt sources onto microBee's external interrupt vector, the helper RTL module named:

`interrupt_order_manager`

can be used.

2.7.1 Purpose of the Module

The `interrupt_order_manager` module combines the 16 single-bit input signals of the form:

`in_0, in_1, ..., in_15`

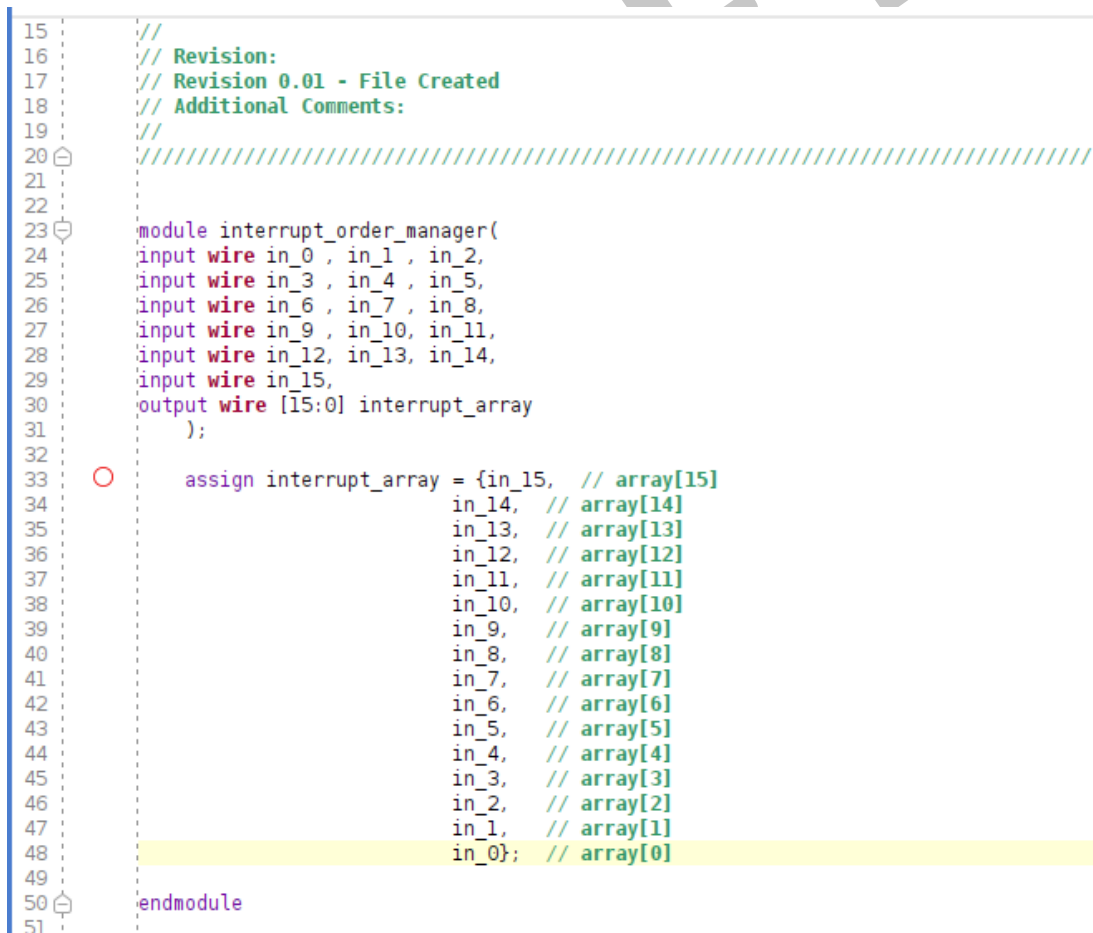
into the vector:

`interrupt_array[15:0]`

Here the bit order is preserved such that:

`in_0 → interrupt_array[0]`

The module source code must have been added into Design Sources (Figure 2.19).



```

15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module interrupt_order_manager(
24 input wire in_0 , in_1 , in_2,
25 input wire in_3 , in_4 , in_5,
26 input wire in_6 , in_7 , in_8,
27 input wire in_9 , in_10, in_11,
28 input wire in_12, in_13, in_14,
29 input wire in_15,
30 output wire [15:0] interrupt_array
31 );
32
33 assign interrupt_array = {in_15, // array[15]
34                          in_14, // array[14]
35                          in_13, // array[13]
36                          in_12, // array[12]
37                          in_11, // array[11]
38                          in_10, // array[10]
39                          in_9,  // array[9]
40                          in_8,  // array[8]
41                          in_7,  // array[7]
42                          in_6,  // array[6]
43                          in_5,  // array[5]
44                          in_4,  // array[4]
45                          in_3,  // array[3]
46                          in_2,  // array[2]
47                          in_1,  // array[1]
48                          in_0}; // array[0]
49
50 endmodule
51

```

Figure 2.19: `interrupt_order_manager`: combining 16 single-bit interrupt inputs into a 16-bit external interrupt vector.

2.7.2 Adding an RTL Module to the Block Design

`interrupt_order_manager` is not a Vivado IP; it is a project RTL source directly. Therefore, within the Block Design:

right-click → **Add Module...**

is used (Figure 2.20).

From the list that opens:

`interrupt_order_manager`

is selected (Figure 2.21).

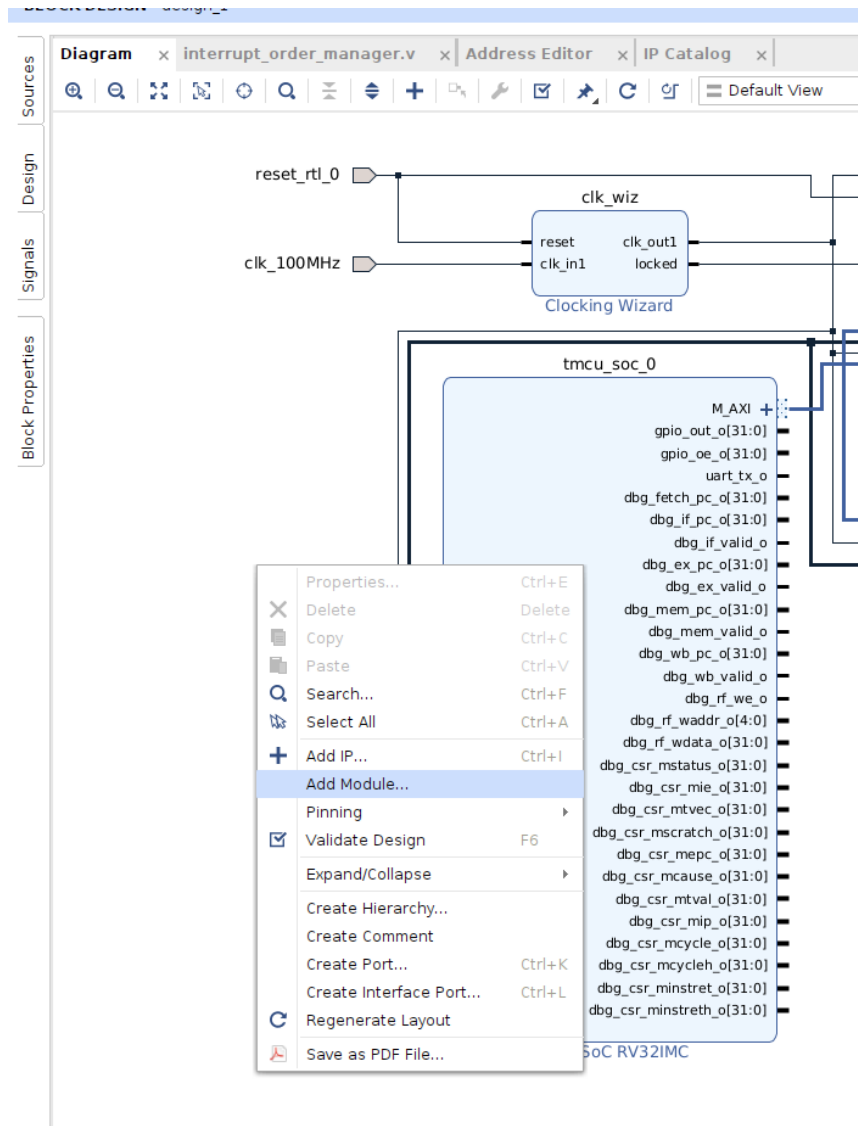


Figure 2.20: Add Module selection on the Block Design.

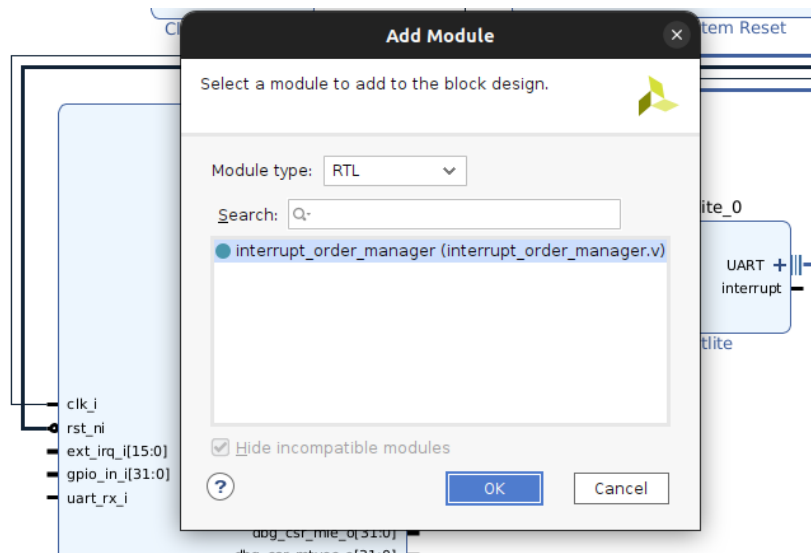


Figure 2.21: Selecting the `interrupt_order_manager` module in the Add Module window.

After the module is added, the connection:

`interrupt_array` → `ext_irq_i`

is made (Figure 2.22 and Figure 2.23).

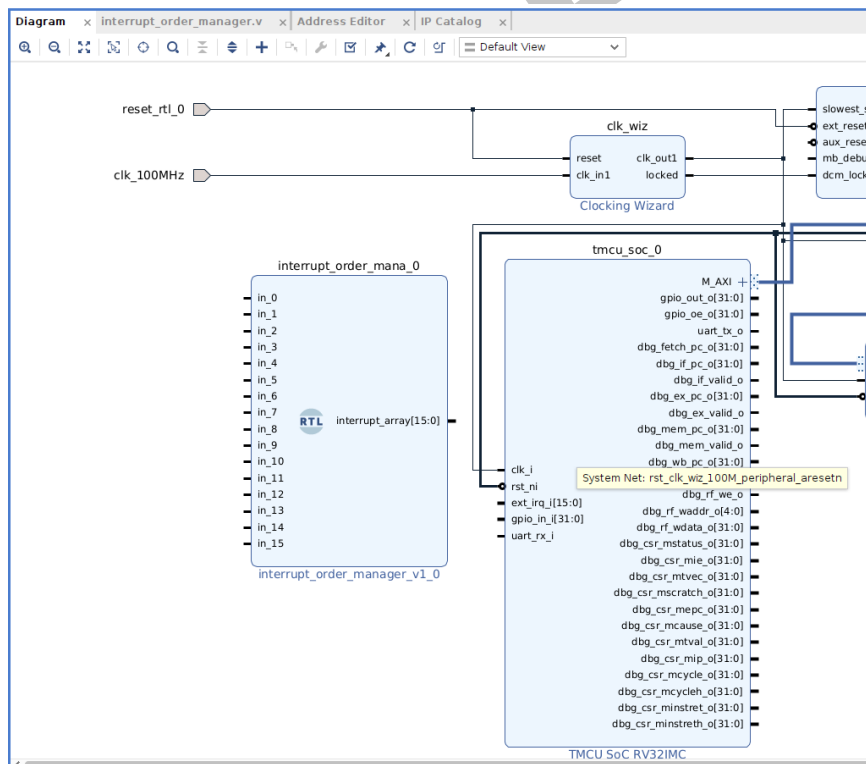


Figure 2.22: Adding the `interrupt_order_manager` module to the Block Design.

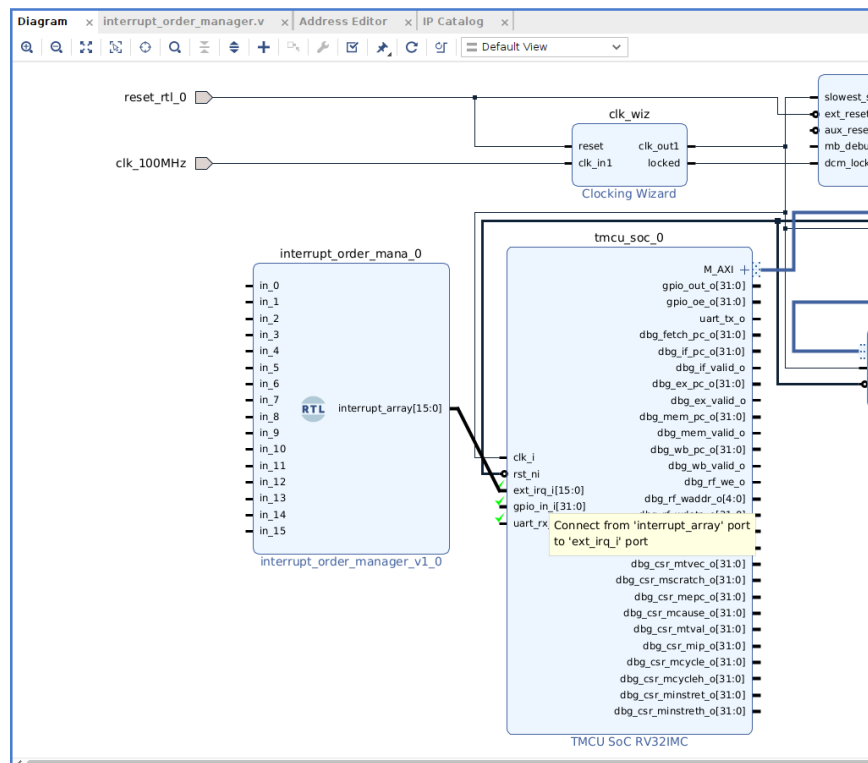


Figure 2.23: Connecting the `interrupt_array[15:0]` output to microBee's `ext_irq_i[15:0]` input.

The AXI UART Lite interrupt output is connected as:

`interrupt` → `in_0`

(Figure 2.24).

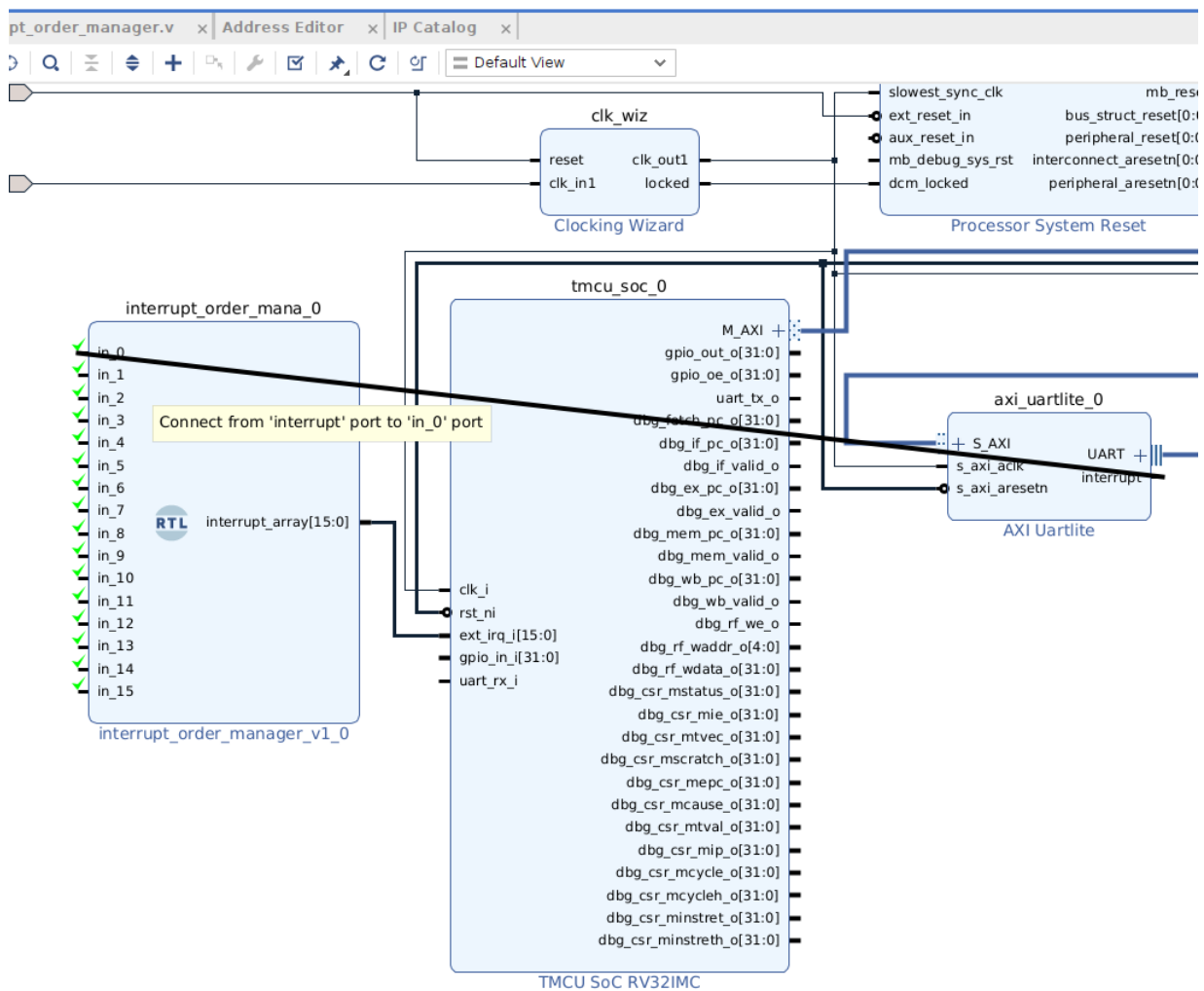


Figure 2.24: Connecting the AXI UART Lite `interrupt` output to the `in_0` input.

Thus the AXI UART Lite interrupt source is connected to the

`ext_irq_i[0]`

line and therefore to ID 1 within the microBee PLIC.

Unused `in_*` inputs must not be left floating (idle).

These inputs can be tied to 0 or connected with a Constant IP. Otherwise unknown X values can propagate into the external interrupt vector in behavioral simulation.

The final Block Design reaches the structure given in Figure 2.1.

2.8 IMEM / DMEM Initialization Files

The program and initial data can be placed into the memories with the IMEM and DMEM initialization files given in the microBee Customize IP window.

Within the Block Design, the **Re-customize IP** window is opened by double-clicking:

`tmcu_soc_0`

- **Imem Init File**

Example:

`verification/system/soc_integration_imem.hex`

(Figure 2.25 and Figure 2.26).

- **Dmem Init File**

Example:

verification/system/soc_integration_dmem_poll.hex

(Figure 2.27 and Figure 2.28).

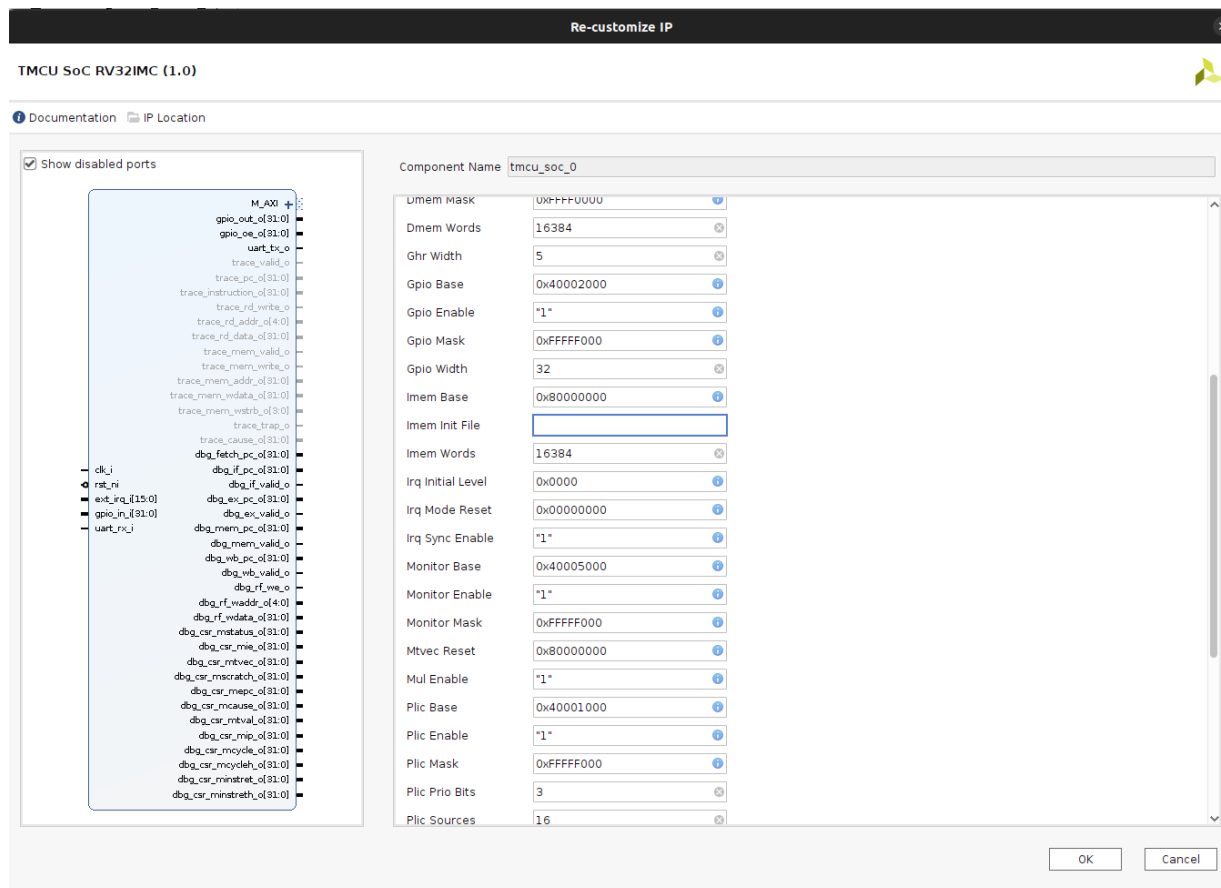


Figure 2.25: Memory and peripheral parameters together with the Imem Init File field in the Customize IP window.

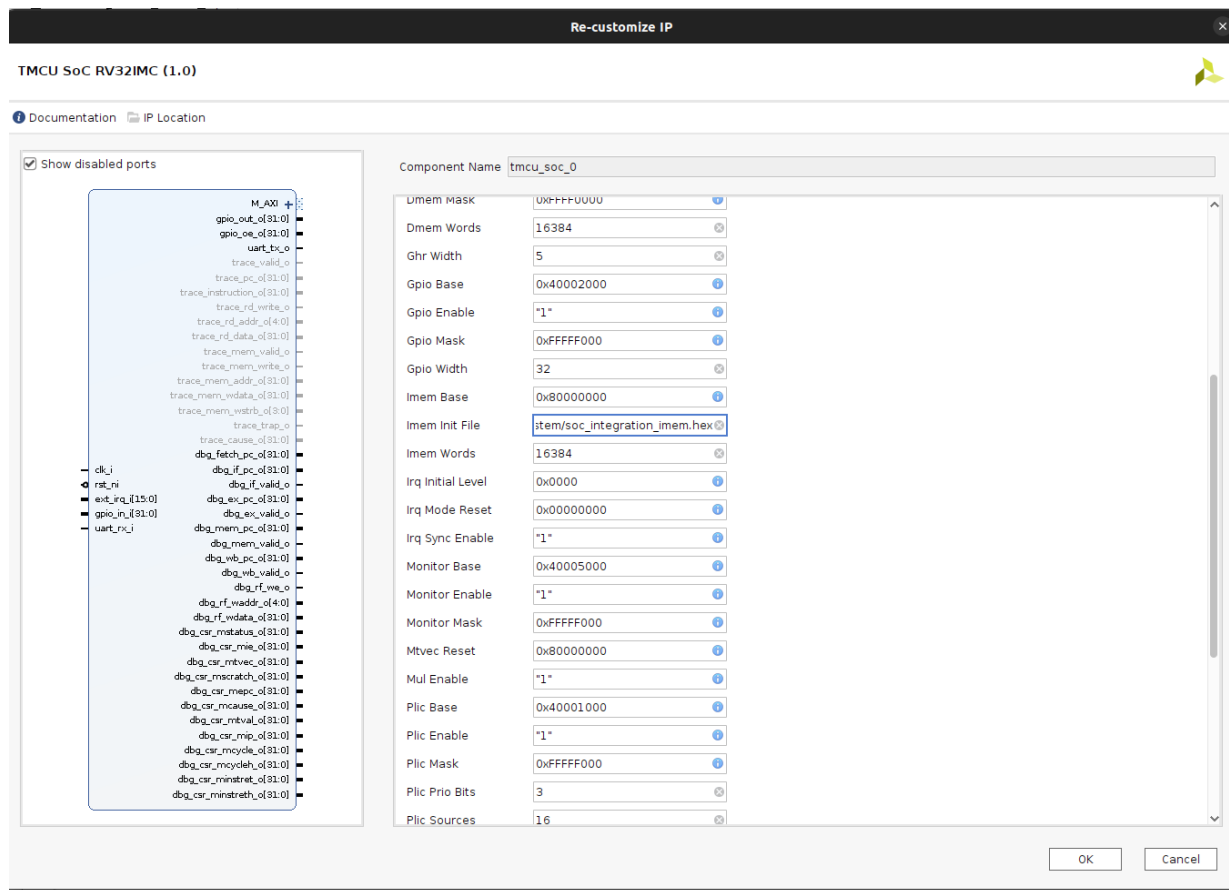


Figure 2.26: Entering the Imem Init File path.

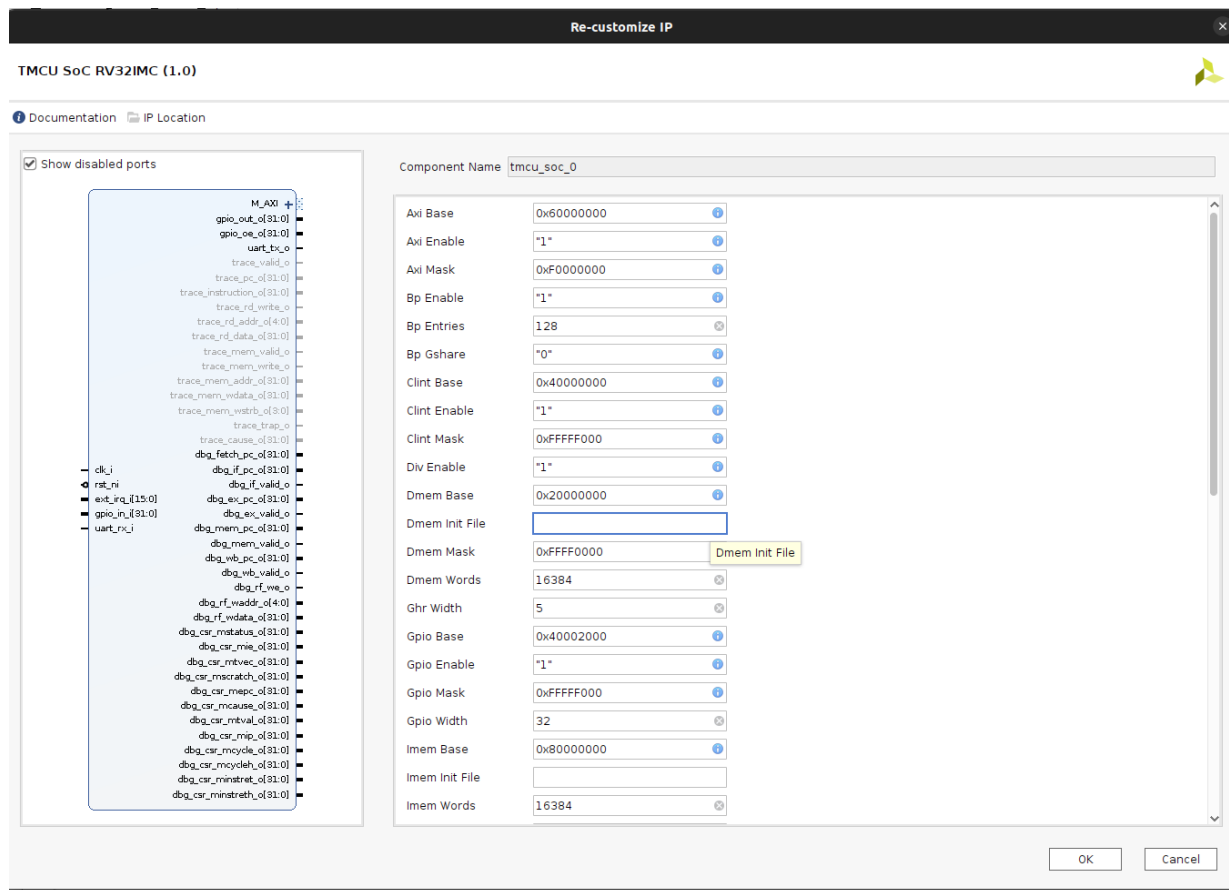


Figure 2.27: Dmem Init File and AXI address parameters in the Customize IP window.

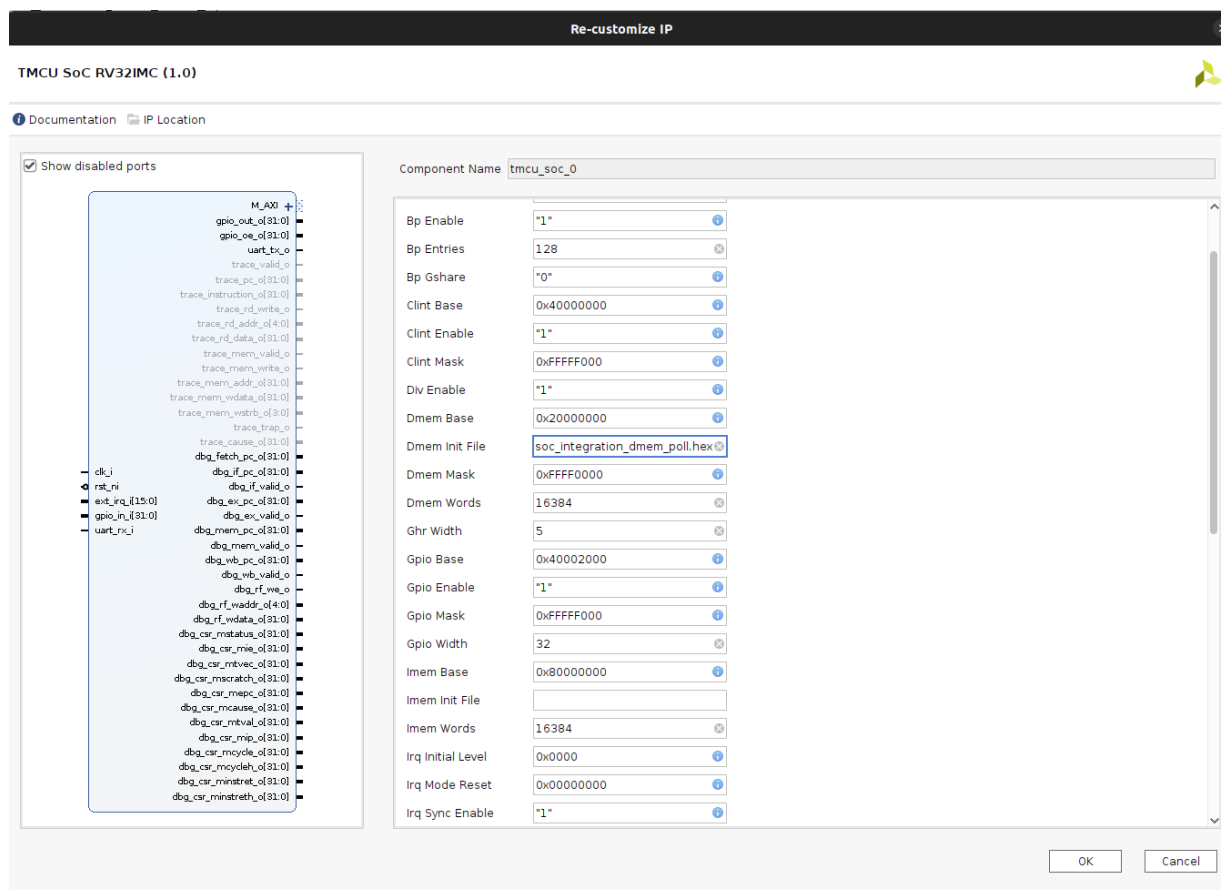


Figure 2.28: Dmem Init File example: soc_integration_dmem_poll.hex.

Note

The paths of the initialization files can be given as relative to Vivado's working directory or as absolute. If the initialization file cannot be found or cannot be loaded correctly, the program or data memory cannot be started with the expected content. In that case the expected UART output may not be obtained in simulation.

The AXI base address must match the address used for AXI UART Lite in the Address Editor. In this laboratory example:

0x60000000

is used.

2.9 HDL Wrapper

The Block Design is not used by itself as the project's top-level RTL module.

Vivado produces an HDL wrapper in order to carry the IP and interface connections inside the Block Design into the RTL hierarchy.

In the Sources window, a right-click is performed on:

design_1.bd

and:

Create HDL Wrapper...

is selected (Figure 2.29).

For the laboratory, the option:

Let Vivado manage wrapper and auto-update

can be used.

After that, the module:

`design_1_wrapper`

is seen under Design Sources (Figure 2.30).

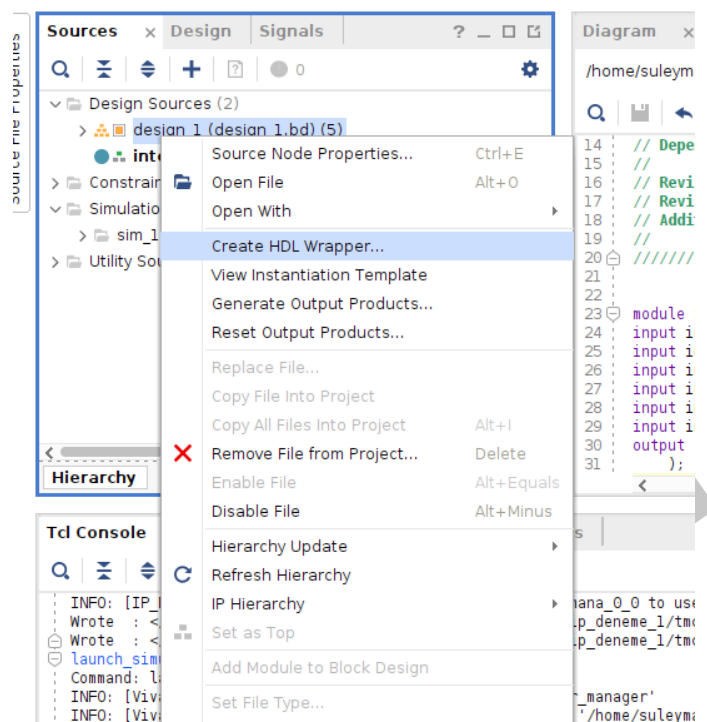


Figure 2.29: Create HDL Wrapper operation for `design_1.bd`.

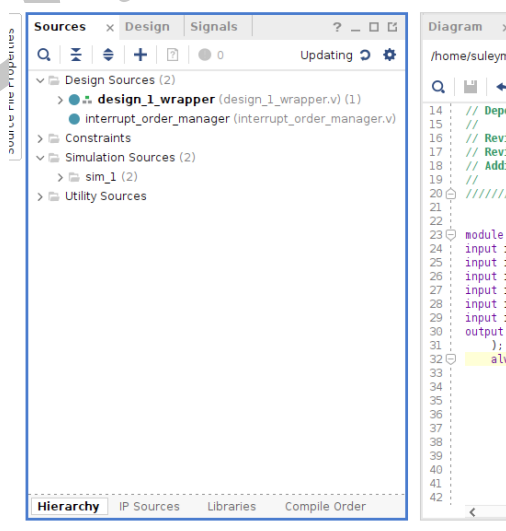


Figure 2.30: The `design_1_wrapper` module under Design Sources.

The wrapper's external ports are later driven or observed by the testbench.

Sample ports:

- `clk_100MHz`,

- `reset_rtl_0`,
- `uart_rtl_0_rxd`,
- `uart_rtl_0_txd`.

2.10 Simulation System

2.10.1 What Is It?

Simulation and emulation tools examine program and system behavior at different levels of abstraction. In this laboratory chapter the main topic is Vivado xsim behavioral simulation. `rv32`, on the other hand, is used to assemble the assembly source and to observe register, data memory and instruction effects step by step; its setup and student applications are described in Chapter 11.

Vivado's xsim simulator runs the functional model prior to synthesis and physical implementation during behavioral simulation.

The testbench:

- generates a clock,
- applies reset,
- instantiates the `design_1_wrapper` module as the DUT,
- samples the serial data on the UART TX line according to the baud timing,
- compares the received characters with the expected ASCII sequence.

The prepared testbench example can be used as:

```
verification/system/tb_design_1_hello.sv
```

In this laboratory testbench the expected message is:

```
"Hello from microBee on Arty A7\r\n"
```

This example verifies the 28-character message with a 100 MHz system clock and 9600 baud configuration.

2.10.2 Adding the Testbench

1. **Add Sources** is selected.
2. **Add or create simulation sources** is selected (Figure 2.31).
3. The `tb_design_1_hello.sv` file is added to the project or is created with **Create File** (Figure 2.32 and Figure 2.33).
4. The testbench port connections are made compatible with the wrapper ports used (Figure 2.34).

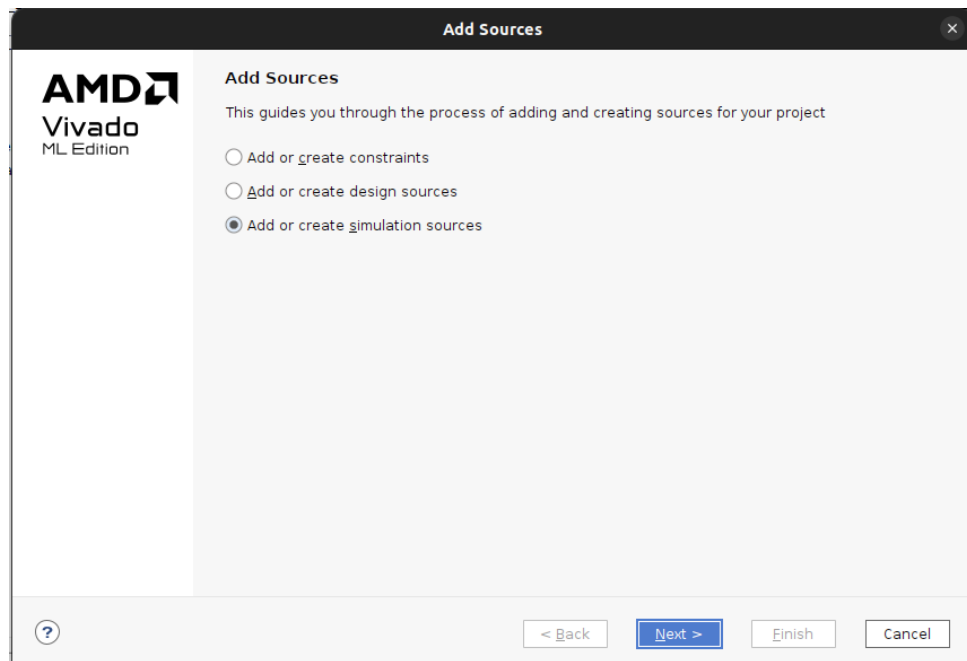
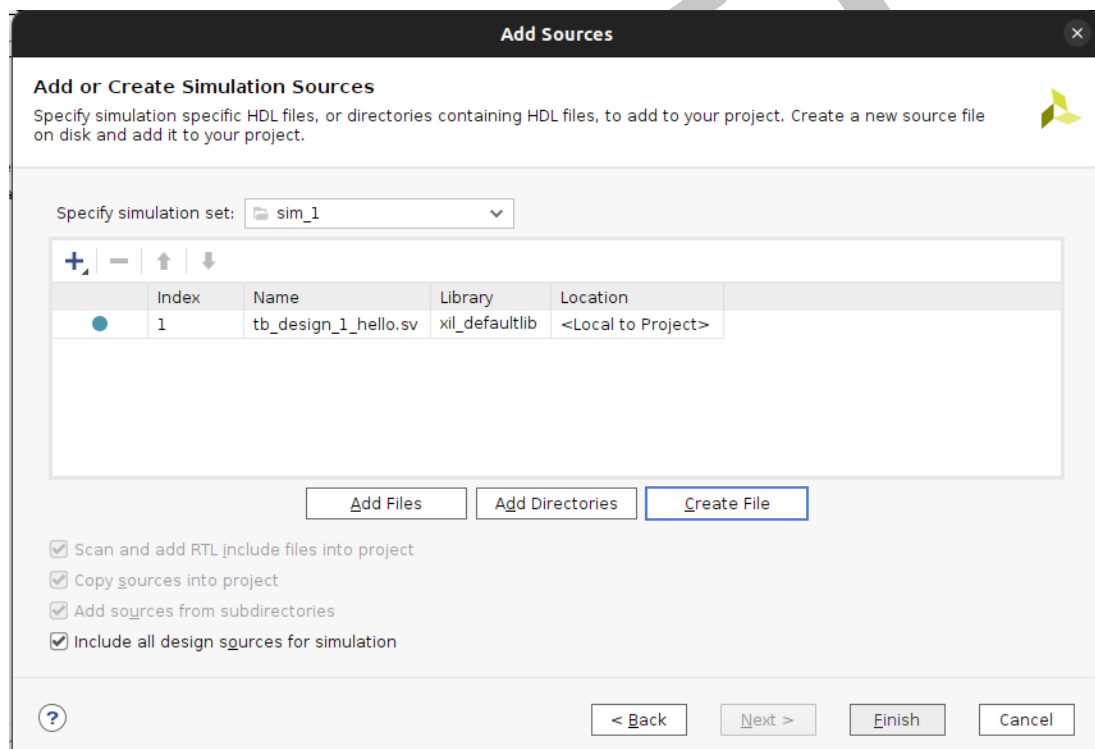


Figure 2.31: Simulation source selection within Add Sources.

Figure 2.32: Adding the `tb_design_1_hello.sv` file as a simulation source.

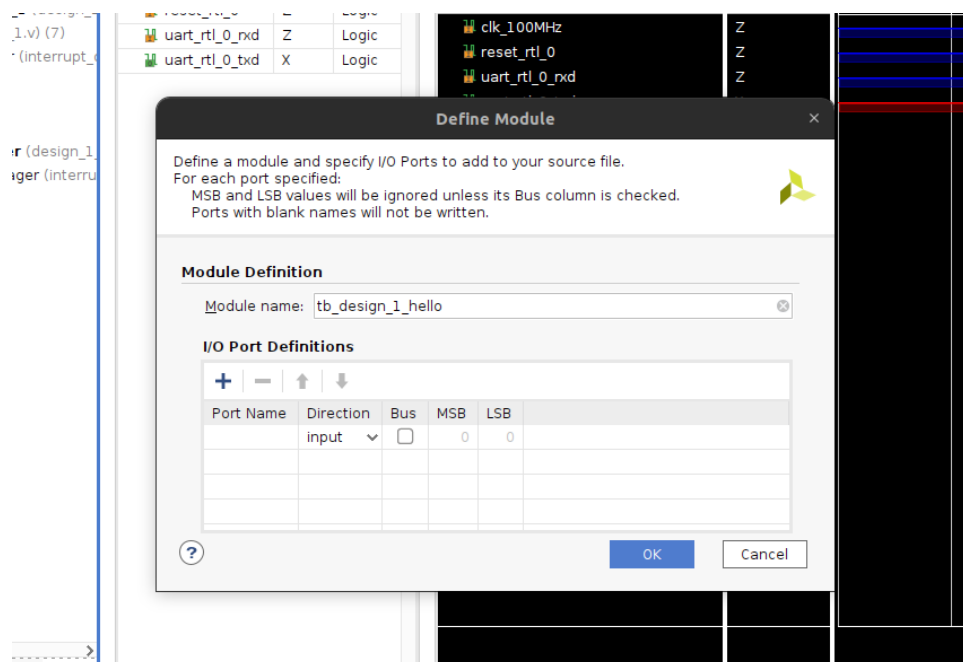


Figure 2.33: Creating the testbench module on the Define Module screen.

```

interrupt_order_manager.v x tb_design_1_hello.sv x Untitled 2 x
/home/suleymanonsay/Desktop/Projeler/TMCU/tmcu_ip_deneme_1/tmcu_ip_deneme_1.srscs/sim_1/new/tb_design_1_hello.sv

1 timescale 1ns/1ps
2
3 // Behavioral TB for student Vivado BD wrapper `design_1_wrapper`.
4 //
5 // Ports:
6 //   clk_100MHz, reset_rtl_0 (ACTIVE_HIGH), uart_rtl_0_rxd, uart_rtl_0_txd
7 //
8 // IMEM/DMEM init on tmcu_soc (Customize IP):
9 //   verification/system/soc_integration_imem.hex
10 //   verification/system/soc_integration_dmem_poll.hex
11 //
12 // UART Lite: 9600 baud @ 100 MHz ACLK
13 // Expected: "Hello from TMCU on Arty A7\r\n" (28 bytes)
14 //
15 // Vivado: Simulation top = tb_design_1_hello
16 // Prefer file type SystemVerilog; written to also parse cleanly as Verilog-style SV.
17
18 module tb_design_1_hello;
19   localparam integer CLK_HZ      = 100000000;
20   localparam integer BAUD       = 9600;
21   localparam integer BIT_TICKS  = CLK_HZ / BAUD;
22   localparam integer HALF_BIT   = BIT_TICKS / 2;
23   localparam integer EXPECT_CHARS = 28;
24   localparam integer RESET_CYCLES = 20000;
25   localparam integer TIMEOUT_MS  = 200;
26
27   // ASCII: "Hello from TMCU on Arty A7\r\n"
28   reg [7:0] HELLO [0:27];
29
30   reg clk_100MHz = 1'b0;
31   reg reset_rtl_0 = 1'b1; // ACTIVE_HIGH
32   reg uart_rxd    = 1'b1;
33   wire uart_txd;
34
35   integer i;
36
37   initial begin
38     HELLO[ 0] = 8'h48; HELLO[ 1] = 8'h65; HELLO[ 2] = 8'h6C; HELLO[ 3] = 8'h6C;
39     HELLO[ 4] = 8'h6F; HELLO[ 5] = 8'h20; HELLO[ 6] = 8'h66; HELLO[ 7] = 8'h72;
40     HELLO[ 8] = 8'h6F; HELLO[ 9] = 8'h6D; HELLO[10] = 8'h20; HELLO[11] = 8'h54;

```

Figure 2.34: Defining clock, reset, UART and the expected “Hello” message in the testbench source.

2.10.3 Setting the Simulation Top

In this laboratory flow, the wrapper inside Design Sources is the top-level module for synthesis and implementation.

In behavioral simulation, however, the top-level module must be the testbench.

Under Simulation Sources, a right-click is performed on:

```
tb_design_1_hello
```

and:

Set as Top

is selected (Figure 2.35).

In the correct hierarchy the testbench becomes the top module of the simulation (Figure 2.36).

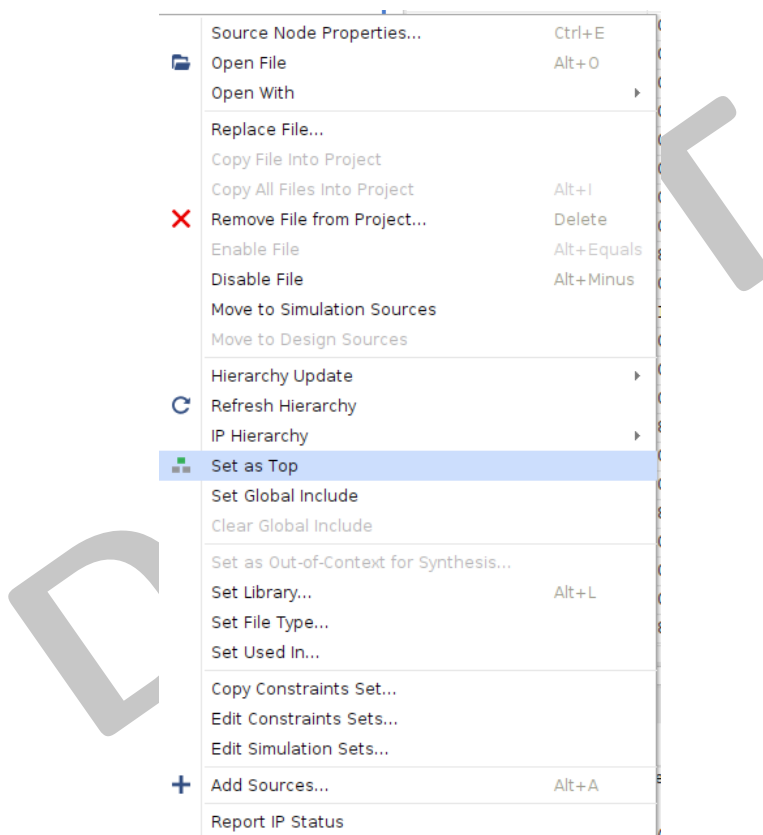


Figure 2.35: Set as Top selection within Simulation Sources.

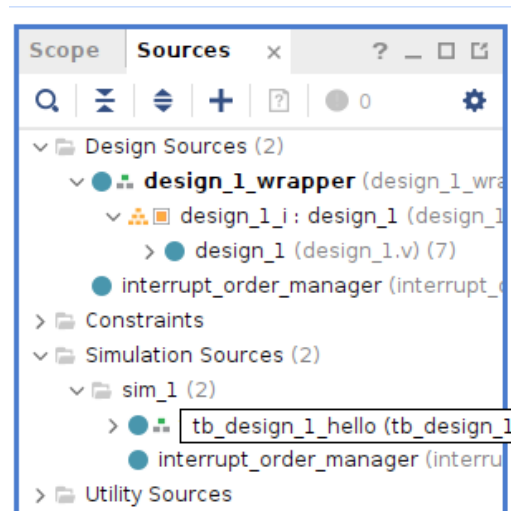


Figure 2.36: Selecting the `tb_design_1_hello` module as the simulation top.

Then:

Flow Navigator → **Run Simulation** → **Run Behavioral Simulation**

is selected (Figure 2.37).

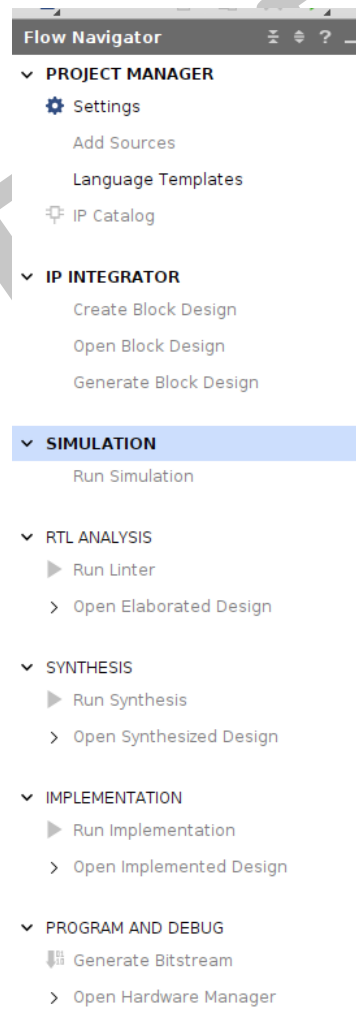


Figure 2.37: Run Simulation within the Flow Navigator.

2.10.4 Limitations

Behavioral simulation is a powerful functional verification tool but it does not represent the entirety of physical FPGA behavior.

- Behavioral simulation does not model the actual FPGA delays that arise after placement and routing. It primarily verifies functional behavior.
- If the UART baud value and the bit duration used by the testbench are not compatible, characters may be sampled incorrectly and the test may conclude with **FAIL**.
- If the IMEM/DMEM initialization files are not loaded correctly, the program may not run as expected.
- If the AXI UART Lite base address does not match the address in the software, UART accesses do not reach the correct peripheral.
- When `DEBUG_ENABLE=1`, the `dbg_*` ports can be used on the waveform. Enabling the debug feature may increase synthesis resource usage because of additional observation logic, but it should not change the architectural behavior of the program.
- This flow is different from the Verilator-based microBee verification flow. Here a behavioral verification that includes the FPGA integration structure is performed through the Vivado Block Design and xsim.
- The Pipeline view of rv32 is for teaching purposes; it does not implement microBee's hazard, stall, forwarding and physical timing model. The emulator's memory initializations and peripheral registers are also not identical to the REF32 map. Comparison across environments is given in Section 11.10.

2.11 Monitoring Simulation Outputs in Vivado

2.11.1 Waveform

After behavioral simulation starts, at least the following can be observed in the Waveform window:

- `clk_100MHz`,
- `reset_rtl_0`,
- the UART TX signal,
- `char_byte` created within the testbench,
- the expected **HELLO** sequence

After reset is released, the testbench decodes the UART TX line.

In correct operation `char_byte` successively takes the values:

`0x48, 0x65, 0x6C, 0x6C, 0x6F, ...`

The ASCII counterpart of these values forms the message:

`Hello...`

Messages produced by the testbench such as:

TB: reset released

and the UART output can also be seen in the Tcl Console (Figure 2.38).

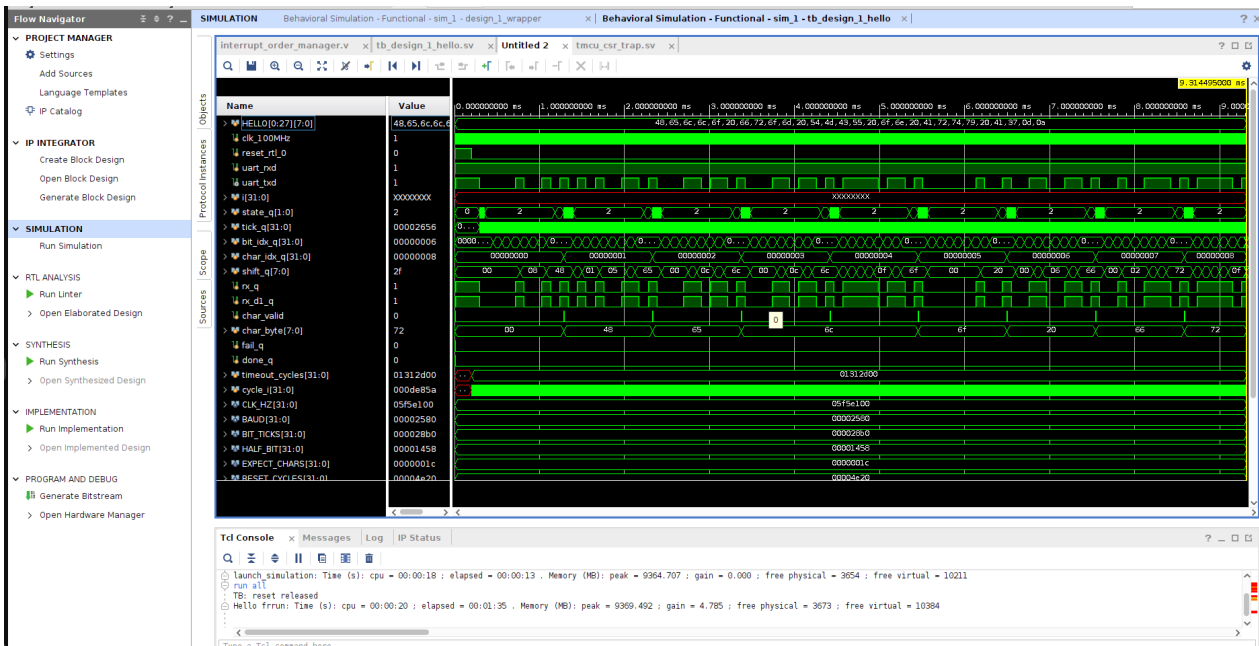


Figure 2.38: Receiving “Hello” characters over UART on the behavioral simulation waveform.

When the end of the simulation is reached, if the testbench has correctly received all 28 expected characters, both the UART text and the success message are written on the console (Figure 2.39):

```

Hello from microBee on Arty A7
PASS: tb_design_1_hello

```

This output shows that the IMEM/DMEM init files were loaded, that microBee ran the program, and that the AXI UART Lite → `uart_rtl_0_txd` path is compatible with the testbench receiver.

On a character mismatch or timeout the FAIL line is seen; in that case the baud/clock match, the Address Editor base and the init paths should be checked again.

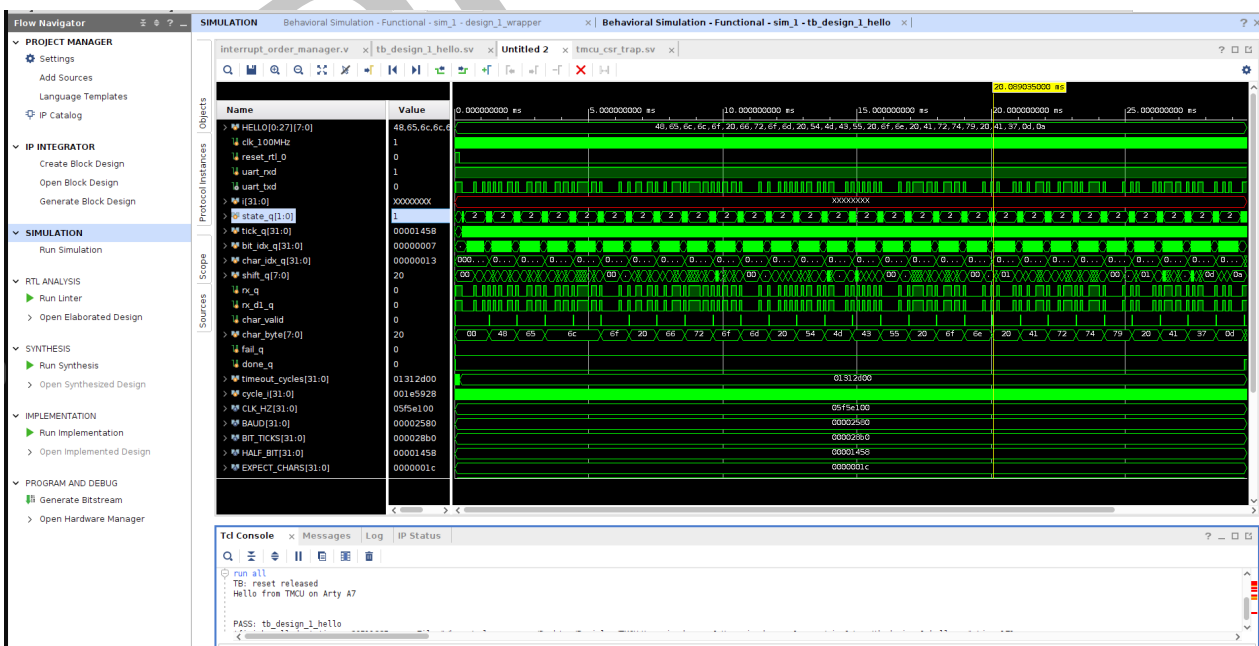


Figure 2.39: Tcd Console: UART “Hello” output and the PASS: `tb_design_1_hello` message.

2.11.2 Debug

In Block Design simulation, microBee debug ports can be used to observe micro-architectural behavior without entering the core's inner RTL hierarchy.

When `DEBUG_ENABLE=1`, the `dbg_*` outputs on `tmcu_soc_0` can be used.

Through these ports, for example:

- PC values in the pipeline stages,
- pipeline valid information,
- register file write enable,
- register file write address,
- register file write data,
- some CSR states

can be observed on the waveform.

The purpose of the debug ports is not to change normal program behavior, but to make the core's inner behavior observable from the outside.

The detailed `dbg_*` port list is given in the Debug Ports section.

The use of ILA or `MARK_DEBUG` on FPGA is not a mandatory part of this introductory laboratory.

In the first stage it is sufficient to verify the UART output in `xsim` and, when necessary, to examine the `dbg_*` ports on the waveform.

2.12 Short Checklist

At the end of the laboratory the following items should be checked:

1. Is the microBee IP repository path correct?

`work/fpga_synth/ip/tmcu_soc`

2. Are microBee and AXI UART Lite present in the Block Design?
3. Were the AXI, clock and reset connections completed with Connection Automation?
4. Is AXI UART Lite within microBee's AXI window in the Address Editor?
5. If the laboratory testbench is used, are the clock and baud settings compatible with 100 MHz / 9600?
6. If an external interrupt will be used, was the AXI UART Lite interrupt output connected to the `ext_irq_i[0]` line through `interrupt_order_manager`?
7. Were unused external interrupt inputs tied to 0?
8. Are the paths of the IMEM and DMEM initialization files correct?
9. Was the HDL wrapper created?
10. Was `tb_design_1_hello` selected as the simulation top?
11. When behavioral simulation is run, is the expected "Hello" message received over UART?
12. If necessary, was `DEBUG_ENABLE=1` set and were the `dbg_*` signals verified on the waveform?

Seeing the expected "Hello" message together with `PASS: tb_design_1_hello` on the waveform (Figure 2.38) or in the simulation console (Figure 2.39) is an important first verification showing that microBee could fetch instructions from program memory, that the software ran, and that the basic AXI UART Lite communication path was set up correctly.

This test does not mean that all peripheral and micro-architectural features in the system have been verified completely. More detailed verifications should be performed in later laboratories using monitoring, interrupt, GPIO and debug mechanisms.

Chapter 3

Architecture

3.1 Computer Architecture

3.1.1 What Is Architecture?

Computer architecture is the framework that defines the design of a computer system and the fundamental choices made in that design. Which choices are made in areas such as ISA, micro-architecture, memory structure, pipeline, bus and I/O, and which trade-offs are accepted, fall within the scope of architecture.

3.1.2 What Does Architecture Affect?

Architecture largely shapes what the system can do and metrics such as performance, area and power. At the same time these metrics do not depend only on architecture; factors such as implementation, manufacturing technology, compiler, memory hierarchy and workload also determine the result. Software and toolchain support are also affected by architectural choices. For example, a GPU's processing-unit layout can significantly affect LLM inference speed; however the final speed also depends on the software stack and runtime layers. Architecture determines the system's fundamental limits and opportunities.

3.1.3 Why Should You Know Architecture?

Knowing architecture is indispensable for an embedded system designer. Each of the thousands of kinds of controllers on the market has its own distinct features. Some controllers may have been designed for high speed, while others may have been designed for high I/O capacity or low power. Knowing these differences enables the designer to use the limits of the architecture in the chosen controller to the fullest.

3.1.3.1 Example:

If the system is superscalar, it aims at parallel execution by issuing multiple suitable independent instructions within the same cycle. This does not mean only having more than one functional unit (for example two adders); the critical point is that instructions without a dependency can be started in the same cycle. When a dependency, hazard or resource conflict exists, producing two results every cycle is not guaranteed. If the designer does not know this, they may unnecessarily serialize operations that contain dependencies and miss the opportunity for parallel execution.

General computer architecture concepts have been summarized up to this section. The following parts make the same concepts concrete through the microBee SoC's high-level block diagram, its pipeline and its memory paths.

3.2 Overall Architecture

3.2.1 Top

At the top level, microBee is a Harvard-architecture SoC. Instruction memory (IMEM) and data memory (DMEM) are separate from each other. In this way instruction fetch operations and load/store operations can be performed over different memory paths.

The core uses a five-stage pipeline:

$$\text{IF} \rightarrow \text{ID} \rightarrow \text{EX} \rightarrow \text{MEM} \rightarrow \text{WB}$$

RV32IMC instructions are processed within the pipeline.

The instruction fetch side accesses IMEM directly over the native IMEM request/response interface. Load/store and MMIO requests on the data side are sent to the `tmcu_request_router` block over the native valid/ready interface.

The request router determines to which target the data request will be directed according to the access address. In the REF32 profile the basic routing order is:

$$\text{CLINT} \rightarrow \text{PLIC} \rightarrow \text{GPIO} \rightarrow \text{TIMER} \rightarrow \text{UART} \rightarrow \text{MONITOR} \rightarrow \text{DMEM} \rightarrow \text{AXI} \rightarrow \text{Error}$$

For this reason a data request can be directed to:

- CLINT,
- PLIC,
- GPIO,
- Native Timer,
- Native UART,
- the Monitoring unit,
- DMEM,
- or the AXI address window

The microBee core does not use the AXI protocol directly. Native requests directed to the AXI address window are converted into AXI4-Lite transactions by the `tmcu_native_to_axi4lite` bridge.

In the REF32 profile the AXI window starts from the address:

0x60000000

Thanks to this structure, while the core and native peripherals use their own simple valid/ready interfaces, AXI-based IPs added into the FPGA Block Design can also be accessed.

Figure 3.1 shows the main blocks within `tmcu_soc`. In the figure the five-stage pipeline, IMEM and DMEM, request router, CLINT, PLIC, GPIO, Native Timer, Native UART, the Monitoring unit, the AXI4-Lite bridge and the interrupt structure are shown together.

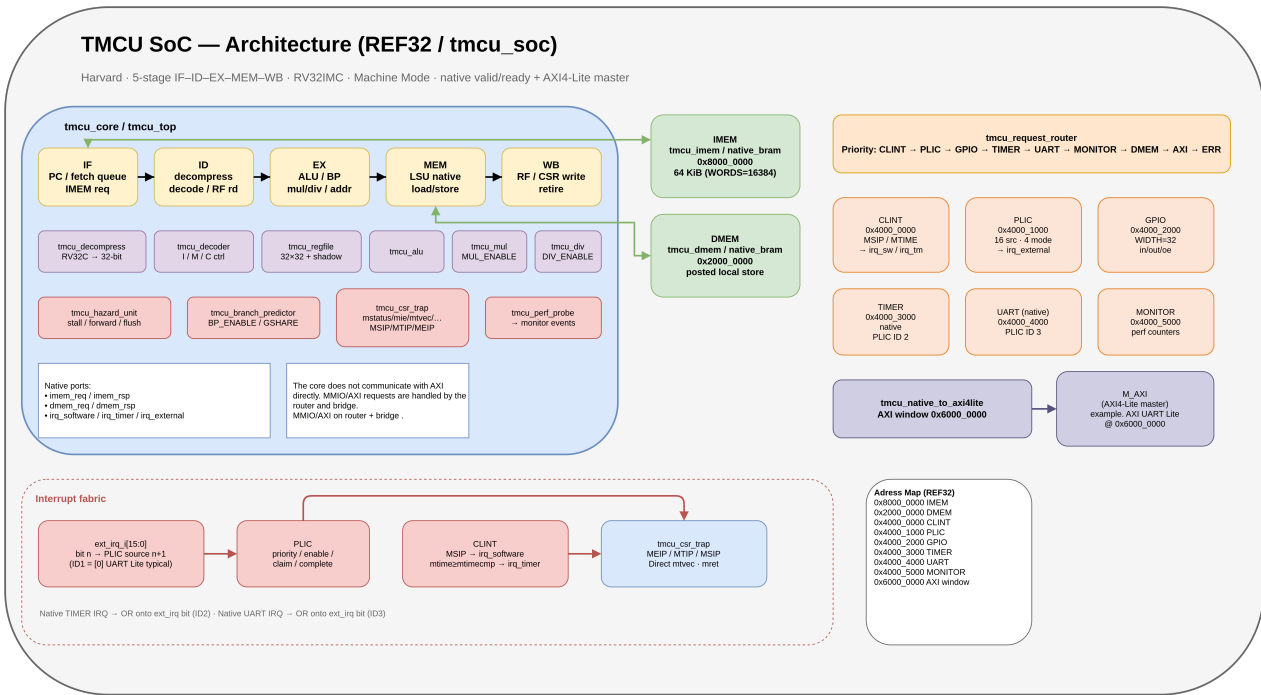


Figure 3.1: microBee SoC high-level architecture (tmcu_soc, REF32).

3.2.2 I/O

The ports that provide microBee's connection with the outside world are at the tmcu_soc level.

The visibility and use of the ports depend on some configuration parameters. In particular `DEBUG_ENABLE`, `TRACE_ENABLE` and `AXI_ENABLE` affect the use of the related port groups.

The basic I/O groups are as follows.

- **Clock and Reset**

`clk_i` is microBee's main clock input.

`rst_ni` is the active-low reset input.

Within the FPGA Block Design these signals are generally not connected to microBee directly from the board pins. While the input clock is processed through the Clocking Wizard, reset generation and synchronization are generally performed by the Processor System Reset (`proc_sys_reset`) block.

- **External Interrupt**

microBee uses the input:

`ext_irq_i[15:0]`

for external interrupt sources.

The relationship between the bit and the PLIC source ID is:

`ext_irq_i[n] → PLIC source ID  $n + 1$`

Accordingly:

`ext_irq_i[0] → PLIC ID 1`

Within the FPGA Block Design the AXI UART Lite interrupt output is typically connected to the `ext_irq_i[0]` line and is used as PLIC ID 1.

Native Timer and Native UART interrupt sources are included in the PLIC inputs within the SoC. In the REF32 profile they are used as:

- Native Timer → PLIC ID 2,
- Native UART → PLIC ID 3

Because these sources can be combined with the related external interrupt lines within the SoC, care must be taken when designing the connection of another source that uses the same PLIC ID.

- **GPIO**

The GPIO interface consists of three basic port groups:

- `gpio_in_i`: carries physical GPIO input values to microBee.

- `gpio_out_o`: outputs the GPIO output data.
 - `gpio_oe_o`: determines whether the related GPIO line will be driven as an output.
- In the REF32 profile:

`GPIO_WIDTH = 32`

is used.

When `gpio_oe_o=1` the related GPIO line is driven as an output; when it is 0 the output driver is disabled.

• Native UART

The Native UART found directly in microBee RTL uses the ports:

`uart_rx_i      uart_tx_o`

The Native UART's MMIO base address is defined in the REF32 profile as:

`0x40004000`

Native UART and the AXI UART Lite added into the Vivado Block Design are different peripherals.

AXI UART Lite is not part of microBee RTL. It is added as a separate AXI IP within the FPGA Block Design and is accessed through microBee's `M_AXI` interface.

Within the Block Design the external RX/TX interface is generally seen with the name:

`uart_rtl_0`

• AXI4-Lite Master

`M_AXI` is the master interface that enables microBee to access AXI4-Lite peripherals on the FPGA.

This interface consists of the channels:

- AW: write address,
- W: write data,
- B: write response,
- AR: read address,
- R: read data

The microBee core does not produce these AXI signals directly. When the native data request is directed to the AXI window through the request router, the AXI bridge forms the required AXI4-Lite operation.

In the REF32 profile the AXI address window is in the range:

`0x60000000–0x6FFFFFFF`

For example AXI UART Lite can be placed at the address:

`0x60000000`

When `AXI_ENABLE=0` the AXI access path is disabled and the `M_AXI` interface is not used actively.

• Debug Ports

When `DEBUG_ENABLE=1`, microBee outputs various micro-architectural information through the `dbg_*` ports.

Among these ports are, for example:

- pipeline PC values,
- pipeline valid signals,
- register file write enable,
- register file write address,
- register file write data,
- CSR snapshot information

Debug ports are used not to change normal program execution, but to observe the core's inner state during waveform and simulation.

When `DEBUG_ENABLE=0` the debug outputs are not used for active observation; depending on the IP packaging setting they may be hidden at the Block Design level or pulled to constant values.

• Trace Ports

When `TRACE_ENABLE=1`, instruction commit information can be exported through the `trace_*` ports.

The trace interface enables tracking of commit-level events such as, for example:

- retired instruction PC,
- instruction value,
- register write information,
- memory access information,
- trap and cause information

While debug ports are used more for instantaneous micro-architectural observation, trace ports are used to follow the architectural event flow at the instruction level.

Simulation-specific ports.

`tb_dmem_we_i`, `tb_dmem_widx_i` and `tb_dmem_wdata_i` are testbench ports used only for simulation purposes.

These ports can be used by the testbench for directly preparing or modifying DMEM content.

These ports are not present in the normal Block Design interface of the packaged microBee FPGA IP.

Figure 3.2 shows the basic I/O groups brought out at the `tmcu_soc` level.

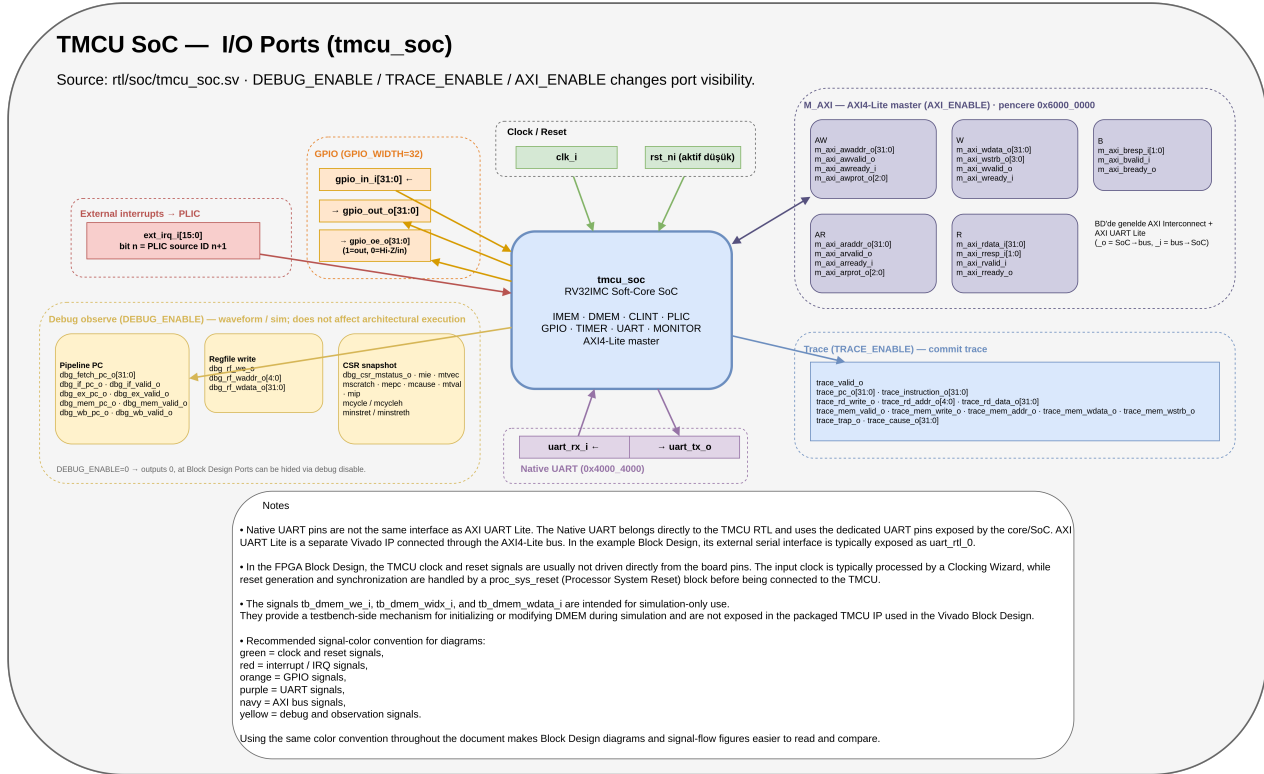


Figure 3.2: microBee SoC external I/O ports and functional groups (`tmcu_soc`).

3.2.3 Pipeline

microBee's core pipeline consists of five stages. These stages are Fetch, Decode, Execute, Memory and Write-back, respectively.

IF → IF/ID → ID → ID/EX → EX → EX/MEM → MEM → MEM/WB → WB

Structures such as IF/ID and ID/EX are pipeline registers. They are not separate stages; they denote the transition registers between stages.

3.2.3.1 Fetch

Fetch manages taking the next instruction from memory. A request is sent to instruction memory with the program counter (PC); incoming words are held in the fetch queue and, if necessary, are prepared for the next stage with 16/32-bit alignment.

3.2.3.2 Decode

Decode interprets the taken instruction and the source registers (`rs1/rs2`) are read from the register file. The outputs are control signals and operands.

3.2.3.3 Execute

Execute performs the instruction's actual computation. ALU operations, the branch/jump decision, multiply/-divide (if enabled) and the memory address for load/store are produced here. The result is transferred to the next stage.

3.2.3.4 Memory

Memory executes the data-path access. The load/store request, byte lane selection and waiting for the response are in this stage. If there is no memory operation the stage forwards the results as they are.

3.2.3.5 Writeback

Writeback updates architectural state. Writing to the register file and, if necessary, to a CSR is completed on this clock edge; the instruction is counted as retired and tracing/performance events are produced from here. Instructions that do not write to the register file, such as store, can also be considered architecturally completed in this stage.

3.3 Register File

There are 32 32-bit registers within microBee. These 32 registers collectively form the register file. While the processor is running it holds operand and result values here. For example, in an addition operation for $A + B = C$ the operands and the result are frequently held in registers; however this is only a typical example. An operand can also come through an immediate, a forwarding path or memory. The processor decodes the addresses it will read and write according to the information in the incoming instruction and reads from and writes to the register file. `x0` is always read as zero and writes to it are ignored.

Table 3.1: RISC-V integer registers [1]

Register	ABI name	Conventional use	Hardware behavior
<code>x0</code>	zero	Constant zero	Read is 0; write is ignored.
<code>x1</code>	ra	Return address	General-purpose storage.
<code>x2</code>	sp	Stack pointer	General-purpose storage.
<code>x3</code>	gp	Global pointer	General-purpose storage.
<code>x4</code>	tp	Thread pointer	General-purpose storage.
<code>x5-x7</code>	t0-t2	Temporary	General-purpose storage.
<code>x8-x9</code>	s0-s1	Saved/frame pointer	General-purpose storage.
<code>x10-x17</code>	a0-a7	Argument/return	General-purpose storage.
<code>x18-x27</code>	s2-s11	Saved	General-purpose storage.
<code>x28-x31</code>	t3-t6	Temporary	General-purpose storage.

Note

ABI (Application Binary Interface) is the contract among software components at the binary level: calling convention, register use, stack layout and similar rules are defined here. While translating the program into machine code, the compiler produces code according to the register roles defined in the ABI. From the hardware standpoint most RISC-V integer registers are general-purpose; exceptions such as `x0` being constant zero are architecturally fixed. In some architectures such as ARM, certain registers (for example the link register) traditionally carry fixed roles as well.

x0-x7	x8-x15	x16-x23	x24-x30
Parameter and Result Registers (x0-x7)	XR (x8)	IP0 (x16)	Callee-saved Registers (x24-x28)
	Corruptible Registers (x9-x15)	IP1 (x17)	
		PR (x18)	
		Callee-saved Registers (x19-x23)	FP (x29)
			LR (x30)

Figure 3.3: ARM architecture ABI map [6].

3.3.1 Features

There are two combinational read ports and one synchronous write port. Reading occurs combinationally, while writing occurs sequentially (on the clock edge). Each register holds 32-bit data; under suitable conditions a read and a write can be performed within the same cycle.

3.3.2 Usage Example

add x1, x2, x3

The sum of x2 and x3 is written to the x1 address.

addi x1, x2, 5

The sum of the value stored in x2 and the immediate 5 is written to the x1 address.

3.3.3 Debugging

There are observable debug ports in the system; while the processor is running, writes made to the register file can be observed through these ports. If the initial state after reset is known and all writes are observed completely, write-port tracking is a practical way to reconstruct the register file contents. In cases of a shadow bank or missed writes, write-port observation alone may not be sufficient.

Table 3.2: Register file debug signals and descriptions

Signal	Description
dbg_rf_we_o	Indicates that a write operation to the register file was performed in the related clock cycle.
dbg_rf_waddr_o[4:0]	Indicates the target register address of the write operation (x0-x31).
dbg_rf_wdata_o[31:0]	Indicates the 32-bit data written to the register file.

3.4 The Instruction Concept

3.4.1 What Is It?

An instruction is the command that the processor will decode and execute. Instructions are decoded within the processor and directed to the related units. Within the RISC-V framework, instructions define the basic operations with which a program can be expressed and their bit-level encodings.

3.4.1.1 Example

RISC and CISC are two fundamental philosophies. RISC prefers to express frequently used operations with simpler instructions. CISC, on the other hand, can define more complex operations within a single instruction. In RISC architecture, basic instructions such as **and**, **or**, **xor**, **add**, **sub** have been defined. These are basic operations at the ISA level; they can be broken into smaller steps inside the micro-architecture.

3.4.2 Where Are Instructions Stored?

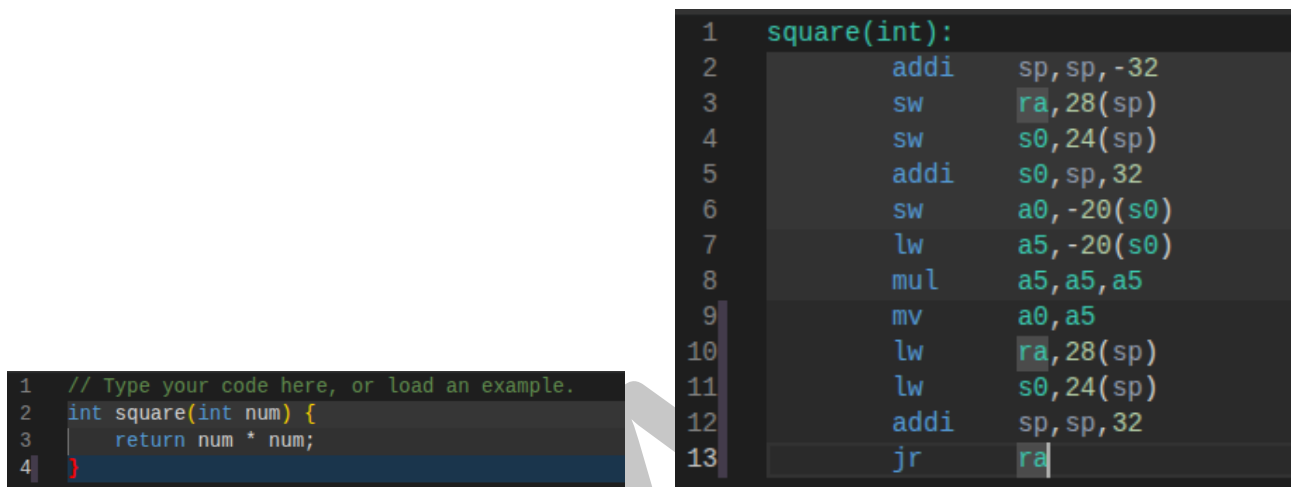
Although instructions may differ among architectures, in Harvard architectures they are held in the instruction memory block called IMEM and are processed by being selected in order with the program counter.

3.4.3 How Are Instructions Generated?

Instructions are produced when compilers translate the code written by users. The compiler topic is deep and important for embedded-system designers. It is skipped in this course because covering it in full would be long and complex. A short example is included for interested readers.

3.4.3.1 Example-1

A compiler input and output example is given below.



```

1 // Type your code here, or load an example.
2 int square(int num) {
3     return num * num;
4 }

```

```

1 square(int):
2     addi    sp, sp, -32
3     sw      ra, 28(sp)
4     sw      s0, 24(sp)
5     addi    s0, sp, 32
6     sw      a0, -20(s0)
7     lw      a5, -20(s0)
8     mul     a5, a5, a5
9     mv      a0, a5
10    lw      ra, 28(sp)
11    lw      s0, 24(sp)
12    addi    sp, sp, 32
13    jr      ra

```

Figure 3.4: Transformation of C source code into RISC-V assembly output by the compiler.

3.4.3.2 Example-2

The importance of knowing the compiler can be exemplified as follows. A compiler must assume that pointers may alias the same memory object, so it does not apply some aggressive optimizations to multi-array operations. If the designer guarantees that the addresses do not overlap (do not alias), the array inputs are marked with the **restrict** qualifier. This does not turn off a security parameter; it tells the compiler that there is no aliasing, which allows more aggressive optimization at the expected performance.

This situation is seen in operations such as $X[i] = Y[i] + Z[i]$. For further reading the topic **Pointer Aliasing** should be consulted.

3.4.4 Who Determines It

RISC-V instruction sets are defined by specifications ratified by RISC-V International [1]. Community members, developers, and researchers may propose extensions; the official ISA documents are published by the foundation. These instruction sets can be extended according to users' requests. However, because of situations such as compiler mismatch and the code not compiling, designers generally develop around the instructions that the compiler supports.

3.5 Arithmetic Logic Unit (ALU)

The ALU is present in every processor. It performs the arithmetic and logic operations of the supported instructions. Some of its operations can be pipelined to raise the system frequency; most operations still complete in a single cycle.

3.6 MUL/DIV

The M extension provides multiply, divide and remainder operations. The `MUL_ENABLE` and `DIV_ENABLE` parameters are independent of each other. When the related unit is disabled, the opcodes belonging to that unit produce an *illegal instruction* trap. Both units operate in the EX stage and the EX stage is held in stall until the result is ready.

3.6.1 MUL

`tmcu_mul` is a 2-cycle pipelined multiply unit and supports the `MUL`, `MULH`, `MULHSU` and `MULHU` instructions. The unit uses a single 33×33-bit signed multiplier. Depending on the operation, the high or low 32-bit part of the product result is selected.

As long as the unit is available, a new multiply operation can be started in every clock cycle. This is the multiplier unit's own throughput property. Because microBee is a single-issue core, this does not mean that the core can issue a new `MUL` every cycle; stall in the EX stage and other pipeline constraints also apply.

3.6.2 DIV

`tmcu_div` is a divide unit that uses the iterative restoring division algorithm and supports the `DIV`, `DIVU`, `REM` and `REMU` instructions.

The typical operation duration is approximately 34 clock cycles (setup + 32 iteration steps + termination). While a divide or remainder operation is in progress, stall is applied to the EX stage. If a pipeline flush occurs, the ongoing operation is cancelled.

3.6.3 Why Separate Units and Pipeline Are Used

While ALU operations can be completed in a single cycle, multiply and especially divide operations can take longer. Thanks to performing `MUL` and `DIV` operations in separate units and stalling the EX stage when necessary:

- single-cycle ALU instructions keep their single-cycle execution duration,
- long-running `MUL/DIV` operations hold the pipeline only when necessary,
- the `MUL` and `DIV` units can be enabled or disabled independently with synthesis parameters.

Performing `MUL` and `DIV` operations entirely with single-cycle combinational logic can lengthen the critical path, lowering the maximum operating frequency and increasing area usage. In contrast, designing the entire core according to long `MUL/DIV` delays also unnecessarily lowers the performance of single-cycle instructions. For this reason, in microBee long-latency operations are performed through separate execution units.

3.7 Branching (Branch)

3.7.1 What Is It?

Branch instructions are used to change the program's normal sequential flow. In the normal case the Program Counter advances to the next instruction address. In a branch instruction, however, a certain condition is checked and if this condition is true the program continues to run from a different address.

This structure is used especially in:

- if-else structures,
- loops,
- comparison-dependent decision mechanisms

For example:

```
beq x1, x2, label
```

instruction compares the `x1` and `x2` registers. If the two values are equal the program branches to the `label` address. If they are not equal the program continues from the next instruction in normal order.

The branch result has two basic states:

- **Taken:** The condition is satisfied and the branch target address is taken.
- **Not-Taken:** The condition is not satisfied and the next instruction is executed.

3.7.2 How Is It Expressed?

Within RISC-V, conditional branching operations are expressed with branch instructions.

The basic branch instructions supported by microBee are the following:

- **BEQ:** The branch is taken if equal.
- **BNE:** The branch is taken if not equal.
- **BLT:** The branch is taken if less in a signed comparison.
- **BGE:** The branch is taken if greater or equal in a signed comparison.
- **BLTU:** The branch is taken if less in an unsigned comparison.
- **BGEU:** The branch is taken if greater or equal in an unsigned comparison.

Branch instructions generally contain two registers and one immediate.

For example:

beq x5, x6, 16

in an instruction of this form **x5** and **x6** are compared. If the condition is satisfied the branch target address is computed by adding the current Program Counter and the immediate value.

In general it can be expressed as:

$$\text{Branch Target} = PC + \text{Immediate}$$

If the branch is not taken the Program Counter advances to the next instruction address in normal order.

Because in the RV32IMC structure the instruction length can be 16-bit or 32-bit, the next PC value is determined according to the instruction length.

3.7.3 How Is It Resolved?

Within microBee the actual result of branch instructions is finalized in the Execute (**EX**) stage.

While the branch advances within the pipeline, the required register values are read in the Decode stage and sent to the Execute stage.

In the Execute stage two basic operations are performed:

1. the source operands are compared according to the branch type,
2. the branch target address is computed with **PC + immediate**.

For example, in a **BEQ** instruction the following comparison is performed in the Execute stage:

$$rs1 == rs2$$

If the result is true the branch is determined as taken, if false as not-taken.

In **BLT** and **BGE** instructions a signed comparison is performed, while in **BLTU** and **BGEU** instructions an unsigned comparison is performed.

Because the branch result is finalized in the EX stage, other instructions may have entered the pipeline on the Fetch and Decode sides until this stage is reached.

If branch prediction is correct the taken instructions remain valid and the pipeline continues to operate normally.

If prediction is wrong, instructions taken over the wrong execution path may be present on the Fetch and Decode sides.

In this case microBee:

- makes the wrong instructions invalid,

- flushes the IF/ID side,
- updates the Program Counter with the correct branch address or the fall-through address,
- restarts the Fetch operation from the correct address.

This results in a branch misprediction penalty.

Giving the branch decision earlier within the pipeline can reduce the penalty; however in that case it may be necessary to add extra comparison and address-computation logic to the Fetch or Decode stages.

Performing branch resolution in the EX stage in microBee is a trade-off made between design complexity and performance.

3.8 Stall and Flush Concepts

In a pipeline structure it is the ideal case that instructions advance to the next stage every cycle. However in some situations the data or result that an instruction needs may not yet be ready.

In such situations the pipeline needs to be controlled.

Two of the basic mechanisms used for this control are the **stall** and **flush** operations.

3.8.1 What Is Stall?

Stall is temporarily stopping the advance of certain instructions within the pipeline.

If the data or hardware resource that an instruction needs is not ready, the instruction is not allowed to advance.

In this case the related pipeline registers keep their current values.

Simply, while:

Normal: IF -> ID -> EX -> MEM -> WB

in the stall case a certain instruction may wait one or more cycles within the same stage.

Stall does not produce a new instruction. It causes the existing instruction to wait until the required conditions are satisfied.

3.8.2 Situations That Produce Stall in microBee

Stall can arise for different reasons within microBee.

3.8.2.1 Load Operations

In a load instruction the data to be read from memory needs to return.

If the memory response is not ready it is not possible for the instruction to advance to the Writeback stage.

In this case the pipeline is stalled until the memory response arrives.

3.8.2.2 Load-Use Hazard

If another instruction that uses the loaded register comes immediately after a load instruction, the data may not yet be ready.

For example:

```
lw  x5, 0(x10)
add x6, x5, x7
```

the second instruction needs the x5 value.

If the required data cannot be provided in time with forwarding, the pipeline is stalled for one or more cycles and the load result is waited for to become ready.

3.8.2.3 MUL and DIV Operations

The MUL and DIV units can take longer than ALU operations.

While these operations run in the Execute stage, the EX stage is held in stall until the result is ready.

Especially because the DIV operation works iteratively, the pipeline can wait for a long time in the EX stage.

3.8.2.4 Store Operations Outside the Core

In store operations performed within the microBee core the core is not stalled.

However if the store operation is directed to a bus or peripheral outside the core, the other side may not accept the request immediately.

In this case the core can be stalled until the external interface is ready.

3.8.3 What Is Flush?

Flush is making invalid the instructions that are in the pipeline but should no longer be executed.

The basic difference between stall and flush is the following:

- **stall**: the instruction is held,
- **flush**: the instruction is cancelled completely.

Flush is used especially in control hazard situations.

When branch prediction is wrong, instructions may have been taken into the pipeline over the wrong execution path.

Executing these instructions causes the program to produce a wrong result.

For this reason the instructions on the wrong path are cleaned out of the pipeline.

3.8.4 Branch Flush in microBee

In microBee the branch result is finalized in the EX stage.

If the branch prediction result does not match the actual branch result, wrong instructions may be present on the Fetch and Decode sides.

In this case the IF/ID pipeline section is flushed and Fetch is restarted from the correct Program Counter value.

For example if the predictor predicted the branch as not-taken but the actual result came out taken:

1. the next instructions were taken from the wrong path,
2. in the EX stage it is understood that the branch is taken,
3. the wrong Fetch/Decode instructions are flushed,
4. the PC is set to the branch target address,
5. Fetch starts again from the correct address.

Similarly, if the predictor predicted taken but the actual result is not-taken, the PC is directed to the fall-through address.

3.8.5 The Difference Between Stall and Flush

Although stall and flush appear similar to each other, their purposes of use are different.

	Stall	Flush
Purpose	To hold	To cancel
Instruction	Remains valid	Becomes invalid
PC	Generally is held	Can be directed to a new address
Example	Load wait	Branch misprediction

Stall is used more in data hazard, long-running execution and memory wait situations.

Flush, on the other hand, is used to clean wrong instruction paths or instructions that are no longer valid out of the pipeline.

3.8.6 The Bubble Concept

As a result of stall or flush there may not be a valid instruction in some cycles within the pipeline.

This empty state is called a **bubble**.

A bubble can advance within the pipeline like a real instruction but it does not produce any change in architectural state.

For example, applying a one-cycle stall because of a load-use hazard can cause a bubble to form within the pipeline.

The places of instructions flushed after a branch misprediction also form temporary gaps within the pipeline.

An increase in the number of bubbles lowers the processor's IPC value and increases the CPI value.

For this reason, reducing stall, flush and bubble formation as much as possible in pipeline design is important from a performance standpoint.

3.9 Branch Prediction

3.9.1 What Is It?

Branch prediction is the processor's attempt to predict the result of a branch instruction before it is computed. In processors with a pipeline structure it is desired to know from which address the next instruction will be taken as early as possible.

In the normal case the program counter advances to the next address after every instruction. However in branch instructions there are two different possibilities:

- if the branch is not taken the program continues from the next instruction in normal order,
- if the branch is taken the program continues from the branch target address.

Knowing the branch result with certainty can last until the Execute stage. Holding the Fetch unit during this time lowers pipeline performance. The branch predictor predicts the branch result in advance in order to reduce this wait.

3.9.2 Why Is It Necessary?

Within a 5-stage pipeline it is possible that the instruction is taken in the Fetch stage while the branch decision is given in the Execute stage. In this case the processor has two options in front of it.

The first option is not to take a new instruction until the branch result is known. This method is simple from a hardware standpoint but the pipeline remains empty for several cycles on every branch instruction.

The second option is to continue taking instructions by predicting the result of the branch. If the prediction is correct the pipeline continues to run without stopping. If the prediction is wrong the instructions taken from the wrong path are cleaned out of the pipeline and Fetch is started again from the correct address.

For this reason branch prediction is fundamentally a trade-off made between performance and hardware complexity.

3.9.3 Taken and Not-Taken

The result of branch instructions is generally expressed in two ways.

Taken: The condition is satisfied and execution continues at the branch target.

Not-Taken: The branch condition has not been satisfied and the program continues from the next instruction.

For example the following instruction can be considered:

beq x1, x2, label

If the values inside **x1** and **x2** are equal the branch is **taken**, if they are not equal it is **not-taken**.

3.9.4 Bimodal Branch Prediction

A 2-bit state is held for each predictor entry. In general there are four states:

State	Prediction
00	Strongly Not-Taken
01	Weakly Not-Taken
10	Weakly Taken
11	Strongly Taken

When the branch is taken the counter is incremented by one, when it is not-taken it is decremented by one. The counter cannot go below the value 00 or above the value 11. For this reason it is named a *saturating counter* because the count reaches saturation (saturates).

In this structure the predictor generally needs to see several reverse results in succession in order to change its decision. In this way a single not-taken result at the end of a loop does not directly change the predictor's entire behavior.

3.9.5 What Happens If Prediction Is Correct?

If the predictor has correctly predicted the result of the branch, the instructions taken by Fetch are on the correct execution path.

In this case there is no need to clean the pipeline and the branch penalty is largely eliminated.

This situation can be thought of as a **correct prediction**.

3.9.6 What Happens If Prediction Is Wrong?

If the predictor has produced a wrong result, the processor has started taking instructions over the wrong instruction path.

When the actual branch result is computed in the Execute stage, the prediction and the actual result are compared.

If they do not match:

1. instructions taken from the wrong path are made invalid,
2. the related stages within the pipeline are flushed,
3. the PC is updated with the correct branch target or the fall-through address,
4. Fetch starts again from the correct address.

The number of lost cycles that this operation produces can be called the branch misprediction penalty.

How late the branch is resolved within the pipeline directly affects the misprediction penalty.

3.9.7 Why Is a Simple Predictor Used in Embedded Systems?

In high-performance processors very large and complex branch predictor structures can be used. In these structures there can be systems in which global history, local history, a tournament predictor or more than one predictor are used together.

In embedded processors, however, only maximum performance is not targeted.

The following parameters are also important:

- the FPGA or silicon area used,
- power consumption,
- predictor access latency,
- design complexity,
- verification cost,
- operating frequency.

For example a very large predictor can increase the prediction accuracy value. However lengthening the critical path in the Fetch stage with the predictor result can lower the processor's maximum frequency.

In this case, although prediction accuracy increases, total system performance may not increase as much as expected.

For this reason, when selecting a predictor in embedded core design it is not correct to look only at the prediction accuracy value.

Area, power, frequency and misprediction penalty should be evaluated together.

3.9.8 Branch Prediction Performance

Branch predictor performance can generally be measured through prediction accuracy.

$$\text{Prediction Accuracy} = \frac{\text{True Guess Count}}{\text{Total Guess Count}}$$

Similarly the misprediction rate can be thought of as:

$$\text{Misprediction Rate} = \frac{\text{Wrong Guess Count}}{\text{Total Guess Count}}$$

However looking only at the accuracy value is not sufficient.

For example if the ratio of branch instructions within the program is low, a small increase in the predictor accuracy value can produce very little change on total CPI. For the related situation **Amdahl's Law** can be consulted.

The approximate effect of branch penalty can be thought of with the following logic:

$$\text{CPI}_{\text{branch}} \approx \text{Branch Frequency} \times \text{Misprediction Rate} \times \text{Misprediction Penalty}$$

This equation is not a complete processor performance model but it is useful for understanding the effect of branch prediction on CPI.

3.9.9 Branch Prediction from the microBee Perspective

In the microBee design a 128-entry bimodal branch predictor is used. There are 128 2-bit saturating counters within the predictor. An entry within this table is selected by using certain bits of the branch instruction's Program Counter value.

Each entry holds one of the following four states:

State	Prediction
00	Strongly Not-Taken
01	Weakly Not-Taken
10	Weakly Taken
11	Strongly Taken

When the branch result is finalized in the Execute stage the related entry is updated. If the branch was taken the counter is incremented, if it was not-taken it is decremented.

Using 128 entries keeps the predictor's area cost low while enabling it to benefit from past branch behaviors. However different Program Counter values can map to the same entry. In this case two different branches share the same predictor state. This situation is named **aliasing**.

For a core that targets embedded use such as microBee, when determining the predictor size, prediction accuracy together with FPGA area usage, power consumption and the critical path in the Fetch stage have been evaluated together.

3.9.10 Example

The following simple loop can be considered:

```
for (int i = 0; i < 100; i++) {
    sum += array[i];
}
```

The compiler can form a conditional branch at the end of this structure.

If the loop runs 100 times the branch can approximately show the behavior:

- taken repeatedly in the first iterations,
- not-taken once when the loop completes

An Always Not-Taken predictor can produce a large number of wrong predictions during this loop.

A 2-bit predictor, on the other hand, can after a few iterations learn the branch's taken behavior and make a correct prediction throughout the loop. Even if one wrong prediction occurs at the loop exit, the predictor's state does not reverse completely.

This example shows why dynamic branch prediction is especially useful in loop-heavy code.

3.10 DMEM

3.10.1 What Is It?

DMEM is the abbreviation of Data Memory and denotes the memory area in which the data used while the processor runs the program are held.

In systems that use Harvard architecture the memory in which instructions are found is separated as IMEM, while the memory in which data are found is separated as DMEM.

Instruction Fetch operations are performed over IMEM. The target of load and store operations is determined according to the address: it can go to DMEM as well as being directed to MMIO peripherals or to the AXI window. DMEM is the local data memory in which general-purpose data such as program variables and the stack in particular are held.

For example in the C language, when a code such as:

```
int a = 10;
int b = 20;
int c = a + b;
```

is executed, depending on the program formed by the compiler the variables can be held on the register file or on data memory.

In situations where the number of registers is not sufficient or the data needs to be in memory, DMEM access is performed.

3.10.2 Load and Store

In RISC-V architecture the processor is not expected to operate on data inside memory directly with ALU instructions.

First the data are taken into the register file.

This operation is performed with **load** instructions.

Writing a piece of data inside a register to memory is performed with **store** instructions.

For example:

```
lw x5, 0(x10)
```

instruction reads 32-bit data over the address found in the x10 register and writes the result to the x5 register. Similarly:

```
sw x5, 0(x10)
```

instruction writes the 32-bit data inside x5 to the memory address pointed to by x10.

3.10.3 How Is the Address Computed?

The address to be used in load and store instructions is computed in the Execute stage.

The general structure is:

$$\text{Memory Address} = rs1 + \text{immediate}$$

For example:

lw x5, 8(x10)

in the instruction the memory address is computed as:

$$\text{Address} = x10 + 8$$

After this address is computed through the ALU it is sent to the Memory stage.

3.10.4 Data Sizes

Within RV32, memory accesses do not have to be only 32-bit. Accesses can be performed in byte, half-word and word sizes.

The basic load instructions:

- LB: 8-bit signed load,
- LBU: 8-bit unsigned load,
- LH: 16-bit signed load,
- LHU: 16-bit unsigned load,
- LW: 32-bit load.

The basic store instructions:

- SB: 8-bit store,
- SH: 16-bit store,
- SW: 32-bit store.

In signed load operations the small data that is read is extended to 32-bit register width with **sign extension**.

In unsigned load operations the remaining upper bits are filled with zero.

3.10.5 Byte Enable

Even if the memory bus is 32-bit, it may not be desired to change the entire 32 bits in every store operation.

For example in an SB instruction only a single byte is written.

For this reason byte enable or write strobe signals that determine which bytes will be written can be used on the memory interface.

For a 32-bit data path four byte lanes can be considered:

[31:24] [23:16] [15:8] [7:0]

Each lane is 8 bits wide.

For example if only the lowest byte will be written:

WSTRB = 4'b0001

can be used.

If a 32-bit word will be written:

WSTRB = 4'b1111

This structure enables the processor to perform byte and half-word store instructions over a 32-bit memory bus.

3.10.6 Memory Stall

The effect of load and store operations on the pipeline is not the same within microBee.

3.10.6.1 Load Operations

In a load instruction the data to be read from memory need to return to the processor. Without this data being ready it is not possible for the instruction to be written to the register file in the Writeback stage.

For this reason, if the memory response is not ready after the load request is formed, the pipeline is held in stall.

Simply it can be thought of as:

Load Request -> Memory Wait -> Load Response -> Writeback

As memory latency increases the stall duration produced by the load instruction can also increase.

3.10.6.2 Store Operations

In store operations the situation is different. Because a store instruction does not write a result to the register file, the processor waiting again for the written data is not required in every case.

In store operations performed within the microBee core, stall is not applied on the core side. After the store request is conveyed to the related memory system the pipeline can continue to operate.

However if the store operation is sent to a target outside the core area the situation can change. For example in a store operation performed over an external bus or peripheral, the other side may not be able to accept the request immediately.

In this case the core can be held in stall until the external interface completes the operation or accepts the request.

For this reason memory stall behavior within microBee can generally be summarized as follows:

- stall can be applied while waiting for the memory response in load operations,
- the core is not stalled in intra-core store operations,
- stall can be applied in store operations that go outside the core if the external bus or target is not ready.

This distinction is especially important in embedded systems. Because memory mapped peripheral or external bus accesses can produce longer latency than local memory accesses within the core.

3.10.7 DMEM and Pipeline

In microBee's 5-stage pipeline structure the basic flow of load/store operations can be thought of as follows:

IF -> ID -> EX -> MEM -> WB

IF: The instruction is taken from memory.

ID: The source registers are read.

EX: The memory address is computed.

MEM: DMEM access is performed.

WB: If it is a load operation the data that were read are written to the register file.

In store instructions register file writeback is not required. The data are sent to the memory system in the MEM stage.

3.10.8 Load-Use Hazard

Load instructions can form an important hazard from a pipeline standpoint.

For example:

```
lw  x5, 0(x10)
add x6, x5, x7
```

the first instruction is reading the `x5` value from memory.

The second instruction, however, needs the `x5` value in the immediately following cycle.

However the load result may not yet be ready.

This situation is called a **load-use hazard**.

In this case, if the required forwarding is not possible, the processor adds a stall to the pipeline so that the second instruction waits until the correct data are ready.

This example shows that memory latency and the pipeline hazard structure are not independent of each other.

3.10.9 DMEM from the Embedded System Perspective

In embedded systems the memory architecture affects total system performance to a large extent.

Even if a very fast processor core has been designed, if slow memory access is performed continuously the processor can spend a large portion of its time in stall.

For this reason system performance should not be evaluated by looking only at processor frequency.

The following parameters should be examined together:

- memory latency,
- memory bandwidth,
- load/store instruction ratio,
- pipeline stall count,
- the bus protocol used,
- memory hierarchy, interconnect and access latency.

Block RAM used close to the processor on FPGA generally provides low latency, while a peripheral reached over external DDR or a bus can have a longer response time.

Therefore in embedded system design the memory system should be evaluated as an inseparable part of the processor architecture.

Chapter 4

Instruction Set

4.1 What Is the ISA?

4.1.1 Definition

The instruction set is the name given to the complete set of instructions that the processor can understand and execute. Instructions produced by the assembler, compiler, or toolchain are decoded by the processor and directed to the corresponding hardware units.

An instruction set defines not only which operations can be performed, but also how those operations are encoded at the bit level.

For example, for an addition instruction:

```
add x1, x2, x3
```

a command can be written in this form.

The processor, however, does not see this command as text. The binary encoding produced by the compiler and assembler reaches the processor. The processor decodes the opcode, funct, and register fields and determines that this is an addition operation.

For this reason, the Instruction Set Architecture, or ISA for short, can be thought of as the fundamental contract between software and processor hardware.

Software must produce instructions that conform to the ISA; hardware must implement the behavior defined by the same ISA.

4.1.2 Who Defines It?

In the RISC-V architecture, instruction sets are defined through specifications determined by RISC-V International [1].

RISC-V uses a modular structure consisting of a base ISA and extensions, rather than a single monolithic instruction set.

For example:

- **I**: The base integer instruction set,
- **M**: Multiply and divide operations,
- **C**: Compressed instructions,
- **Zicsr**: CSR access instructions,
- **Zifencei**: Instruction fetch synchronization

there are different instruction groups of this kind.

A processor designer can select the extensions they need and form their own processor's ISA profile.

For example, microBee:

RV32IMC + Zicsr + Zifencei

supports this instruction set.

Here:

- **RV32**: that XLEN is 32 bits (register and integer operation width),
- **I**: the base integer instructions,
- **M**: multiply and divide operations,
- **C**: 16-bit compressed instructions

are what these fields express.

In addition, the RISC-V architecture also leaves custom encoding spaces for vendor-specific instruction development.

In this way, while the standard ISA is preserved, accelerators or instructions specific to particular applications can be added to the system.

4.1.3 Purpose of the ISA

The fundamental purpose of defining an instruction set is to create a standard interface between software and hardware.

If the ISA is not defined, it is unknown which binary code the compiler should produce, or how the processor should interpret the incoming bits.

The ISA answers the following questions:

- Which instructions are present?
- What is the bit encoding of the instructions?
- How many registers are there?
- How are operands expressed?
- How is memory access performed?
- How do branch and jump operations work?
- How are exception and CSR accesses performed?

Thanks to this structure, different processors that use the same ISA can run the same programs.

For example, two different vendors can design completely different pipeline architectures.

One processor may use a 3-stage pipeline while another uses a 5-stage pipeline, a branch predictor, or different ALU structures.

However, if both processors correctly support the same RISC-V ISA, they can execute the same instructions from a software perspective.

At this point, the ISA and the microarchitecture must be separated from each other.

The ISA defines what the processor does; the microarchitecture determines how it does it.

For example, the **MUL** instruction is defined within the ISA. Whether this instruction is implemented in a single cycle, in a two-cycle pipelined structure, or with an iterative multiplier, however, depends on a microarchitectural design decision.

4.1.4 Why a Fixed ISA?

It is especially important for the software ecosystem that the ISA remain as fixed and standard as possible.

If the processor designer could continually change instruction encodings, the compiler and assembler would have to generate code differently for every processor.

For example, if on one processor:

ADD opcode = A

and on another processor:

ADD opcode = B

the same binary would not run on both processors.

In that case, software compatibility would largely disappear.

Thanks to a fixed ISA, the compiler generates code for a specific target architecture.

For example, for microBee the compiler can be given target-architecture information approximately as:

```
-march=rv32imc_zicsr_zifencei
```

Using this information, the compiler produces only the instructions that the processor supports.

The ISA being standard is also important from the standpoint of compiler optimizations.

Because the compiler knows an instruction's behavior with certainty:

- instruction scheduling,
- register allocation,
- constant propagation,
- loop optimization,
- instruction selection

it can perform operations such as these safely.

If instruction behaviors or encodings change from design to design, it becomes harder for the compiler to apply these optimizations.

The same problem also applies to the operating system, debugger, assembler, linker, and other tools.

For this reason, one of the greatest advantages of a standard ISA is not only that it simplifies processor design, but also that it enables a large software ecosystem to operate on the same architecture.

However, a completely fixed ISA may not be sufficient for every application.

For example, in particular domains such as image processing, artificial intelligence, or cryptography, it may be desired to develop special instructions that run faster than the standard instructions.

RISC-V solves this problem with custom instruction spaces.

Design-specific instructions can be defined in custom opcode spaces. In this way, standard RISC-V instruction encodings are not changed, and existing compiler/toolchain compatibility that relies on the standard encodings is not broken. For software that uses custom instructions, toolchain support is additionally required.

If it is desired that the compiler automatically use these new instructions, support for the relevant instructions must also be added to the compiler backend.

Otherwise, custom instructions can be used manually through methods such as assembly or intrinsics.

For this reason, adding a new instruction is not only an RTL design task. Compiler and toolchain support is also an important part of the design so that the instruction can be used efficiently by software.

4.2 I Set

The I set is the base integer instruction set of the RV32I architecture [2]. Because microBee is a 32-bit processor, the register width (**XLEN**) is 32 bits. Instructions in the I set are 32 bits long.

The fundamental fields used in the instruction encoding are as follows:

- **opcode**: Determines the instruction's base class.
- **rd**: The destination register to which the result is written.
- **rs1**: The first source register.
- **rs2**: The second source register.
- **funct3**: Distinguishes operations within the same opcode.
- **funct7**: Provides additional distinction, especially in R-type operations.
- **imm**: A constant value carried within the instruction.

Within RV32I, different instruction formats are used: R, I, S, B, U, and J.

4.2.1 R-Type Format

R-Type instructions operate on two registers and write the result to another register.

31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	opcode

General operation:

$$rd \leftarrow operation(rs1, rs2)$$

4.2.1.1 ADD

Adds the values of two registers.

add rd, rs1, rs2

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	000	rd	0110011

$$rd = rs1 + rs2$$

4.2.1.2 SUB

sub rd, rs1, rs2

31:25	24:20	19:15	14:12	11:7	6:0
0100000	rs2	rs1	000	rd	0110011

$$rd = rs1 - rs2$$

4.2.1.3 SLL

Performs a logical left shift. The shift amount is determined by `rs2[4:0]`.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	001	rd	0110011

$$rd = rs1 \ll rs2[4:0]$$

4.2.1.4 SLT

Performs a signed comparison.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	010	rd	0110011

$$rd = \begin{cases} 1, & signed(rs1) < signed(rs2) \\ 0, & otherwise \end{cases}$$

4.2.1.5 SLTU

Performs an unsigned comparison.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	011	rd	0110011

$$rd = \begin{cases} 1, & unsigned(rs1) < unsigned(rs2) \\ 0, & otherwise \end{cases}$$

4.2.1.6 XOR

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	100	rd	0110011

$$rd = rs1 \oplus rs2$$

4.2.1.7 SRL

Performs a logical right shift and fills the upper bits with zeros. The shift amount is determined by `rs2[4:0]`.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	101	rd	0110011

$$rd = rs1 \gg rs2[4:0]$$

4.2.1.8 SRA

Performs an arithmetic right shift. The sign bit is preserved. The shift amount is determined by `rs2[4:0]`.

31:25	24:20	19:15	14:12	11:7	6:0
0100000	rs2	rs1	101	rd	0110011

$$rd = \text{signed}(rs1) \gg rs2[4:0]$$

(Right shift with the sign bit preserved.)

4.2.1.9 OR

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	110	rd	0110011

$$rd = rs1 | rs2$$

4.2.1.10 AND

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	111	rd	0110011

$$rd = rs1 \& rs2$$

4.2.2 I-Type Arithmetic Instructions

General I-Type format:

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	funct3	rd	opcode

The immediate is 12 bits and is sign-extended to 32 bits in arithmetic operations.

4.2.2.1 ADDI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	000	rd	0010011

$$rd = rs1 + \text{signextend}(imm)$$

4.2.2.2 SLTI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	010	rd	0010011

Performs a signed comparison.

4.2.2.3 SLTIU

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	011	rd	0010011

Performs an unsigned comparison. The immediate is first sign-extended to 32 bits; the comparison is then performed as unsigned.

4.2.2.4 XORI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	100	rd	0010011

4.2.2.5 ORI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	110	rd	0010011

4.2.2.6 ANDI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	111	rd	0010011

4.2.3 Shift Immediate Instructions**4.2.3.1 SLLI**

31:25	24:20	19:15	14:12	11:7	6:0
0000000	shamt[4:0]	rs1	001	rd	0010011

4.2.3.2 SRLI

31:25	24:20	19:15	14:12	11:7	6:0
0000000	shamt[4:0]	rs1	101	rd	0010011

4.2.3.3 SRAI

31:25	24:20	19:15	14:12	11:7	6:0
0100000	shamt[4:0]	rs1	101	rd	0010011

4.2.4 Load Instructions

Load instructions use the I-Type format.

Effective address:

$$\text{Address} = rs1 + \text{signextend}(imm[11 : 0])$$

4.2.4.1 LB

Reads 8-bit signed data and sign-extends it to 32 bits.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	000	rd	0000011

4.2.4.2 LH

Reads 16-bit signed data and sign-extends it to 32 bits.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	001	rd	0000011

4.2.4.3 LW

Reads a 32-bit word.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	010	rd	0000011

4.2.4.4 LBU

Reads 8-bit unsigned data and applies zero-extension.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	100	rd	0000011

4.2.4.5 LHU

Reads 16-bit unsigned data and extends it to 32 bits with zero-extension.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	101	rd	0000011

4.2.5 Store Instructions

Store instructions use the S-Type format.

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode

4.2.5.1 SB

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011

rs2[7:0] is written to memory.

4.2.5.2 SH

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011

rs2[15:0] is written to memory.

4.2.5.3 SW

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011

The 32-bit rs2 value is written to memory.

4.2.6 Branch Instructions

Branch instructions use the B-Type format.

31	30:25	24:20	19:15	14:12	11:8	7	6:0
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	1100011

Branch target:

$$\text{Target} = \text{PC} + \text{signextend}(\{imm[12:1], 1'b0\})$$

4.2.6.1 BEQ

If $rs1 == rs2$, the branch is taken.

31	30:25	24:20	19:15	14:12	11:8	7	6:0
imm[12]	imm[10:5]	rs2	rs1	000	imm[4:1]	imm[11]	1100011

4.2.6.2 BNE

If $rs1 \neq rs2$, the branch is taken.

funct3 = 001

4.2.6.3 BLT

Signed $rs1 < rs2$.

funct3 = 100

4.2.6.4 BGE

Signed $rs1 \geq rs2$.

funct3 = 101

4.2.6.5 BLTU

Unsigned $rs1 < rs2$.

funct3 = 110

4.2.6.6 BGEU

Unsigned $rs1 \geq rs2$.

funct3 = 111

In all branch instructions the other bit fields are the same as in BEQ, and only the funct3 field changes.

4.2.7 Jump Instructions

4.2.7.1 JAL

Uses the J-Type format.

31	30:21	20	19:12	11:7	6:0
imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	1101111

$$rd = \text{PC} + 4$$

$$PC = PC + \text{signextend}(\{imm[20 : 1], 1'b0\})$$

Because the JAL instruction is 32 bits, the link value is always **PC+4**.

4.2.7.2 JALR

Uses the I-Type format.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	000	rd	1100111

$$rd = PC + 4$$

$$PC = (rs1 + \text{signextend}(imm)) \& \sim 1$$

The least significant bit of the target address is cleared.

4.2.8 Upper Immediate Instructions

4.2.8.1 LUI

It is in U-Type format.

31:12	11:7	6:0
imm[31:12]	rd	0110111

$$rd = imm[31 : 12] \ll 12$$

4.2.8.2 AUIPC

31:12	11:7	6:0
imm[31:12]	rd	0010111

$$rd = PC + (imm[31 : 12] \ll 12)$$

4.2.9 FENCE

Used to order the observation of memory operations.

31:28	27:24	23:20	19:15	14:12	11:7	6:0
fm	pred	succ	00000	000	00000	0001111

A standard FENCE instruction uses **fm=0000**.

pred specifies the classes of preceding memory operations, and **succ** specifies the classes of succeeding memory operations.

4.2.10 Environment Instructions

4.2.10.1 ECALL

Creates a service request to the execution environment.

31:20	19:15	14:12	11:7	6:0
000000000000	00000	000	00000	1110011

32-bit encoding:

0x000000073

4.2.10.2 EBREAK

Used to create a breakpoint trap to the debugger or execution environment.

31:20	19:15	14:12	11:7	6:0
0000000000001	00000	000	00000	1110011

32-bit encoding:

0x00100073

4.3 M Set

The M extension defines integer multiply, divide, and remainder operations [3].

All RV32M instructions use the R-Type format and have:

`funct7 = 0000001`

this value.

4.3.1 MUL

Returns the low 32 bits of a 32×32-bit product.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	000	rd	0110011

$$rd = (rs1 \times rs2)[31 : 0]$$

4.3.2 MULH

Gives the high 32 bits of a signed × signed product.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	001	rd	0110011

4.3.3 MULHSU

`rs1` is treated as signed, `rs2` as unsigned.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	010	rd	0110011

4.3.4 MULHU

Gives the high 32 bits of an unsigned × unsigned product.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	011	rd	0110011

4.3.5 DIV

Performs signed integer division.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	100	rd	0110011

$$rd = \text{signed}(rs1) / \text{signed}(rs2)$$

Special cases:

- Division by zero: the result is 0xFFFFFFFF.
- Signed overflow ($\text{INT_MIN} / -1$): the result is **rs1** (0x80000000).

4.3.6 DIVU

Performs unsigned division.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	101	rd	0110011

On division by zero the result is 0xFFFFFFFF.

4.3.7 REM

Gives the remainder of a signed division.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	110	rd	0110011

Special cases:

- Division by zero: the remainder is **rs1**.
- Signed overflow ($\text{INT_MIN} \% -1$): the remainder is 0.

4.3.8 REMU

Gives the remainder of an unsigned division.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	111	rd	0110011

On division by zero the remainder is **rs1**.

4.4 C Set

The C extension allows frequently used RISC-V instructions to be expressed with 16-bit encoding [4]. Its primary purpose is to reduce program size and instruction fetch bandwidth demand.

Because microBee uses RV32C, 16-bit and 32-bit instructions can coexist within the same program.

The lowest two bits of a 16-bit instruction determine the instruction quadrant:

- 00: Quadrant 0
- 01: Quadrant 1
- 10: Quadrant 2
- 11: 32-bit instruction (within the microBee/RV32IMC scope)

In some compressed instructions only 8 registers can be used:

Compressed address	Actual register
000	x8
001	x9
010	x10
011	x11
100	x12
101	x13
110	x14
111	x15

These registers are shown as **rd'**, **rs1'**, and **rs2'**.

4.4.1 C.ADDI4SPN

Adds a positive immediate to the stack pointer and writes it to one of the **x8-x15** registers.

15:13	12:11	10:7	6	5	4:2	1:0
000	nzuimm[5:4]	nzuimm[9:6]	nzuimm[2]	nzuimm[3]	rd'	00

$$rd' = x2 + nzuimm$$

The immediate cannot be zero.

4.4.2 C.LW

15:13	12:10	9:7	6	5	4:2	1:0
010	uimm[5:3]	rs1'	uimm[2]	uimm[6]	rd'	00

$$rd' = MEM[rs1' + uimm]$$

4.4.3 C.SW

15:13	12:10	9:7	6	5	4:2	1:0
110	uimm[5:3]	rs1'	uimm[2]	uimm[6]	rs2'	00

4.4.4 C.NOP

C.NOP is the compressed ADDI **x0,x0,0** instruction.

15:13	12	11:7	6:2	1:0
000	0	00000	00000	01

Encoding:

0x0001

4.4.5 C.ADDI

15:13	12	11:7	6:2	1:0
000	imm[5]	rd/rs1	imm[4:0]	01

$$rd = rd + \text{signextend}(imm)$$

4.4.6 C.JAL

Used within RV32C for a short-range subroutine call.

15:13	12	11	10:9	8	7	6	5:3	2	1:0
001	imm[11]	imm[4]	imm[9:8]	imm[10]	imm[6]	imm[7]	imm[3:1]	imm[5]	01

$$x1 = PC + 2$$

$$PC = PC + \text{offset}$$

4.4.7 C.LI

15:13	12	11:7	6:2	1:0
010	imm[5]	rd	imm[4:0]	01

$$rd = \text{signextend}(imm)$$

4.4.8 C.ADDI16SP

Modifies the stack pointer by an immediate that is a multiple of 16.

15:13	12	11:7	6	5	4:3	2	1:0
011	nzimm[9]	00010	nzimm[4]	nzimm[6]	nzimm[8:7]	nzimm[5]	01

$$x2 = x2 + \text{signextend}(nzimm)$$

4.4.9 C.LUI

15:13	12	11:7	6:2	1:0
011	nzimm[17]	rd	nzimm[16:12]	01

The rd value cannot be x0 or x2.

4.4.10 C.SRLI

15:13	12	11:10	9:7	6:2	1:0
100	0	00	rd'/rs1'	shamt[4:0]	01

Within RV32C, shamt[5]=0.

4.4.11 C.SRAI

15:13	12	11:10	9:7	6:2	1:0
100	0	01	rd'/rs1'	shamt[4:0]	01

4.4.12 C.ANDI

15:13	12	11:10	9:7	6:2	1:0
100	imm[5]	10	rd'/rs1'	imm[4:0]	01

4.4.13 C.SUB

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	00	rs2'	01

4.4.14 C.XOR

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	01	rs2'	01

4.4.15 C.OR

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	10	rs2'	01

4.4.16 C.AND

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	11	rs2'	01

4.4.17 C.J

15:13	12	11	10:9	8	7	6	5:3	2	1:0
101	imm[11]	imm[4]	imm[9:8]	imm[10]	imm[6]	imm[7]	imm[3:1]	imm[5]	01

$$PC = PC + \text{offset}$$

4.4.18 C.BEQZ

15:13	12	11:10	9:7	6:5	4:3	2	1:0
110	imm[8]	imm[4:3]	rs1'	imm[7:6]	imm[2:1]	imm[5]	01

If $rs1' == 0$, the branch is taken.

4.4.19 C.BNEZ

15:13	12	11:10	9:7	6:5	4:3	2	1:0
111	imm[8]	imm[4:3]	rs1'	imm[7:6]	imm[2:1]	imm[5]	01

If $rs1' != 0$, the branch is taken.

4.4.20 C.SLLI

15:13	12	11:7	6:2	1:0
000	0	rd/rs1	shamt[4:0]	10

4.4.21 C.LWSP

Performs a stack-pointer-based word load.

15:13	12	11:7	6:4	3:2	1:0
010	uimm[5]	rd	uimm[4:2]	uimm[7:6]	10

$$rd = MEM[x2 + uimm]$$

The $rd=x0$ encoding is reserved.

4.4.22 C.JR

15:12	11:7	6:2	1:0
1000	rs1	00000	10

$rs1$ cannot be zero.

$$PC = rs1 \& \sim 1$$

The least significant bit of the target address is cleared in accordance with JALR semantics.

4.4.23 C.MV

15:12	11:7	6:2	1:0
1000	rd	rs2	10

$$rd = rs2$$

rd and rs2 must not be zero.

4.4.24 C.EBREAK

15:12	11:7	6:2	1:0
1001	00000	00000	10

Encoding:

$$0x9002$$

4.4.25 C.JALR

15:12	11:7	6:2	1:0
1001	rs1	00000	10

$$x1 = PC + 2$$

$$PC = rs1 \& \sim 1$$

The least significant bit of the target address is cleared in accordance with JALR semantics.

4.4.26 C.ADD

15:12	11:7	6:2	1:0
1001	rd/rs1	rs2	10

$$rd = rd + rs2$$

4.4.27 C.SWSP

Performs a stack-pointer-based word store.

15:13	12:9	8:7	6:2	1:0
110	uimm[5:2]	uimm[7:6]	rs2	10

$$MEM[x2 + uimm] = rs2$$

4.5 NOP

NOP is the abbreviation of No Operation. The instruction is executed by the processor but does not produce any change in the program's architectural state.

Within RISC-V, a separate base opcode is not allocated for NOP. The standard 32-bit NOP instruction:

ADDI x0, x0, 0

is encoded as this.

Bit equivalent:

31:20	19:15	14:12	11:7	6:0
000000000000	00000	000	00000	0010011

Hex equivalent:

0x00000013

Because writes to the `x0` register are ignored, no change occurs in architectural state.

Within the C extension there is also a 16-bit `C.NOP`:

0x0001

NOP instructions can be used in pipeline control, instruction alignment, debug, and some compiler operations.

4.6 Zicsr

The Zicsr extension defines the instructions used to access the Control and Status Register (CSR) space.

The CSR address space is 12 bits. Therefore:

$$2^{12} = 4096$$

possible CSR encodings exist. Not all of these are implemented CSRs; the processor implements only the CSRs that are defined in the specification and present in the design.

In all CSR instructions:

opcode = 1110011

is used.

Base encoding:

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1/uimm	funct3	rd	1110011

4.6.1 CSRRW

CSR Read and Write.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1	001	rd	1110011

Base behavior:

$$old = CSR$$

$$CSR = rs1$$

$$rd = old$$

When `rd=x0`, even if the CSR value is read, it is not required that the result be written to the register file.

4.6.2 CSRRS

CSR Read and Set.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1	010	rd	1110011

$$old = CSR$$

$$CSR = old \mid rs1$$

$$rd = old$$

If $rs1=x0$, no write to the CSR is performed and the instruction can be used solely as a read operation.

4.6.3 CSRRC

CSR Read and Clear.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1	011	rd	1110011

$$old = CSR$$

$$CSR = old \& \sim rs1$$

$$rd = old$$

Bits that are 1 in $rs1$ are cleared in the CSR.

4.6.4 CSRRWI

This is the immediate version of the CSRRW instruction.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	uimm[4:0]	101	rd	1110011

$uimm[4:0]$ is zero-extended and written to the CSR.

$$CSR = zeroextend(uimm)$$

4.6.5 CSRRSI

31:20	19:15	14:12	11:7	6:0
csr[11:0]	uimm[4:0]	110	rd	1110011

$$CSR = CSR \mid zeroextend(uimm)$$

When $uimm=0$, no write operation to the CSR is performed.

4.6.6 CSRRCI

31:20	19:15	14:12	11:7	6:0
csr[11:0]	uimm[4:0]	111	rd	1110011

$$CSR = CSR \& \sim zeroextend(uimm)$$

When $uimm=0$, no write to the CSR is performed.

4.7 Zifencei

The Zifencei extension defines the **FENCE.I** instruction.

The processor can perform instruction and data accesses over different paths. In the general ISA semantics, when a program modifies its own instruction memory region with data-stores, it is possible that instruction information previously present on the Fetch side is now invalid.

FENCE.I ensures that instruction-memory updates performed earlier can be seen by instruction fetch operations performed later.

microBee behavior

Zifencei opcode support is present. The **FENCE.I** instruction is accepted as valid but is implemented as a no-op in the current cacheless microarchitecture.

In the microBee memory map there is no normal data-store access to IMEM. There is also no dedicated hardware mechanism for instruction cache invalidate or instruction-fetch coherence. Therefore, executing **FENCE.I** does not create an additional operation on the pipeline or fetch side in the current design.

4.7.1 FENCE.I

Encoding:

31:20	19:15	14:12	11:7	6:0
000000000000	00000	001	00000	0001111

Hex equivalent:

0x0000100F

In the general ISA semantics, the purpose of the instruction can be thought of as:

Previous instruction memory updates

⇓

FENCE.I

⇓

New instruction fetches

In a general implementation, when **FENCE.I** is executed, temporary state on the instruction fetch side or the instruction cache may be cleared and the pipeline may begin fetch operations again. This behavior differs from microBee's current no-op implementation; the microBee box above should be taken as the reference.

This instruction is especially important in systems such as self-modifying code, bootloaders, program loaders, and JIT compilers, where changes are made to instruction memory during operation.

Chapter 5

Interrupts

5.1 What Is It?

An interrupt is an event that causes the processor to temporarily leave the normal program flow and transfer to a predefined handler. Sources are generally treated as independent of the normal instruction flow; they may be asynchronous sources such as a timer or an external line, or they may be sources triggered synchronously by software, such as a software interrupt.

Interrupt and exception use the trap mechanism in a similar way, but their sources differ.

- **Exception:** Occurs as a result of the executed instruction or operation. Illegal instruction or an access fault can be given as examples.
- **Interrupt:** An event that is treated as independent of the processor's normal instruction flow (timer, software, or an external source).

microBee operates only in Machine Mode [5]. When an interrupt is accepted, the processor leaves the normal program flow and goes to the trap handler address found in the `mtvec` register.

During an interrupt, the following are performed in essence:

- the return address to `mepc`,
- the interrupt cause to `mcause`,
- `mtval` is cleared (there is no extra fault information on an interrupt),
- the `MIE→MPIE`, `MIE=0` and `MPP=11` updates within `mstatus`

are done. The details of these updates are described in the later `mstatus` section.

When the handler completes, the normal program flow is returned to using the `mret` instruction.

5.2 What Is It For?

The main purpose of the interrupt mechanism is to prevent the processor from having to continuously check whether an event has occurred.

In a system that does not use interrupts, the processor may, for example, continuously check whether data has arrived over UART:

```
while (1) {
    if (uart_data_ready()) {
        read_uart();
    }
}
```

This structure is called **polling**.

Polling is a simple method, but a significant portion of processor time can be spent checking a peripheral even when no event has occurred.

When interrupts are used, the processor continues to do its normal work. When the event occurs, the related hardware produces an interrupt and the processor transfers to the handler.

In this way:

- peripherals do not have to be checked continuously,
- processor time can be used more efficiently,
- a faster response can be given to real-time events,
- event-driven operation can be applied in low-power systems.

5.3 Interrupt Sources in microBee

The microBee core supports three fundamental interrupt types:

Table 5.1: microBee interrupt sources and `mcause` values.

Interrupt	Source	mcause
Software Interrupt	<code>irq_software_i</code>	0x80000003
Timer Interrupt	<code>irq_timer_i</code>	0x80000007
External Interrupt	<code>irq_external_i</code>	0x8000000B

Software and timer interrupts are produced by the CLINT.

The external interrupt is generated by the PLIC.

5.4 Interrupt Acceptance Condition

A source merely being pending is not sufficient for the interrupt to be taken.

For the interrupt to be accepted, three fundamental conditions must be satisfied together:

- the related interrupt must be pending in `mip`,
- the related enable bit must be active in `mie`,
- the global interrupt enable bit `mstatus.MIE` must be active.

For example, for an external interrupt:

```
mip.MEIP = 1
mie.MEIE = 1
mstatus.MIE = 1
```

must hold.

When these conditions are satisfied, an interrupt request is asserted on the CSR side.

However, microBee does not take the trap immediately in the same cycle.

Note

mip: The Machine Interrupt Pending register. It shows which interrupt sources are in the pending state.
mie: The Machine Interrupt Enable register. It determines which interrupt types can be accepted by the core.
mstatus: One of the CSRs that hold the general processor state in Machine Mode. From the interrupt perspective, its most important bit is MIE.
MEIP: The Machine External Interrupt Pending bit, and it is found inside the `mip` register.
MEIE: The Machine External Interrupt Enable bit, and it is found inside the `mie` register.

5.4.1 mip Behavior in microBee

Inside microBee, the `mip` value read from software does not consist only of the writable pending bits held in the CSR.

The read value can generally be thought of as:

```
mip_read = mip_shadow | irq_level
```

That is, as long as the `irq_software_i`, `irq_timer_i`, or `irq_external_i` line is active, the related pending bit continues to appear as 1 in `mip`.

For this reason, writing 0 to the `mip` CSR while, for example, the external interrupt source is still active is not sufficient to clear the interrupt.

The actual source of the interrupt must be cleared on the CLINT, PLIC, or related peripheral side.

5.5 Interrupt Priority

More than one interrupt may be pending and enabled at the same time.

The priority order inside microBee is:

External > Software > Timer

That is, if external and timer interrupts are pending at the same time, the external interrupt is taken first.

If software and timer interrupts are pending at the same time, the software interrupt is processed first.

If a synchronous exception and an interrupt occur at the same time, the synchronous exception has higher priority.

In an exception trap, `mstatus.MIE` is cleared. As long as the interrupt source remains pending, the information is preserved on the CLINT, PLIC, or `mip` side. After `mret`, or when MIE is enabled again inside the handler, the interrupt can be taken at an appropriate time.

5.6 Precise Interrupt and Pipeline Drain

One of the important properties of the interrupt mechanism inside microBee is **precise interrupt** behavior.

The instructions in the pipeline are not discarded directly at the moment the interrupt request is asserted.

Instead, the processor:

1. stops issuing a new IMEM request,
2. waits for the instructions previously taken into the pipeline to complete,
3. accepts the interrupt trap when the pipeline has fully emptied.

At RTL level the fundamental condition can be thought of as follows:

$$\text{interrupt_trap_valid} = \text{csr_interrupt_request} \ \& \ \text{pipeline_empty}$$

Thanks to this method, instructions that have already started execution when the interrupt arrives are prevented from being left half-finished.

When the pipeline has emptied, the architectural state is now at a determined and consistent point.

The `pipeline_empty` condition for trap acceptance.

`pipeline_empty` is not a simple signal that only means the classical pipeline stages are empty. Inside the microBee RTL, fetch, memory, and operation states that have been presented but have not yet completed are also part of this condition. For this reason, the interrupt is accepted only when the core has truly reached a safe architectural boundary.

5.7 mepc During an Interrupt

One of the important differences between interrupt and exception is how the `mepc` value is formed.

In the interrupt case, because the pipeline is fully drained, there is no active faulting instruction.

In this case:

$$\text{mepc} \leftarrow \text{fetch_pc_q}$$

is performed.

Here `fetch_pc_q` is the address of the **next architectural instruction** that would have been executed if normal execution had continued.

After the handler completes, when `mret` is executed, the program continues from this address.

In the exception case, a particular instruction has produced an error. `mepc` holds the PC value of the instruction that caused the fault.

The faulting instruction is not retired in the normal way and its architectural result is not committed. The instruction is used as the trap source, and younger instructions are flushed and made invalid.

5.8 mstatus on Trap Entry

When an interrupt or exception is accepted, the interrupt control bits inside `mstatus` are updated automatically.

On trap entry:

$$\begin{aligned} \text{MPIE} &\leftarrow \text{MIE} \\ \text{MIE} &\leftarrow 0 \\ \text{MPP} &\leftarrow 2'b11 \end{aligned}$$

operations are performed.

Because microBee uses only Machine Mode, `MPP` remains `2'b11`.

On interrupt acceptance, additionally:

$$\text{mtval} \leftarrow 0$$

is performed. On exception acceptance, `mtval` takes the fault address, illegal instruction encoding, or related value according to the event type; it is not cleared on every trap.

5.9 mret

When the interrupt or exception handler completes, the `mret` instruction is used to return to the normal program.

When `mret` is executed:

$$\begin{aligned} \text{PC} &\leftarrow \text{mepc} \\ \text{MIE} &\leftarrow \text{MPIE} \\ \text{MPIE} &\leftarrow 1 \\ \text{MPP} &\leftarrow 2'b11 \end{aligned}$$

operations are performed.

In this way the global interrupt state from before the trap is restored.

5.10 Nested Interrupt

Inside microBee there is no separate hardware lock that specially prevents a nested interrupt.

On a normal trap entry:

$$\text{mstatus.MIE} = 0$$

is performed, so a new interrupt is not taken automatically while the handler is running.

However, if the handler software again:

```
mstatus.MIE = 1
```

then another interrupt can be accepted.

The pipeline drain condition is applied again for the new interrupt.

However, when a nested interrupt is taken:

- mepc,
- mcause,
- the related trap state

are written again.

For this reason, nested interrupt use must be managed carefully by software.

5.11 mtvec

mtvec determines at which address the handler is found when a trap occurs.

microBee supports only **Direct Mode**.

The handler address is computed as:

```
PC <- {mtvec[31:2], 2'b00}
```

If an attempt is made to write a different value to the mtvec[1:0] field, these bits are forced to zero with WARL behavior.

That is:

```
mtvec[1:0] = 00
```

remains.

Writing vectored mode does not form an illegal instruction. The mode bits are ignored and Direct Mode is used.

5.12 CLINT

CLINT (historically named Core-Local Interruptor) is the local software and timer interrupt unit inside microBee.

Inside microBee it produces two fundamental interrupt types:

- Software Interrupt
- Timer Interrupt

The CLINT base address is used as:

```
0x40000000
```

5.12.1 Software Interrupt

The software interrupt is generated through the MSIP register.

The register is at address:

```
MSIP @ 0x40000000
```

When:

is written to bit 0, the software interrupt becomes pending.

To clear the interrupt, the same bit is written:

0

For a machine-mode software interrupt to be accepted:

- `mie.MSIE = 1`,
- `mstatus.MIE = 1`

must hold.

When the software interrupt is accepted:

`mcause = 0x80000003`

holds.

5.12.2 Timer Interrupt

Inside microBee, `mtime` and `mtimecmp` are 64-bit registers.

Because RV32 is used, these registers are accessed through two 32-bit MMIO registers.

Table 5.2: CLINT timer register addresses.

Register	Offset	Address
MTIMECMP LO	0x400	0x40000400
MTIMECMP HI	0x404	0x40000404
MTIME LO	0xFF8	0x40000FF8
MTIME HI	0xFFC	0x40000FFC

The timer interrupt condition occurs when:

$$mtime \geq mtimecmp$$

For the timer interrupt to be accepted:

- `mie.MTIE = 1`,
- `mstatus.MIE = 1`

must hold.

After the interrupt is taken, the handler generally writes the `mtimecmp` value to a later time in order to determine the next timer event.

The timer interrupt cause is the value:

`mcause = 0x80000007`

5.13 PLIC

PLIC is a Platform-Level Interrupt Controller structure.

It is used to collect interrupts coming from peripherals and hardware sources outside the core, to prioritize them, and to convey them to the core over a single external interrupt line.

Inside the microBee PLIC there are:

- 16 real external interrupt sources,
- 3-bit priority,
- threshold,

- enable mask,
- claim/complete mechanism,
- four different trigger modes

The PLIC base address is used as:

0x40001000

Table 5.3: PLIC configuration addresses and access states.

Offset	Absolute address	Register	Access
0x000	0x40001000	CAPABILITY	RO
0x004	0x40001004	PRIORITY[1]	RW
0x008	0x40001008	PRIORITY[2]	RW
0x00C	0x4000100C	PRIORITY[3]	RW
0x010	0x40001010	PRIORITY[4]	RW
0x014	0x40001014	PRIORITY[5]	RW
0x018	0x40001018	PRIORITY[6]	RW
0x01C	0x4000101C	PRIORITY[7]	RW
0x020	0x40001020	PRIORITY[8]	RW
0x024	0x40001024	PRIORITY[9]	RW
0x028	0x40001028	PRIORITY[10]	RW
0x02C	0x4000102C	PRIORITY[11]	RW
0x030	0x40001030	PRIORITY[12]	RW
0x034	0x40001034	PRIORITY[13]	RW
0x038	0x40001038	PRIORITY[14]	RW
0x03C	0x4000103C	PRIORITY[15]	RW
0x040	0x40001040	PRIORITY[16]	RW
0x100	0x40001100	PENDING	RO
0x104	0x40001104	SOURCE_MODE	RW
0x108	0x40001108	INPUT_STATE	RO
0x10C	0x4000110C	RESET_LEVEL	RO
0x180	0x40001180	ENABLE	RW
0x300	0x40001300	THRESHOLD	RW
0x304	0x40001304	CLAIM	RO (side-effecting)
0x308	0x40001308	COMPLETE	WO

Note

RO Read-only. A write is ignored or a bus response is produced according to the register; the architectural state does not change (CAPABILITY, PENDING, INPUT_STATE, RESET_LEVEL).

RW Read and write. The upper bits of the PRIORITY[i] and THRESHOLD fields are RAZ/WI (3-bit priority). ENABLE bit 0 is reserved; bit k enables source k . SOURCE_MODE is 2 bits for each source: $((i-1)*2) + :$ 2.

RO (side-effecting) — CLAIM A read returns the ID of the highest active source (or 0 if none); at the same time pending is cleared and the source becomes in-service. `irq_external_o` falls if there is no other active source.

WO — COMPLETE The written ID is completed if it is in-service. A read returns 0. On a level source, pending can be established again if the line is still active; therefore the peripheral must be cleared first, then COMPLETE must be written.

PENDING Bit 0 is reserved; bit k is the pending state of source k . It is not cleared directly by software; the gateway / CLAIM path is used.

PRIORITY formula PRIORITY[i] @ $0x40001000 + 0x004 + 4(i-1)$, $i = 1..16$. Priority 0 does not enter selection; enable=1 and priority > threshold are also required for selection.

5.13.1 Interrupt ID

Inside microBee there are 16 real external interrupt sources:

`ext_irq_i[15:0]`

The mapping is:

$$\text{ext_irq_i}[n] \rightarrow ID = n + 1$$

That is:

- `ext_irq_i[0]` → ID 1,
- `ext_irq_i[15]` → ID 16.

ID 0 is not a real interrupt source and is reserved in the sense of:

no interrupt

5.13.2 Priority and Enable

There is a 3-bit priority value for each external interrupt source.

Therefore the priority range is:

0...7

A source whose priority value is 0 does not enter selection and behaves practically as if it were disabled.

However, the priority value merely being greater than 0 is not sufficient.

For a source to be selectable by the PLIC:

- the related source must be enabled,
- the priority value must be greater than 0,
- the priority value must be greater than the threshold value.

That is, the fundamental selection condition is:

```
enable[i]    = 1
priority[i]  > 0
priority[i]  > threshold
```

If enable is off, the source cannot be selected even if the priority value is 7.

If more than one interrupt with the same priority value is pending at the same time, the **source that has the lower ID value** is selected with priority.

5.13.3 Trigger Modes

There is a 2-bit `SOURCE_MODE` field for each PLIC source.

For source *i* the related mode field is held in the bits:

$((i-1)*2) +: 2$

Supported modes:

Code	Mode
00	Rising edge
01	Falling edge
10	Active-low level
11	Active-high level

5.13.4 Claim and Complete

When an external interrupt is generated by the PLIC, the handler reads the **CLAIM** register in order to learn which source produced the interrupt.

Inside microBee it is at address:

CLAIM @ 0x40001304

CLAIM gives the ID value of the highest-priority pending interrupt.

After the handler has cleared the interrupt cause on the related peripheral, the same ID value is written to the register:

COMPLETE @ 0x40001308

The fundamental flow is:

Pending -> Priority Selection -> CLAIM -> Handler -> COMPLETE

Level-triggered interrupt.

On a level-triggered interrupt source, merely writing **COMPLETE** is not sufficient.

The interrupt cause on the peripheral must be cleared first.

If **COMPLETE** is written while the source line is still at the active level, the PLIC can see the source as pending again and the same interrupt can be taken again.

For this reason, the fundamental order for a level-triggered interrupt should be applied as:

CLAIM -> Peripheral Clear -> COMPLETE

5.14 Shadow Register Bank

There is optional shadow register bank support inside microBee.

When **SHADOW_BANK_ENABLE = 1**, the active register bank switches from the normal bank to the shadow bank on every trap entry.

This behavior is the same for:

- interrupt,
- synchronous exception

Inside the shadow bank:

- a second register set is found for **x1-x31**,
- **x0** keeps hardwired zero behavior in every case.

The contents of the normal register bank are not copied to the shadow bank.

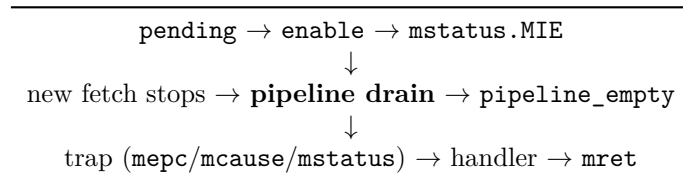
Only the active register bank is changed.

In this way, some of the software save/restore (register save/restore) operations needed to save the normal program's register values when the handler starts running can be reduced.

When **mret** is executed, the normal register bank is selected again.

5.15 Interrupt Resolution Mechanism

The processing of an interrupt from beginning to end inside microBee can be summarized with the following flow. The distinguishing point: the pipeline is not flushed at the moment the interrupt arrives; it is first **drained**, then the trap is accepted.



Step by step:

1. The CLINT or PLIC makes the interrupt source pending.
2. The pending information is reflected in the related mip bit on the CSR side.
3. The related enable bit inside mie and mstatus.MIE are checked.
4. If there is more than one interrupt:

External > Software > Timer

the priority order is applied.

5. When the interrupt request is asserted, new instruction fetch operations are stopped.
6. Completion of the existing instructions in the pipeline is waited for.
7. When pipeline_empty holds, the interrupt trap is accepted.
8. mepc is updated with the next architectural PC.
9. mcause is written according to the interrupt type.
10. If it is an interrupt, mtval = 0 is performed.
11. The MPIE <- MIE, MIE <- 0 operations are performed.
12. If the shadow bank is active, the active register bank switches to the shadow bank.
13. PC is directed to the mtvec address.
14. The handler services and clears the interrupt source.
15. If it is an external interrupt, PLIC CLAIM is read, the peripheral is cleared, and COMPLETE is written.
16. mret is executed.
17. A return is made to the normal register bank, MIE is restored, and the program continues from the mepc address.

5.16 Interrupt and Exception Difference

Although interrupt and exception use the same trap infrastructure, their pipeline behaviors are not the same.

In the interrupt case, microBee first waits for the pipeline to empty.

For this reason, at the moment the interrupt is accepted:

- IF/ID,
- ID/EX,
- EX/MEM,
- MEM/WB,
- fetch queue,
- outstanding fetch,
- outstanding memory operations,
- posted operations,
- MUL/DIV operations

are completed or in the empty state.

Therefore interrupt acceptance is not carried out in the form of instantaneously flushing the active pipeline stages as in branch misprediction.

The pipeline is first **drained**, then PC is directed to mtvec.

In the exception case, the fault can appear while a particular instruction is being executed.

The faulting instruction becomes the trap source but is not retired in the normal way and its architectural result is not committed.

mepc holds the PC value of the faulting instruction and younger instructions are flushed and made invalid.

This difference is an important part of precise interrupt and precise exception behavior.

5.17 Student Setup Example (PLIC source 1)

When the system starts running after reset, the handler address, PLIC settings, and CSR enable bits must first be set up in order to use the interrupt.

This example enables the `ext_irq_i[0]` line (PLIC ID 1) in active-high level mode, assigns priority=2 and threshold=0, and writes the handler address to `mtvec`.

Example assumptions.

- This example assumes a **special configuration**: `SHADOW_BANK_ENABLE=1`. In the REF32 profile, `SHADOW_BANK_ENABLE=0`; in REF32 the handler needs software save/restore for `t0/t1`.
- During setup, mode/priority are written before the source is yet enabled; enable is written last.
- The interrupt arrives when `ext_irq_i[0]=1` (the TB in simulation, or a connected peripheral on the FPGA).
- Level source: the handler must first perform the peripheral clear that lowers the line, then write `COMPLETE`. The handler below shows `CLAIM+COMPLETE` for teaching; for a real UART etc., peripheral clear must be added.

```
# -----
# microBee student example: PLIC ID1 (ext_irq_i[0]) setup + handler
# Addresses: CLINT 0x40000000, PLIC 0x40001000
# -----

.section .text
.globl _start
_start:
# 1) Write the handler address to mtvec (Direct mode)
la    t0, irq_handler
csrw mtvec, t0

# 2) PLIC: PRIORITY[1] = 2 @ 0x40001004
li    t0, 0x40001004
li    t1, 2
sw    t1, 0(t0)

# 3) PLIC: THRESHOLD = 0 @ 0x40001300
li    t0, 0x40001300
sw    x0, 0(t0)

# 4) PLIC: SOURCE_MODE source1 = 11 (active-high) @ 0x40001104
# Source1 bits [1:0]; leave other sources at 00.
li    t0, 0x40001104
li    t1, 0x3
sw    t1, 0(t0)

# 5) PLIC: ENABLE bit1 (ID1) @ 0x40001180
li    t0, 0x40001180
li    t1, 0x2          # bit 1 = source 1
sw    t1, 0(t0)

# 6) Core: mie.MEIE = 1 (bit 11)
li    t0, (1 << 11)
csrs mie, t0

# 7) Core: mstatus.MIE = 1 (bit 3) -- open last
li    t0, (1 << 3)
csrs mstatus, t0

# Main program: loop until the interrupt arrives
```

```

main_loop:
    j    main_loop

# -----
# Code that runs at interrupt time (mtvec target)
# -----

    .align 4
irq_handler:
    # In the special configuration with shadow bank=1, t0/t1 are in a
    # separate context.
    # For REF32 (shadow=0) add save/restore first.

    # Read CLAIM -> source ID (expected: 1)
    li    t0, 0x40001304
    lw     t1, 0(t0)          # t1 = claimed ID

    # (Optional) counter: show in DMEM that an irq arrived
    # li t0, 0x20000000 ; lw t2,0(t0) ; addi t2,t2,1 ; sw t2,0(t0)

    # On a level source: clear the peripheral IRQ cause here
    # (UART/GPIO register clear). Otherwise, after COMPLETE
    # the same interrupt can become pending again.

    # COMPLETE: write the claimed ID
    li    t0, 0x40001308
    sw     t1, 0(t0)

    mret                                # return to mepc, MIE is restored

```

Summary of the setup order:

1. `mtvec` $\leftarrow$ `irq_handler`
2. `PRIORITY[1]=2`, `THRESHOLD=0`, `mode=active-high`
3. `ENABLE` bit1
4. `mie.MEIE`, then `mstatus.MIE`
5. `trap` $\rightarrow$ handler when `ext_irq_i[0]` rises
6. `CLAIM` $\rightarrow$ (peripheral clear) $\rightarrow$ `COMPLETE` $\rightarrow$ `mret`

5.18 Software Interrupt Example

CLINT MSIP can be used to try this without an external pin.

Example assumption — not REF32.

The example below is for the `SHADOW_BANK_ENABLE=1` special configuration; the handler does not perform save/restore for `t0/t1`. In the REF32 profile (`SHADOW_BANK_ENABLE=0`) the same handler must save the registers to the stack and restore them.

```

la    t0, sw_irq_handler
csrw mtvec, t0

li    t0, (1 << 3)          # mie.MSIE
csrs mie, t0

li    t0, (1 << 3)          # mstatus.MIE
csrs mstatus, t0

li    t0, 0x40000000        # CLINT MSIP
li    t1, 1

```

```

    sw    t1, 0(t0)           # Form a software interrupt

main_sw:
    j     main_sw

sw_irq_handler:
    li    t0, 0x40000000
    sw    x0, 0(t0)          # MSIP = 0
    mret

```

When the trap is accepted, `mcause = 0x80000003` holds.

5.19 Hardware Interrupt Flow (Summary)

External interrupt sources are conveyed to the microBee core through the PLIC. The setup and handler assembly are in the previous “Student Setup Example” section; here only the conceptual flow is summarized.

The `ext_irq_i[0]` line is seen inside the PLIC as `ID = 1`. After it is selected with the source enable, priority > 0 and priority $>$ threshold conditions, `irq_external_i = 1` holds; on trap acceptance `mcause = 0x8000000B` is formed.

Handler order: `CLAIM` $\rightarrow$ peripheral clear $\rightarrow$ `COMPLETE` $\rightarrow$ `mret`. On a level-triggered source the peripheral clear must be done before `COMPLETE`.

5.20 Summary

The microBee interrupt system uses a three-level structure that operates in Machine Mode:

External > Software > Timer

Software and timer interrupts are managed by the CLINT, and external interrupts by the PLIC.

During interrupt acceptance the pipeline is not flushed directly. First new fetch operations are stopped and the pipeline is fully drained. When the pipeline has emptied, the trap is taken and `mepc` holds the PC value of the next instruction to be executed.

On the PLIC side, the priority value alone is not sufficient for an external source to be selected; the source must be enabled and the priority value must be greater than the threshold value.

The `mip` value does not consist only of the writable pending bits inside the CSR. Active interrupt inputs are also ORed and reflected into the pending state that is read.

Thanks to this structure, interrupts are taken at a precise architectural boundary and when the handler completes, execution continues safely from the point at which it left off.

Chapter 6

General-Purpose Input/Output Ports (GPIO) and Peripherals

This chapter describes the native `tmcu_gpio` block found inside microBee, its memory-mapped register structure, and how it is used. At the end of the chapter, the native timer and UART peripherals found in the same MMIO address region and the AXI UART Lite structure that can be used on the FPGA side are briefly introduced.

6.1 GPIO Design

GPIO is found in the SoC (System on Chip) in the memory-mapped region that starts from the address:

`GPIO_BASE = 0x40002000`

When `GPIO_ENABLE=1`, the router directs load/store requests made to this address range to the GPIO unit.

In the REF32 / default SoC configuration:

- `NUM_BANKS = 1`,
- `GPIO_WIDTH = 32`,
- bank stride = 0x20 byte,
- `INPUT_VALID_MASK = 32'hFFFF_FFFF`,
- `OUTPUT_VALID_MASK = 32'hFFFF_FFFF`

are used.

Inside the microBee SoC, the `tmcu_gpio` block is instantiated primarily with the `BASE_ADDRESS` and `GPIO_WIDTH` parameters. The input and output masks are left at the module's default values.

`INPUT_VALID_MASK` and `OUTPUT_VALID_MASK` are independent parameters. In this way, different configurations in which particular GPIO bits are used only as input or only as output can be configured later.

In the REF32 profile, all 32 GPIO bits are valid from both the input and the output perspective.

6.1.1 GPIO Ports

The fundamental signals of the GPIO block on the SoC side:

Port	Meaning
<code>gpio_in_i[31:0]</code>	Physical GPIO inputs
<code>gpio_out_o[31:0]</code>	GPIO output data
<code>gpio_oe_o[31:0]</code>	Output enable, 1: output, 0: input/Hi-Z

The output and input paths generally operate as (bitwise AND / masking):

Signal	Computation
gpio_out_o	OUT & OUTPUT_VALID_MASK
gpio_oe_o	OE & OUTPUT_VALID_MASK
IN	gpio_in_sync_q & INPUT_VALID_MASK

Thanks to this structure, even if a value is written in the register to an invalid GPIO bit, the related mask is applied on the physical output side.

6.1.2 Input Synchronization

Signals coming from GPIO pins can be asynchronous with respect to the processor clock domain. Using an asynchronous signal directly inside synchronous logic can form a metastability risk.

For this reason, a 2-FF synchronizer is used on the microBee GPIO inputs.

The fundamental structure can be thought of as:

gpio_in_i -> FF1 -> FF2 -> gpio_in_sync_q

During reset both synchronizer registers are also cleared. For this reason, the true level of the GPIO input may not be seen immediately in the IN register after reset.

Typically, an input change becomes readable by software after approximately two clocks.

Narrow pulses.

The 2-FF synchronizer is not a pulse-capture circuit. If a pulse that lasts shorter than the processor clock occurs on a GPIO input, capturing that pulse is not guaranteed. For such a requirement, separate edge/pulse detection hardware should be used.

6.1.3 Reset Behavior

After reset:

OUT = 0

and:

OE = 0

are initialized to 0.

In this case the GPIO output drivers inside the SoC are off; the pins are intended as input / high impedance (Hi-Z). Seeing Hi-Z behavior on the actual FPGA pin also depends on the upper-level IOBUF / pin connection.

6.1.4 Bus Access

GPIO accesses are carried out over the microBee native valid/ready bus.

Addresses must be 32-bit word-aligned (4-byte aligned):

address[1:0] = 2'b00

When an unaligned or undefined register address is accessed:

rsp_err = 1

is produced.

6.2 GPIO Functions

GPIO can be used in three fundamental ways:

- **Output:** Driving an LED, relay, enable line, or a general digital output.
- **Input:** Reading a button, switch, or digital sensor level.
- **Atomic bit operations:** Changing particular output bits through the `OUT_SET`, `OUT_CLR`, and `OUT_TGL` registers.

For a bit that will be used as output, set `OE` to 1; for a bit that will be used as input, set `OE` to 0.

OE operations.

Although `SET`, `CLR`, and `TGL` registers exist for `OUT`, there are no similar registers for `OE`. Therefore, if a single `OE` bit needs to be changed, a read-modify-write operation must be performed.

The GPIO block does not produce a PLIC interrupt on its own. If an edge or level change of a GPIO pin is desired to form an interrupt, additional interrupt-generation logic outside GPIO must be used for this.

6.3 GPIO Address Map

The GPIO base address is defined as:

0x40002000

In the REF32 profile only bank 0 is found.

Table 6.1: GPIO register address map and access types.

Offset	Absolute address	Register	Access
0x00	0x40002000	IN	RO
0x04	0x40002004	OUT	RW
0x08	0x40002008	OUT_SET	WO, read 0
0x0C	0x4000200C	OUT_CLR	WO, read 0
0x10	0x40002010	OUT_TGL	WO, read 0
0x14	0x40002014	OE	RW
0x18	0x40002018	VALID	RO
0x1C	0x4000201C	—	Invalid

When the invalid 0x1C offset is accessed, `rsp_err` is produced.

6.3.1 IN

Address:

IN @ 0x40002000

Shows the synchronized state of the GPIO inputs.

The value that is read is:

`IN = gpio_in_sync_q & INPUT_VALID_MASK`

`IN` is used as read-only (RO). The RO definition in this book does not mean that a write will necessarily produce a bus error: writing to `IN` does not form a bus error; the write is a no-op and does not change the GPIO state.

6.3.2 OUT

Address:

OUT @ 0x40002004

It is the GPIO output data register.

The value written to the register determines the physical output level.

During a write:

```
apply_wstrb(OUT, wdata, wstrb)
```

the operation is applied.

In this way the write strobe information in byte or half-word store operations is preserved.

Then:

```
OUT = apply_wstrb(OUT,wdata,wstrb) & OUTPUT_VALID_MASK
```

is applied.

6.3.3 OUT_SET

Address:

OUT_SET @ 0x40002008

Makes the bits that are 1 in the written value 1 inside OUT.

```
OUT <- OUT | (wdata & OUTPUT_VALID_MASK)
```

`wstrb` is not applied on this register. The entire 32-bit `wdata` is evaluated.

For this reason, using SW when accessing this register is recommended.

6.3.4 OUT_CLR

Address:

OUT_CLR @ 0x4000200C

Clears the output bits that are 1 in the written value.

```
OUT <- OUT & (wdata & OUTPUT_VALID_MASK)
```

`wstrb` is not applied on this register either.

6.3.5 OUT_TGL

Address:

OUT_TGL @ 0x40002010

Toggles the output bits that are 1 in the written value.

```
OUT <- OUT xor (wdata & OUTPUT_VALID_MASK)
```

A bit that is 0 is made 1, and a bit that is 1 is made 0.

`wstrb` is not applied on this register either.

6.3.6 OE

Address:

OE @ 0x40002014

Determines whether each GPIO channel is used as input or output.

- OE[n]=1: the related GPIO drives an output,
- OE[n]=0: the related GPIO is intended as input (the SoC output driver is off; Hi-Z on the FPGA pin depends on the IOBUF connection).

OE uses the same write-strobe behavior as OUT.

That is:

apply_wstrb

is applied and then it is masked with OUTPUT_VALID_MASK.

6.3.7 VALID

Address:

VALID @ 0x40002018

When read, it returns the value:

OUTPUT_VALID_MASK

This register shows only the valid GPIO bits on the output side.

There is no separate register visible to software for INPUT_VALID_MASK.

Writing to the VALID register does not form a bus error. The write operation is accepted as a no-op.

6.3.8 Multiple GPIO Banks

The module is parameterized so that it can form more than one GPIO bank.

For bank b the base address is:

$$\text{GPIO_BASE}_b = 0x40002000 + b \times 0x20$$

In the REF32 profile:

NUM_BANKS = 1

so only bank 0 is used.

6.4 GPIO_ENABLE = 0 Behavior

When GPIO_ENABLE=0, the GPIO block is not selected as a valid target inside the router.

In this case, if the 0x40002000 address range has not been mapped to another unit, the request router selects the SEL_ERR target as a result of decode. SEL_ERR is the internal response selection that the router produces for an undefined/inaccessible address; it is reflected to the core as an access-fault.

The core processes this situation as:

- load access fault or
- store access fault

When the GPIO module is removed from synthesis, the SoC-side:

```
gpio_out_o
```

and:

```
gpio_oe_o
```

outputs are tied to zero.

6.5 Register Update Time

GPIO register writes occur on the clock edge on which GPIO accepts the request.

The fundamental accept condition can be thought of as:

```
request_fire
```

The OUT or OE register is updated on the related **posedge**.

On the same clock edge, **rsp_valid** is also formed.

That is, the GPIO register is updated on the clock edge on which the request is accepted; it does not wait for the response to arrive in the next cycle.

From the software perspective, when the store instruction has completed, the GPIO register state has been updated. The change on the physical FPGA pin, however, depends on the IOBUF and the pin path.

6.6 How Is GPIO Used?

The fundamental usage order:

1. Set the OE bits to 1 for the bits that will be used as output.
2. Determine the output level with OUT, OUT_SET, OUT_CLR, or OUT_TGL.
3. Set OE to 0 on the bits that will be used as input.
4. Obtain the input level by reading the IN register.

6.6.1 Example: LED Control with GPIO Bit 0

In the example below, GPIO bit 0 is made an output and the SET/CLR/TGL registers are used.

```
# GPIO base = 0x40002000
```

```
li    t0, 0x40002000
```

```
# OE @ 0x14: bit0 = 1 (output)
```

```
li    t1, 0x1
```

```
sw    t1, 0x14(t0)
```

```
# LED ON: write directly to OUT @ 0x04
```

```
sw    t1, 0x04(t0)
```

```
# LED OFF: OUT_CLR @ 0x0C
```

```
sw    t1, 0x0C(t0)
```

```
# LED ON: OUT_SET @ 0x08
```

```
sw    t1, 0x08(t0)
```

```
# LED TOGGLE: OUT_TGL @ 0x10
```

```
sw    t1, 0x10(t0)
```

6.6.2 Example: GPIO Bit 1 Input Read

Because there is no separate SET/CLR register for OE, a bit change is performed with read-modify-write.

```
li    t0, 0x40002000

# Make OE bit1 = 0, preserve the other bits
lw    t1, 0x14(t0)
andi  t1, t1, -3
sw    t1, 0x14(t0)

# Read the IN register
lw    t2, 0x00(t0)

# Isolate only bit1
andi  t2, t2, 0x2
```

After the last operation: if `t2=0`, the input is low; if `t2=2`, the input is high.

6.7 GPIO Verification Hints

During simulation or FPGA verification the following points can be checked:

- `gpio_out_o` and `gpio_oe_o` can be observed on the waveform.
- The output register change (`gpio_out_o/gpio_oe_o`) should be seen on the clock edge on which `request_fire` occurs. This is register-level behavior; the physical change on the FPGA pin is not guaranteed at the same time.
- An input change can be seen with approximately two clocks of delay because of the 2-FF synchronizer.
- Because the synchronizer registers are zero after reset, the first GPIO reads can be 0.
- Access to an undefined offset such as `0x4000201C` or to an unaligned address should produce `rsp_err`.
- Because `wstrb` is not applied on the `OUT_SET`, `OUT_CLR`, and `OUT_TGL` registers, a word store should be used.

6.8 Other Native Peripherals

Besides GPIO, other native peripherals are also found in the same `0x40000000` MMIO address region.

Unit	Base	PLIC ID	Role
CLINT	0x40000000	—	Software / timer interrupt
PLIC	0x40001000	—	External interrupt management
GPIO	0x40002000	—	General-purpose I/O
TIMER	0x40003000	2	Native 64-bit timer
UART	0x40004000	3	Native UART
MONITOR	0x40005000	—	Performance counters

6.9 Native Timer

Inside microBee there is a general-purpose native timer peripheral independent of the CLINT timer.

The base address is defined as:

`0x40003000`

Its main registers (64-bit fields are LO/HI pairs):

Register	Offset	Role
CTRL	0x00	Timer control
PRESCALE	0x04	Clock divide / prescale
COUNT_LO	0x08	Counter low 32 bits
COUNT_HI	0x0C	Counter high 32 bits
CMP0_LO	0x10	Compare 0 low 32 bits
CMP0_HI	0x14	Compare 0 high 32 bits
CMP1_LO	0x18	Compare 1 low 32 bits
CMP1_HI	0x1C	Compare 1 high 32 bits
PERIOD_LO	0x20	Period low 32 bits
PERIOD_HI	0x24	Period high 32 bits
STATUS	0x28	Status, RW1C
CAPABILITY	0x2C	Hardware capability information

The COUNT, CMP0, CMP1, and PERIOD fields are 64-bit; RV32 software accesses them through LO/HI MMIO registers.

The interrupt generated by the timer is connected inside the SoC to PLIC source ID 2:

$$\text{TIMER_IRQ_ID} = 2$$

This timer is separate from the `mtime/mtimecmp` structure inside the CLINT. While the CLINT produces a machine timer interrupt, the native timer produces an interrupt through the PLIC as a general-purpose peripheral.

6.10 Native UART

Inside microBee there is a native UART peripheral for basic serial communication.

The base address is defined as:

$$0x40004000$$

Its main registers:

Register	Offset	Role
DATA	0x00	TX/RX data
STATUS	0x04	UART status information
CTRL	0x08	UART control
BAUDDIV	0x0C	Baud-rate divider
IRQ_STATUS	0x10	Interrupt status
FIFO_LEVEL	0x14	FIFO occupancy level
CAPABILITY	0x18	Hardware capability information

The UART uses the basic 8N1 serial communication structure.

The baud rate is computed with the approximate formula specific to the microBee native UART implementation:

$$\text{Baud} \approx \frac{f_{clk}}{\text{BAUDDIV}}$$

This formula is not valid for AXI UART Lite.

The minimum value for BAUDDIV is 8.

For example, for a 50 MHz clock and 115200 baud:

$$\text{BAUDDIV} \approx \frac{50\,000\,000}{115\,200} \approx 434$$

a value of approximately 434 can be used.

The interrupt generated by the native UART is connected to PLIC source ID 3:

$$\text{UART_IRQ_ID} = 3$$

6.11 AXI UART Lite

In the FPGA implementation, an AXI UART Lite peripheral can also be used besides the native UART. In the reference FPGA structure, AXI UART Lite is mapped to the address:

0x60000000

(the same as the Chapter 2 laboratory flow).

This UART is a peripheral different from the microBee native UART block.

In the reference connection, the AXI UART Lite interrupt line can be connected to PLIC source ID 1.

Therefore, in the example FPGA system a connection of the form:

PLIC ID	Source
1	AXI UART Lite
2	Native Timer
3	Native UART

can be used.

Native and FPGA peripherals.

Native GPIO, native timer, and native UART are part of the microBee RTL design. AXI UART Lite, however, is an IP block added outside microBee inside the FPGA Block Design. For this reason, the presence and address of AXI UART Lite depend on the FPGA integration that is used.

The native GPIO pins must also be separately connected to physical FPGA pins inside the FPGA Block Design, and the related pin constraints must be defined.

6.11.1 AXI UART Lite register summary (PG142)

Table 6.2: AXI UART Lite register addresses and fundamental roles.

Offset	Absolute address	Register	Meaning
0x00	0x60000000	RX_FIFO	Read: RX character
0x04	0x60000004	TX_FIFO	Write: TX character
0x08	0x60000008	STAT	RX/TX FIFO and error states
0x0C	0x6000000C	CTRL	FIFO reset and interrupt enable

Table 6.3: AXI UART Lite STAT and CTRL bits (PG142).

Register	Bit	Meaning
STAT	0	RX FIFO valid data
STAT	1	RX FIFO full
STAT	2	TX FIFO empty
STAT	3	TX FIFO full
STAT	4	Interrupt enabled
STAT	5	Overrun error
STAT	6	Frame error
STAT	7	Parity error
CTRL	0	TX FIFO reset
CTRL	1	RX FIFO reset
CTRL	4	Interrupt enable

Baud is in the Vivado IP / BD setting (REF32: typical 115200 at 50 MHz); there is no baud register in software.

6.11.2 Application: setup from scratch, PLIC, AXI write/read

The example below sets the system up from the beginning:

1. Writes the handler address to `mtvec`,
2. Enables PLIC source 1 (AXI UART Lite) with priority / mode / enable,
3. Enables `mie.MEIE` and `mstatus.MIE` in the core,
4. Performs FIFO reset and interrupt enable with AXI UART Lite CTRL,
5. Writes a character over AXI (TX) and reads (RX),
6. If an interrupt arrives, performs CLAIM → RX drain → COMPLETE.

Connection assumption.

- AXI UART Lite @ 0x60000000, interrupt → `ext_irq_i[0]` → PLIC ID 1 (active-high).
- This example is the `SHADOW_BANK_ENABLE=1` special configuration; the handler does not perform save/restore for `t0/t1/t2`. In REF32 (`SHADOW_BANK_ENABLE=0`) software save/restore is required.
- The handler below is for teaching purposes: when the interrupt arrives, only the RX FIFO is drained. The AXI UART Lite interrupt can also come from other status/error conditions; in production code the `STAT` bits should also be examined.
- For RX, data must arrive from a host/terminal or a simulation BFM; in a TX-only test the RX loop can wait.

```
# -----
# microBee: AXI UART Lite @ 0x60000000 + PLIC source 1
# Setup from the very beginning, AXI write (TX) and read (RX)
# -----

.section .text
.globl _start
_start:
    li    sp, 0x2000FF00

    # ---- 1) Trap handler ----
    la    t0, irq_handler
    csrw  mtvec, t0

    # ---- 2) PLIC: source 1 (AXI UART Lite) ----
    # PRIORITY[1] = 2 @ 0x40001004
    li    t0, 0x40001004
    li    t1, 2
    sw    t1, 0(t0)

    # THRESHOLD = 0 @ 0x40001300
    li    t0, 0x40001300
    sw    x0, 0(t0)

    # SOURCE_MODE: source1 = 11 (active-high) @ 0x40001104
    # (write while the source is not yet enabled)
    li    t0, 0x40001104
    li    t1, 0x3
    sw    t1, 0(t0)

    # ENABLE bit1 (ID1) @ 0x40001180
    li    t0, 0x40001180
    li    t1, 0x2
    sw    t1, 0(t0)

    # ---- 3) Core interrupt enable ----
    li    t0, (1 << 11)          # mie.MEIE
    csrs  mie, t0
```

```

    li    t0, (1 << 3)          # mstatus.MIE
    csrs  mstatus, t0

    # ---- 4) AXI UART Lite init @ 0x60000000 ----
    li    s0, 0x60000000
    li    t1, 0x03              # CTRL: bit0 TX reset + bit1 RX reset
    sw     t1, 0x0C(s0)
    li    t1, (1 << 4)          # CTRL: IE = 1 (interrupt enable)
    sw     t1, 0x0C(s0)

    # ---- 5) Write to AXI: "Hi\r\n" ----
    li    a0, 'H'
    jal    ra, uart_putc
    li    a0, 'i'
    jal    ra, uart_putc
    li    a0, '\r'
    jal    ra, uart_putc
    li    a0, '\n'
    jal    ra, uart_putc

    # ---- 6) Read from AXI: at least one character (polling) ----
    jal    ra, uart_getc        # a0 = byte that was read
    # (optional) echo:
    # jal    ra, uart_putc

main_loop:
    j      main_loop

# -----
# a0 = character to send
# Wait until STAT bit3 (TX full)=0, then write TX_FIFO
# (the FIFO does not have to be completely empty; writing while full is SLVERR)
# -----
uart_putc:
    li    t0, 0x60000000
1:
    lw     t1, 0x08(t0)         # STAT
    andi   t1, t1, 0x08         # TX full?
    bnez   t1, 1b              # if full, wait
    sw     a0, 0x04(t0)         # TX_FIFO
    ret

# -----
# Wait for STAT bit0 (RX valid), read RX_FIFO -> a0
# -----
uart_getc:
    li    t0, 0x60000000
2:
    lw     t1, 0x08(t0)
    andi   t1, t1, 0x01         # RX valid?
    beqz   t1, 2b
    lw     a0, 0x00(t0)         # RX_FIFO
    andi   a0, a0, 0xFF
    ret

# -----
# Teaching handler: RX drain after CLAIM (only ID==1)
# -----
    .align 4
irq_handler:
    # CLAIM

```

```

    li    t0, 0x40001304
    lw     t1, 0(t0)           # t1 = ID (expected 1)
    beqz   t1, irq_done       # if 0, nothing to process

    # If ID == 1, UART Lite RX drain (required for a level IRQ)
    li     t0, 1
    bne    t1, t0, irq_complete

    li     t0, 0x60000000
drain_rx:
    lw     t2, 0x08(t0)
    andi   t2, t2, 0x01
    beqz   t2, irq_complete
    lw     t2, 0x00(t0)       # read RX / empty the FIFO
    j      drain_rx

irq_complete:
    fence
    li     t0, 0x40001308     # COMPLETE
    sw     t1, 0(t0)

irq_done:
    mret

```

Setup order summary:

1. mtvec ← handler,
2. PLIC: PRIORITY[1], THRESHOLD, mode=active-high, ENABLE bit1,
3. mie.MEIE, mstatus.MIE,
4. AXI CTRL: TX+RX FIFO reset (0x03), then IE,
5. TX: wait for STAT.TX_FULL=0 → write TX_FIFO,
6. RX: wait for STAT.RX_VALID → read RX_FIFO,
7. IRQ: CLAIM → RX drain → COMPLETE → mret.

Chapter 7

Configurations

microBee does not have a single fixed hardware configuration; it is a soft-core design that can be shaped with synthesis parameters.

These parameters are determined during synthesis or FPGA IP configuration. After the bitstream is generated they cannot be changed at runtime by software. For example `MUL_ENABLE` or `GPIO_WIDTH` cannot be changed at runtime.

In this document **REF32 profile** denotes a specific microBee configuration to be used in lectures, simulations, and FPGA integration. The values are consistent with the `tmcu_soc` defaults and the settings in the Arty A7 Block Design script (`scripts/create_arty_a7_system.tcl`). The address map is in Appendix A.

Parameters can generally be divided into two groups:

- **Fixed** — architectural features that have no configuration parameter in the RTL and remain the same in all microBee instances,
- **Configurable** — features that can be selected during synthesis or FPGA IP configuration.

7.1 Fixed Configurations

The following features cannot be enabled or disabled on `tmcu_core` / `tmcu_soc`.

Table 7.1: Fixed architecture independent of synthesis.

Feature	Behavior
Pipeline	Five stages, single-issue, in-order: IF-ID-EX-MEM-WB
Hart count	1. A second hart does not exist in the RTL.
Base ISA	RV32I + Zicsr + Zifencei [2]
Compressed instructions	RV32C is always enabled; there is no <code>C_ENABLE</code> parameter.
Privilege	Machine Mode only; <code>mtvec</code> Direct mode
Register file	32×32-bit, <code>x0</code> hardwired zero; FF array, no BRAM option
Memory organization	Harvard: separate IMEM and DMEM native valid/ready paths
Local DMEM store	Posted store: after the request is accepted the MEM stage proceeds without waiting for a response; load and MMIO/AXI store wait for a response
Observation ports	The <code>dbg_*</code> observation ports are enabled by <code>DEBUG_ENABLE</code> (Chapter 10)
IMEM/DMEM type	inside <code>tmcu_native_bram</code> , <code>ram_style</code> = "block"

7.1.1 RV32C

The fetch queue operates in 16-bit units (at 16-bit granularity). `tmcu_decompress` expands a 16-bit instruction to a common 32-bit internal form; the decoder proceeds along a single RV32I/M path [4].

There is no parameter on the hardware side that disables the C extension. The compiler can produce only 32-bit instructions with an option such as `-march=rv32im`; however the core continues to process incoming compressed instructions.

Multiply and divide/remainder support belonging to the M extension can be controlled separately with the `MUL_ENABLE` and `DIV_ENABLE` parameters [3].

7.1.2 Register file and LUT/FF

The “L / B” (LUT-FF or BRAM register file) in the technical profile code is not a parameter in this RTL. `tmcu_regfile` holds two FF banks (if the shadow bank parameter is enabled). IMEM and DMEM are marked to be inferred as block RAM. What is changed for area savings is not the register file type, but the memory WORDS values below.

7.2 Configurable Configurations

Configurable parameters are on `tmcu_soc` (FPGA IP) and `tmcu_core` (`tmcu_top` simulation). A disabled unit is removed from synthesis with `generate`. The router does not select that window; access produces an access fault (Chapter 6, same model as `GPIO_ENABLE=0`).

7.2.1 REF32 summary

Table 7.2: Units enabled in REF32 and core switches. `tmcu_soc` default unless otherwise stated.

Parameter	REF32	Note
<code>MUL_ENABLE</code>	1	2-cycle multiplier
<code>DIV_ENABLE</code>	1	Iterative divider, approximately 34 cycle latency
<code>BP_ENABLE</code>	1	2-bit saturating branch predictor
<code>BP_ENTRIES</code>	128	128-entry prediction table
<code>BP_GSHARE</code>	0	Bimodal PC-based indexing
<code>SHADOW_BANK_ENABLE</code>	0	SoC/FPGA. The <code>tmcu_top</code> default is 1.
<code>AXI_ENABLE</code>	1	0x60000000 window
<code>CLINT_ENABLE</code>	1	MSIP, <code>mtime</code>
<code>PLIC_ENABLE</code>	1	16 sources, 3-bit priority
<code>GPIO_ENABLE</code>	1	32 bit, 1 bank
<code>TIMER_ENABLE</code>	1	PLIC ID 2
<code>UART_ENABLE</code>	1	Native UART, PLIC ID 3
<code>MONITOR_ENABLE</code>	1	0x40005000
<code>TRACE_ENABLE</code>	0	SoC trace outputs not connected
<code>DEBUG_ENABLE</code>	1	<code>dbg_*</code> pipeline / CSR observation
<code>IMEM_WORDS</code> / <code>DMEM_WORDS</code>	16384	64 KiB + 64 KiB
<code>RESET_VECTOR</code> / <code>MTVEC_RESET</code>	0x80000000	IMEM base
FPGA clock / baud	50 MHz / 115200	BD / constraint; not an SoC parameter

7.2.2 MUL/DIV

`MUL_ENABLE` and `DIV_ENABLE` are independent of each other.

When `MUL_ENABLE=1` the multiply unit operates 2-cycle pipelined; while the unit is available a new MUL can be started, but in the single-issue core the EX stage can be stalled for the related instruction until the result is ready.

When `DIV_ENABLE=1` the divide unit is iterative (approximately 34 cycle latency); the EX stage is stalled during the operation.

When the related unit is disabled, instructions belonging to that group result in an illegal instruction trap [3].

When the related unit is disabled the execution block is removed from synthesis with a `generate` construct. Thus the basic hardware resources belonging to that unit are not present in the design. Shared logic belonging to decode and exception control continues to exist in the core. Software should not produce those instructions with `-march=rv32i` or `rv32ic`; otherwise a trap is taken.

7.2.3 Branch predictor

- `BP_ENABLE=0`: the table is not generated, every branch is predicted not-taken.

- **BP_ENTRIES:** Determines the number of entries in the branch prediction table. Each entry holds a 2-bit saturating counter. The index width is approximately $\lceil \log_2(\text{BP_ENTRIES}) \rceil$ bits. In practice powers of two are used.
- **BP_GSHARE=0:** index is PC-fold (bimodal). This is REF32.
- **BP_GSHARE=1:** index = PC-fold XOR zero-extended GHR. The GHR_WIDTH range must be $[1, \text{clog}_2(\text{BP_ENTRIES})]$.

In the REF32 profile a 128-entry bimodal branch predictor is used. Each entry holds a 2-bit saturating counter. A larger prediction table can increase accuracy; however it can impose extra cost on LUT/FF usage, place/route cost, and maximum frequency (Fmax). Therefore BP_ENTRIES should be chosen by evaluating performance and area results together. Predictor size and performance comparison are in Chapter 9.

7.2.4 Shadow bank

When SHADOW_BANK_ENABLE=1, at trap entry the shadow bank is selected instead of the normal register bank. Thus inside a normal interrupt handler, registers such as `t0`, `t1` may not additionally need to be saved to the stack.

However if nested interrupts will be accepted by enabling `mstatus.MIE` again inside the handler, this advantage is not sufficient by itself. Because a new trap can use the same shadow register space, software must itself provide the required register protection.

REF32 SoC and Arty A7 Block Design use SHADOW_BANK_ENABLE=0. The `tmcu_core` / `tmcu_top` module default is 1. When running Chapter 5/6 examples on FPGA with REF32, the handler must take caller-saved registers onto the stack. Assembly examples in these chapters that show SHADOW_BANK_ENABLE=1 are a special configuration; they are not the REF32 assumption.

7.2.5 IMEM / DMEM size and preload

IMEM_WORDS, DMEM_WORDS, IMEM_INIT_FILE, DMEM_INIT_FILE

In REF32 both memories have a capacity of 16384 words (64 KiB). Reducing them lowers the BRAM count; the software's `.text/.data` must fit. Linker script bases must remain the same as IMEM_BASE / DMEM_BASE.

If INIT_FILE is given, FPGA BRAM initial contents can be placed into the bitstream. When an init file is not used, program contents must be provided via simulation, a boot mechanism, or a different loading method.

7.2.6 PLIC

PLIC_ENABLE=0 disables the external interrupt: `irq_external_i` remains 0 in the core, the 0x40001000 window faults.

When enabled:

- **PLIC_SOURCES** — 16 in REF32; in this case the external input is `ext_irq_i[15:0]` wide. Bit n is source ID $n + 1$.
- **PLIC_PRIO_BITS** — 3 in REF32 (priority 0–7).
- When **IRQ_SYNC_ENABLE=1** each external interrupt source is passed through a 2-FF synchronizer. This structure reduces metastability risk on asynchronous inputs. For short edge-triggered pulses not to be lost, keeping the pulse duration long enough to be sampled reliably with the core clock is a safe design recommendation (for example a pulse lasting multiple core clocks). If asynchronous FPGA inputs are used, selecting **IRQ_SYNC_ENABLE=1** is recommended.
- **IRQ_MODE_RESET**, **IRQ_INITIAL_LEVEL** — trigger mode and level after reset. REF32 keeps both at 0.
- **TIMER_IRQ_ID=2**, **UART_IRQ_ID=3** — PLIC numbers of the native timer and native UART. AXI UART Lite is connected to ID 1 with `ext_irq_i[0]` in the BD.

PLIC ID 0 represents the “no interrupt” condition and is not used as a real interrupt source. Details in Chapter 5 and Appendix A.

7.2.7 GPIO

GPIO_ENABLE and GPIO_WIDTH are SoC parameters. REF32: enabled, 32 bit, single bank. INPUT_VALID_MASK and OUTPUT_VALID_MASK are two separate mask parameters; they are not additionally exported at the `tmcu_soc` level. In the REF32 instance the `tmcu_gpio` defaults are used: single bank and 0xFFFFFFFF for both masks.

When `GPIO_ENABLE=0` the GPIO peripheral block is removed from synthesis and the router does not select the `0x40002000` window. Access to this address results in an access fault. At the SoC level the GPIO outputs (`gpio_out_o`, `gpio_oe_o`) are pulled to zero. Usage is in Chapter 6.

7.2.8 AXI

When `AXI_ENABLE=1` the AXI4-Lite master bridge is enabled; native requests are converted to AXI transactions through this bridge. The window is with `AXI_BASE/AXI_MASK` (REF32: `0x60000000–0x6FFFFFFF`).

When `AXI_ENABLE=0` the AXI bridge is not active and accesses to the `0x60000000–0x6FFFFFFF` address window result in a fault. Unused SoC AXI outputs are pulled to appropriate constant values. AXI UART Lite on the FPGA is in this window; native UART (`0x40004000`) is a separate `UART_ENABLE` parameter.

The `AXI_WAIT` counter counts only wait cycles that occur while the AXI access path is active. When `AXI_ENABLE=0` such an AXI wait event does not occur.

7.2.9 CLINT, timer, native UART, monitoring

Each of these units is controlled with a separate enable parameter. In the REF32 profile all of these peripherals are active.

- If CLINT is disabled, a software/timer interrupt is not generated; there is no `mtime` MMIO. The core `irq_software_i` / `irq_timer_i` inputs are pulled to 0.
- If timer or native UART is disabled, the related interrupt is not forwarded to the PLIC input.
- `MONITOR_ENABLE=0` removes the monitoring block; `0x40005000` faults, and the `BUS_ERROR` counter is also absent (Chapter 8).

7.2.10 TRACE_ENABLE and DEBUG_ENABLE

`TRACE_ENABLE` is on the SoC wrapper. `DEBUG_ENABLE` is on both `tmcu_soc` and the core (it is forwarded to the core).

When `TRACE_ENABLE=0` the SoC-level `trace_*` outputs are constant (`trace_valid_o = 0`). Commit/trace information may still be generated internally within the core. In the `tmcu_top` simulation the `trace_*` ports are always external (there is no SoC wrapper).

`DEBUG_ENABLE` controls the pipeline PC, register write, and CSR snapshot/observation (`dbg_*`) ports. The default is 1 (BD waveform / lecture observation). When `DEBUG_ENABLE=0`:

- SoC and core `dbg_*` outputs are pulled to 0,
- in Vivado IP these ports are hidden in Customize IP (`enablement_dependency`),
- core execution behavior does not change (observation only).

Chapter 10.

7.2.11 Reset vector and address windows

`RESET_VECTOR` is the first fetch PC. `MTVEC_RESET` is the initial value to be loaded into the `mtvec` register after reset. Because microBee supports only Direct mode, the `mtvec[1:0]` bits are held as 00. REF32 sets both to `0x80000000`.

`IMEM_BASE`, `DMEM_BASE/DMEM_MASK` and `MMIO *_BASE/*_MASK` are router windows. Changing them requires updating the linker script, software constants, and the Appendix A tables together. Address windows should not be changed in lecture measurements; `*_ENABLE` is sufficient to disable a unit.

The `DMEM_BASE/DMEM_MASK` in the core also selects the local posted-store window. The SoC forwards these values to the core.

7.2.12 Single-parameter change principle

In a performance comparison one parameter is changed at a time (Chapter 9). Disabling MUL and enlarging the BP table at the same time hides which effect changed CPI.

7.3 Fixed Features and Frequently Confused Points

1. There is no switch that disables RV32C.
2. There is no parameter to place the register file in BRAM.
3. Clock frequency is not an SoC parameter; it is determined by the FPGA clocking structure and timing constraints.
4. `dbg_*` outputs are disabled with `DEBUG_ENABLE`; `TRACE_ENABLE` controls the SoC `trace_*` outputs.
5. The shadow-bank default of `tmcu_top` and `tmcu_soc` differs (1 versus 0).
6. Dual hart, FPU, and the vector extension are not present in this RTL.

DRAFT

Chapter 8

Monitoring and Performance Counters

8.1 What Is It?

The monitoring unit provides counting of micro-architectural events that occur while the processor is running. It is not sufficient for a processor only to run the program correctly. During design the following questions must also be answered:

- In how many cycles (clock cycles) did a program complete?
- How many instructions were retired?
- How long did the pipeline remain in the stall state?
- How accurately did the branch predictor predict?
- How much did the memory system stall the processor?
- How much were long-latency units such as MUL/DIV used?
- How many times did events such as interrupt or bus error occur?

Tracking this information from the waveform alone is not practical in long benchmark runs.

Therefore there are memory-mapped monitoring registers inside microBee that count processor events at the hardware level.

Software can read these counters with ordinary load instructions and measure system performance.

In this chapter **cycle**, unless otherwise stated, denotes a core clock cycle.

8.2 Counter Model

The monitoring unit:

```
MONITOR_ENABLE = 1
```

becomes active when this condition holds.

Micro-architectural events are stored in 32-bit counters.

In the REF32 profile, counters saturate at:

```
0xFFFF_FFFF
```

That is, after a counter reaches its maximum value it does not wrap back to zero.

For example:

```
counter = 0xFFFF_FFFF
counter + 1 -> 0xFFFF_FFFF
```

it remains as such.

This behavior prevents the measurement from being misinterpreted due to overflow.

8.3 MON_CTRL

The basic control register of the monitoring unit is the

MON_CTRL

register.

The offset is 0x000.

Counters are stored in two copies: the **live** copy holds the instantaneous count; the **shadow** copy is the frozen image of the live values at the **SNAPSHOT** instant.

Main control bits:

Bit	Field	Function
0	ENABLE	Runs the monitoring counters
1	FREEZE	Stops the counters at their current values
2	CLEAR	Clears the counters, W1P
3	SNAPSHOT	Copies live counters to the shadow area, W1P

8.3.1 ENABLE

When **ENABLE=1** the monitoring unit starts counting events.

When **ENABLE=0** the counters do not increment but they retain their current values.

This feature can be used to measure the performance of a specific code region.

For example:

1. Counters are cleared.
2. Monitoring is enabled (**ENABLE=1**).
3. The function to be measured is run.
4. Monitoring is frozen (**FREEZE=1**).
5. Counters are read by software.

8.3.2 FREEZE

When **FREEZE=1** the current value of the counters is preserved.

Even if the monitoring unit is in the enable state, new events are not counted.

With this, continued change of the measurement while multiple counters are being read can be prevented.

For example if first **CYCLES**, then **RETIRED** is read while the system continues to run, the two counters can represent different time points.

When **FREEZE** is used all counters remain fixed in the same measurement interval.

8.3.3 CLEAR

CLEAR has W1P, that is Write-One-Pulse, behavior.

When 1 is written to this bit the counters are cleared.

The bit does not stay 1; the write is treated as a pulse.

Basic usage:

```
MON_CTRL.CLEAR = 1
```

afterwards the counters are set to 0:

```
counter <- 0
```

8.3.4 SNAPSHOT

SNAPSHOT again has W1P behavior.

When a snapshot is taken, the then-current values of the live counters are copied to the shadow register area. Thus counter values belonging to the same time point can be read while monitoring continues.

By adding

+0x100

to the live counter offsets, the related shadow counter is read.

For example:

CYCLES = 0x004

then the snapshot counterpart:

CYCLES_SHADOW = 0x104

can be read at that address.

The advantage of the snapshot mechanism is that a consistent measurement can be taken without stopping monitoring.

8.4 Control Priority

If more than one control condition occurs in the same cycle, microBee applies the following priority:

Reset > Clear > Snapshot > Count

Counting is performed only when:

ENABLE = 1

FREEZE = 0

Therefore the general behavior depends on the:

ENABLE & !FREEZE

condition.

8.5 Monitoring Register Map

Live counters are in the memory-mapped register area between 0x0004 and 0x0064.

Snapshot copies of the same counters can be read by adding a +0x100 offset.

Table 8.1: Monitoring registers

Offset	Register	Access	Counted event
0x000	MON_CTRL	RW	Bit0 ENABLE, bit1 FREEZE, bit2 CLEAR (W1P), bit3 SNAPSHOT (W1P).
0x004	CYCLES	RO	Number of core cycles elapsed while ENABLE=1 and FREEZE=0.
0x008	RETIRED	RO	Architecturally retired instruction.
0x00C	STALL	RO	Pipeline stall cycle.

Offset	Register	Access	Counted event
0x010	FLUSH	RO	Redirect/flush event (trap/mret, JAL/JALR, mispredict recovery, exception).
0x014	BUBBLE	RO	Bubble cycle with no valid instruction in the pipeline.
0x018	HAZARD	RO	Hazard-unit activity (load-use stall or forwarding selection).
0x01C	LOAD_USE	RO	Load-use interlock occurrence.
0x020	BRANCH	RO	Conditional branch instruction count.
0x024	BRANCH_TAKEN	RO	Number of branches that result as taken.
0x028	MISPREDICT	RO	Incorrect branch prediction count, retire-aligned.
0x02C	IMEM_WAIT	RO	Instruction memory request/response wait cycles.
0x030	DMEM_WAIT	RO	Data memory request/response wait cycles.
0x034	AXI_WAIT	RO	Number of cycles waited on the AXI bridge.
0x038	BUS_ERROR	RO	Native IMEM/DMEM error response count.
0x03C	IRQ_TAKEN	RO	Number of interrupts accepted by the core.
0x040	ALU_USE	RO	Number of retired integer ALU instructions.
0x044	MUL_USE	RO	Number of retired multiply instructions.
0x048	DIV_USE	RO	Number of retired divide/remainder instructions.
0x04C	CAPABILITY	RO	Counter width, MAP revision 2 and support mask.
0x050	LOAD_COUNT	RO	Number of retired load instructions.
0x054	STORE_COUNT	RO	Number of retired store instructions.
0x058	BRANCH_STALL	RO	Fetch wait cycles after mispredict recovery.
0x05C	MEM_WAIT_TOTAL	RO	Number of cycles in which IMEM_WAIT or DMEM_WAIT is active.
0x060	PREDICT_CORRECT	RO	Correct branch prediction count, retire-aligned.
0x064	SPEC_REDIRECT	RO	Number of speculative fetch redirects made due to a taken prediction.
+0x100	Shadow copy	RO	Live counter image at the SNAPSHOT instant.

8.6 Meaning of the Basic Counters

8.6.1 CYCLES

CYCLES holds the total number of core cycles elapsed while **ENABLE=1** and **FREEZE=0**.

This counter is the basis of many performance metrics.

For example when CPI, IPC and stall rate are computed, **CYCLES** is used as the denominator or numerator.

8.6.2 RETIRED

RETIRED shows the number of instructions the core has architecturally retired.

Not every instruction taken into the pipeline is counted as retired.

For example:

- instruction flushed as a result of branch misprediction,
- instruction cancelled due to an exception,
- speculatively fetched wrong-path instruction

is not added to the retired counter.

Therefore RETIRED is used to measure the number of instructions architecturally completed by the program.

8.6.3 STALL

STALL counts cycles in which the pipeline could not advance.

Stall causes include:

- load wait,
- load-use hazard,
- MUL/DIV execution wait,
- off-core store wait,
- memory or bus response wait

A high stall count shows that one (or several) of the wait sources in the list frequently stopped the pipeline; by itself it does not mean only “waiting for data/memory”.

8.6.4 FLUSH

FLUSH counts cycles/events in which `pipeline_flush_redirect` or `exception_flush` is active in the RTL. This set includes trap/mret, JAL/JALR redirect, mispredict recovery and exception. Speculative prediction redirects go to the separate SPEC_REDIRECT counter.

A high flush count can indicate a control-flow intensive application, low prediction success, or frequent trap/exception.

8.6.5 BUBBLE

A bubble is a cycle or slot state in which there is no valid instruction in the pipeline.

A bubble is not the same thing as a stall.

A bubble can form as a result of a stall but a bubble can also occur for other reasons such as pipeline redirect or hazard control.

An increase in the bubble count causes the pipeline to move away from its theoretical throughput value.

8.6.6 HAZARD and LOAD_USE

HAZARD counts cycles in which the hazard unit is active: occurrence of a load-use stall or selection of operand forwarding. This is not a retired instruction count, it is a hazard-unit decision/activity counter.

LOAD_USE is a more specific subcase of this (the case where the load-use interlock is seen together with pipeline advance).

For example:

```
lw  x5, 0(x10)
add x6, x5, x7
```

in this code the second instruction needs the `x5` value that the first instruction will bring from memory.

If the data is not ready in time, a load-use interlock can occur.

This situation can be tracked from the LOAD_USE counter.

8.7 Branch Counters

To examine branch predictor performance, more than one counter should be used together.

8.7.1 BRANCH

BRANCH holds the number of conditional branch instructions executed.

This value can be used to understand the control-flow intensity of the program.

8.7.2 BRANCH_TAKEN

BRANCH_TAKEN holds the number of branches whose actual result is taken.

Taken rate:

$$\text{Taken Rate} = \frac{\text{BRANCH_TAKEN}}{\text{BRANCH}}$$

can be computed as shown.

8.7.3 MISPREDICT

MISPREDICT holds the number of branches incorrectly predicted by the branch predictor.

Misprediction rate:

$$\text{Misprediction Rate} = \frac{\text{MISPREDICT}}{\text{BRANCH}}$$

can be computed as shown.

8.7.4 PREDICT_CORRECT

PREDICT_CORRECT holds the number of correctly predicted branches.

Prediction accuracy:

$$\text{Prediction Accuracy} = \frac{\text{PREDICT_CORRECT}}{\text{PREDICT_CORRECT} + \text{MISPREDICT}}$$

is computed as shown.

When BP_ENABLE=1 the equality $\text{PREDICT_CORRECT} + \text{MISPREDICT} = \text{BRANCH}$ is expected on the retire-aligned conditional branch set; in this case

$$\text{Prediction Accuracy} = \frac{\text{BRANCH} - \text{MISPREDICT}}{\text{BRANCH}}$$

gives the same result.

8.7.5 SPEC_REDIRECT

SPEC_REDIRECT counts events in which the Fetch unit is speculatively redirected to the branch target because the predictor predicted the branch taken.

This event by itself is not an error.

If the prediction is correct, the speculative redirect yields a performance gain.

If the prediction is wrong, misprediction recovery is later required.

Therefore:

$$\text{SPEC_REDIRECT and MISPREDICT}$$

examining these counters together is useful for understanding predictor behavior.

8.7.6 BRANCH_STALL

BRANCH_STALL holds the cycles in which the Fetch side waited, after branch misprediction recovery, to form the correct instruction stream again.

This counter helps understand not only how many mispredictions occurred, but the actual cycle cost of incorrect predictions on the pipeline.

8.8 Memory Wait Counters

The memory system can directly affect pipeline performance.

Therefore instruction and data memory waits are held in separate counters.

8.8.1 IMEM_WAIT

IMEM_WAIT counts cycles in which an instruction fetch request has been formed but the required instruction has not yet been received.

If this value is high, instruction memory, the bus, or the fetch path may be limiting processor performance.

8.8.2 DMEM_WAIT

DMEM_WAIT holds data memory request/response wait cycles.

Especially in load operations a pipeline stall can occur while waiting for the memory response, so this counter can have a direct effect on CPI.

8.8.3 MEM_WAIT_TOTAL

MEM_WAIT_TOTAL measures cycles that have a memory wait on the IMEM or DMEM side in aggregate.

This counter:

$$\text{IMEM_WAIT} + \text{DMEM_WAIT}$$

is not a simple sum of that form.

If both IMEM and DMEM wait conditions exist in the same cycle, the cycle is counted only once.

The general logic is:

$$\text{MEM_WAIT_TOTAL} = \text{count}(\text{imem_wait_event} \mid \text{dmem_wait_event})$$

Here $\mid$ is the logical OR of the event signals; it is not the numerical sum of the counter values.

8.8.4 AXI_WAIT

AXI_WAIT counts cycles in which AXI accesses outside the core waited on the bridge.

This value can be used especially to see the effect of external peripheral or memory accesses on system performance.

8.9 BUS_ERROR and IRQ_TAKEN

8.9.1 BUS_ERROR

BUS_ERROR counts error pulses in the IMEM fetch response or DMEM memory response error pulses (RTL: `imem_bus_error_pulse` / `dmem_bus_error_pulse`).

This counter is one of the counters that is generally expected to be zero in a normal benchmark run.

An unexpected increase may require investigating situations such as:

- invalid MMIO address (access fault response),
- disabled peripheral access,
- external/native bus error response

Misaligned load/store accesses can be caught in the core as a misaligned exception and can produce a trap without going out to the bus; in this case the event is reflected on the exception/flush path instead of `BUS_ERROR`.

8.9.2 IRQ_TAKEN

`IRQ_TAKEN` holds not only pending interrupts, but the number of interrupts actually accepted by the core. This distinction is important.

An interrupt source can remain pending for a long time, but `IRQ_TAKEN` is incremented only when the trap is accepted.

8.10 Execution Unit Usage

`ALU_USE`, `MUL_USE` and `DIV_USE` counters hold retired instruction counts; they show the completed instruction/event count belonging to the related unit, not cycle usage.

For example:

- high `ALU_USE`: integer operation intensity,
- high `MUL_USE`: multiplication intensity,
- high `DIV_USE`: division/remainder intensity

can be interpreted as such.

Especially even if the `DIV_USE` count is low, because the latency value of `DIV` instructions is high, their effect on total CPI can be large.

Therefore instruction count and stall cycle count should be evaluated together.

8.11 Load and Store Counters

`LOAD_COUNT` holds the retired load instruction count, and `STORE_COUNT` holds the retired store instruction count.

Memory instruction rate:

$$\text{Memory Instruction Rate} = \frac{\text{LOAD_COUNT} + \text{STORE_COUNT}}{\text{RETIRED}}$$

can be computed as shown.

In applications where this value is high, the memory system becomes more important for performance.

8.12 The CAPABILITY Register

`CAPABILITY` reports to software which features the monitoring unit supports. The register contents include the counter width, the monitoring map revision value (revision 2 in REF32) and the supported counter mask.

Thus software can read from hardware whether a particular monitoring register exists, instead of assuming it as fixed.

8.13 Performance Measurements

Raw counter values by themselves are not always sufficient.

Many performance metrics are derived from these counters.

Table 8.2: Derived performance measurements

Measurement	Formula	Reporting rule
IPC	retired / cycles	Instructions completed per cycle.
CPI	cycles / retired	Average cycles required for one instruction.
Stall rate	stall / cycles	Shows how long the pipeline waited.
Branch accuracy	predict correct / (predict correct + mispredict)	Meaningful if the predictor is enabled.
Mispredict rate	mispredict / branch	Shows how many of the branches were incorrectly predicted.
Taken rate	branch taken / branch	Shows branch behavior.
IMEM wait rate	imem wait / cycles	Shows instruction memory effect.
DMEM wait rate	dmem wait / cycles	Shows data memory effect.
Memory instruction rate	(load count + store count) / retired	Shows the program's memory intensity.

8.13.1 IPC

IPC stands for Instructions Per Cycle.

$$\text{IPC} = \frac{\text{RETIRED}}{\text{CYCLES}}$$

The IPC value shows how many instructions the processor completed on average in one cycle.

For a single-issue pipeline such as microBee, in long workloads the IPC value approaches 1 in the ideal case. In short benchmarks a full 1 may not be expected due to initial fill and drain effects.

Stall, flush, memory wait or long-latency instructions lower the IPC value.

8.13.2 CPI

CPI stands for Cycles Per Instruction.

$$\text{CPI} = \frac{\text{CYCLES}}{\text{RETIRED}}$$

It is inversely related to IPC:

$$\text{CPI} = \frac{1}{\text{IPC}}$$

In long workloads, in the ideal single-issue case CPI approaches approximately 1.

An increase in CPI shows that the processor spent more cycles per instruction.

8.13.3 Stall Rate

How much of the pipeline's total execution time it spent in the stall state:

$$\text{Stall Rate} = \frac{\text{STALL}}{\text{CYCLES}}$$

can be computed as shown.

As a percentage:

$$\text{Stall Percentage} = \frac{\text{STALL}}{\text{CYCLES}} \times 100$$

can be used.

8.13.4 Branch Prediction Accuracy

Using the counters directly:

$$\text{Prediction Accuracy} = \frac{\text{PREDICT_CORRECT}}{\text{PREDICT_CORRECT} + \text{MISPREDICT}}$$

can be computed.

Even if this value is high, the branch instruction rate and the misprediction penalty value should also be looked at to understand the total performance effect of the branch predictor.

8.14 How Are Counters Interpreted Together?

Making a performance interpretation from a single counter is in most cases not correct.

For example if CPI is high the cause can be:

- memory wait,
- branch misprediction,
- DIV usage,
- load-use hazard,
- external bus wait

Therefore counters should be examined by relating them to each other.

For example:

CPI high + DMEM_WAIT high

then data memory latency can be an important bottleneck.

CPI high + MISPREDICT high

then the branch predictor or a branch-heavy workload should be examined.

CPI high + DIV_USE high

then the iterative divider may be increasing total execution time.

STALL high + LOAD_USE high

then compiler scheduling or the forwarding structure can be examined.

In this way the monitoring unit can be used not only to measure performance, but also to understand where the bottleneck in the micro-architecture comes from.

8.15 Points to Watch While Measuring

To be able to compare two different benchmark results, recording only CPI or IPC values is not sufficient.

The following information must also be kept together with the result:

- benchmark binary,
- benchmark input data,
- compiler version,
- compiler optimization flags,
- ISA profile used,
- core configuration,
- clock frequency,
- IMEM/DMEM wait-state behavior,
- AXI or external peripheral configuration.

The same C code can produce a different instruction count and different memory access behavior with different compiler flags.

Therefore the performance result should not be thought of as a value belonging only to the processor.

The compiler, memory system and benchmark together form the result.

8.16 Interpreting the CPI Target

The condition appearing in the decision matrix:

$$\text{CPI} > 0.8$$

is not a discriminating target for a single-issue core. microBee in steady state is typically at $\text{CPI} \approx 1$ or above; CPI going below 0.8 is not an expected situation.

For CPI, a lower value generally indicates better performance; however for a meaningful comparison on a single-issue architecture, raw counters and derived rates should be reported together.

Because microBee is in a single-issue structure, in long workloads ideal IPC is approximately 1, and ideal CPI is at approximately the 1 level. In short measurement windows fill/drain effects can create deviation.

Therefore when reporting the performance result only the target condition should not be given; raw values should also be shared.

Recommended reporting includes:

- total cycle,
- retired instruction,
- CPI,
- IPC,
- stall cycle,
- flush count,
- branch and misprediction count,
- IMEM/DMEM wait values

This approach enables understanding the processor's actual behavior instead of looking at a single performance metric.

8.17 Example Measurement Flow

Measuring a benchmark with the monitoring unit can generally be done as follows:

1. Zero the monitoring counters with CLEAR.
2. If needed, remove the FREEZE value.
3. Set the ENABLE bit.
4. Run the benchmark.
5. At the end of the benchmark take a SNAPSHOT or apply FREEZE.
6. Read the CYCLES and RETIRED counters.
7. Read the stall, branch, memory and execution counters.
8. Compute CPI, IPC and the other rates.
9. Record the results together with the benchmark and compiler information.

With this structure, different benchmarks or different micro-architectural changes can be compared on the same microBee configuration.

8.17.1 Example: writing the CYCLES counter to AXI UART Lite

The following example uses REF32 addresses:

- Monitoring base: 0x40005000, CYCLES offset 0x04
- AXI UART Lite: 0x60000000

Flow: UART TX init, CLEAR+ENABLE the counters, then ENABLE only, a short work loop, FREEZE, read CYCLES, print the value as hex to UART.

The reason for two separate MON_CTRL writes: in the control priority CLEAR is higher than counting. When 0x5 (ENABLE|CLEAR) is written the counters are cleared; then with 0x1, after the CLEAR pulse ends, only ENABLE=1 is left and the measurement interval is started.

```
# -----
# microBee: write the CYCLES counter via AXI UART Lite @ 0x60000000
# MONITOR @ 0x40005000
# -----

.section .text
.globl _start
_start:
    li    sp, 0x2000FF00

    # ---- AXI UART Lite init (polling TX) ----
    li    s0, 0x60000000
    li    t1, 0x03          # CTRL: TX+RX FIFO reset
    sw    t1, 0x0C(s0)
    sw    x0, 0x0C(s0)      # CTRL: interrupt off

    # ---- Monitoring: CLEAR then ENABLE ----
    # 0x5 = ENABLE|CLEAR: CLEAR has priority and clears
    # 0x1 = ENABLE only: starts the measurement interval
    li    s1, 0x40005000    # MONITOR base
    li    t1, 0x5           # bit0 ENABLE | bit2 CLEAR (W1P)
    sw    t1, 0(s1)
    li    t1, 0x1           # ENABLE=1, FREEZE=0
    sw    t1, 0(s1)

    # ---- Work to measure (example loop) ----
    li    t0, 1000
work_loop:
    addi  t0, t0, -1
    bnez  t0, work_loop

    # ---- FREEZE, read CYCLES ----
    li    t1, 0x3           # ENABLE | FREEZE
    sw    t1, 0(s1)
    lw    s2, 0x04(s1)      # s2 = CYCLES

    # ---- write "CYCLES=0x" ----
    li    a0, 'C'
    jal   ra, uart_putc
    li    a0, 'Y'
    jal   ra, uart_putc
    li    a0, 'C'
    jal   ra, uart_putc
    li    a0, 'L'
    jal   ra, uart_putc
    li    a0, 'E'
    jal   ra, uart_putc
    li    a0, 'S'
    jal   ra, uart_putc
    li    a0, '='
    jal   ra, uart_putc
    li    a0, '0'
    jal   ra, uart_putc
    li    a0, 'x'
    jal   ra, uart_putc

    # 8 hex digits, MSB first
```

```

    mv    a0, s2
    jal   ra, print_hex32

    li    a0, '\r'
    jal   ra, uart_putc
    li    a0, '\n'
    jal   ra, uart_putc

done:
    j     done

# a0 = 32-bit value -> 8 hex characters
print_hex32:
    addi  sp, sp, -8
    sw    ra, 4(sp)
    sw    s3, 0(sp)
    mv    s3, a0
    li    t3, 28                # nibble shift
hex_nibble:
    srl   t4, s3, t3
    andi  t4, t4, 0xF
    li    t5, 10
    blt   t4, t5, hex_digit
    addi  a0, t4, 55            # 10..15 -> 'A'..'F'
    j     hex_out
hex_digit:
    addi  a0, t4, 48            # 0..9 -> '0'..'9'
hex_out:
    jal   ra, uart_putc
    addi  t3, t3, -4
    bgez  t3, hex_nibble
    lw    s3, 0(sp)
    lw    ra, 4(sp)
    addi  sp, sp, 8
    ret

# a0 = character; wait until STAT bit3 (TX full)=0
uart_putc:
    li    t0, 0x60000000
1:
    lw    t1, 0x08(t0)          # STAT
    andi  t1, t1, 0x08          # TX full?
    bnez  t1, 1b
    sw    a0, 0x04(t0)          # TX_FIFO
    ret

```

Chapter 9

Preparing the Performance Report

Chapter 8 defines the structure of the monitoring unit, its counters, and the metrics that can be obtained from those counters. This chapter describes how that information is used in a real performance measurement.

The goal of a performance measurement is not only to obtain a single CPI or IPC value. The real goal is to understand why the processor exhibits a given performance.

For example, if one of two different microBee configurations is slower than the other, saying only:

$$\text{CPI} = 1.8$$

is not sufficient.

It is also necessary to examine whether the difference comes from:

- memory wait (IMEM_WAIT/DMEM_WAIT),
- a wrong branch prediction (mispredict),
- a pipeline stall,
- long-latency instructions such as DIV,
- or a different instruction stream produced by the compiler

For this reason a performance report generally consists of three parts:

- how the system is observed,
- the raw monitoring counters,
- the performance values derived from those counters, and commentary.

9.1 How Is the System Observed?

microBee performance can be observed with three basic methods:

- waveform and debug signals,
- memory-mapped monitoring counters,
- simulation log and trace outputs.

These methods are not alternatives to one another. Each answers a different question.

Table 9.1: microBee performance observation methods.

Method	What does it show?	What is it not suitable for?
Waveform / dbg_*	Cycle-level PC, the PC/valid state in pipeline stages, register write, and CSR changes	Summarizing a long benchmark result
MMIO monitoring	Total cycle, retired, stall, branch, and memory wait counts	Showing on which instruction a single stall occurred
Simulation log / trace	Commit trace, test result, and ISS comparison	Showing physical timing behavior on the FPGA

9.1.1 Waveform and Debug Signals

A waveform is used to understand why a short instruction stream behaves in a particular way. For example:

“Why did a bubble form after this load instruction?”

This question can readily be answered from the waveform.

There are various `dbg_*` signals inside microBee for observation.

The principal debug signals:

- `dbg_fetch_pc_o`,
- `dbg_if_pc_o`,
- `dbg_ex_pc_o`,
- `dbg_mem_pc_o`,
- `dbg_wb_pc_o`,
- `dbg_rf_we_o`,
- `dbg_rf_waddr_o`,
- `dbg_rf_wdata_o`.

In addition, CSR values such as `mstatus`, `mcause`, `mip`, `mcycle`, and `minstret` can also be observed.

A waveform is quite useful for understanding short-term behavior. However, analyzing a benchmark that lasts millions of cycles from a waveform is not practical.

9.1.2 Monitoring Counters

In long performance measurements, the monitoring unit is one of the basic methods.

The monitoring unit is located at address:

`0x40005000`

The values in the register table are offsets. The absolute address:

`absolute address = 0x40005000 + offset`

For example, because the offset for `CYCLES` is `0x04`, the absolute address is `0x40005004`.

During a basic measurement, software:

1. clears the counters,
2. enables monitoring,
3. runs the program to be measured,
4. freezes the counters or takes a snapshot,
5. reads the counter values.

For example, values such as:

- `CYCLES`,
- `RETIRED`,
- `STALL`,
- `BRANCH`,
- `MISPREDICT`,
- `IMEM_WAIT`,
- `DMEM_WAIT`

can be read directly by software.

An important advantage of this method is that it does not require a special debugger. Software can read the processor's monitoring counters and export the result over UART.

Note the UART output.

If monitoring values will be printed over UART, the counters must first be frozen or a snapshot must be taken.

Otherwise the instructions that run to print characters to UART, and UART waits, are also included in the monitoring counters. Thus the cost of the measured program and of the reporting code become mixed.

9.1.3 Simulation Log and Trace

microBee SoC tests can also be run without an FPGA.

In SoC tests used in the Verilator environment, software can use the same monitoring registers, and the output produced over AXI UART Lite can be seen on the simulation console.

This method can be used, before moving to an FPGA board, to:

- validate the monitoring software,
- check counter values,
- compare different core configurations

However, the simulation cycle count is not directly real time.

For example, the value:

CYCLES = 1 000 000

denotes one million architectural clock cycles.

If a theoretical run time is to be computed, the target clock frequency (the frequency modeled in simulation, or the FPGA clock frequency) must also be known.

9.2 Which Method Should Be Used When?

In practice the following order can be used:

1. First form the overall performance table with the monitoring counters.
2. If CPI or IPC differs from what is expected, look at which counters are high.
3. Once a likely bottleneck has been identified, inspect the relevant waveform.
4. Examine the relevant instruction stream at cycle level.
5. Finally run the same test again on the FPGA.

For example, if `DMEM_WAIT` is very high, instead of immediately inspecting the waveform of the entire benchmark, a small region where memory accesses are dense can be examined.

Similarly, if `LOAD_USE` is high, the dependent instructions that immediately follow load instructions can be checked on the waveform.

Finding the problem first with counters, then examining the cause with the waveform, is a more efficient method.

9.3 System Observation Tools

9.3.1 AXI UART Lite and Host Terminal

In the REF32 FPGA configuration, AXI UART Lite is located at address:

0x60000000

For a 50 MHz system clock, a typical baud value of 115200 can be used.

Using an interrupt is not required to export performance counters. Simple polling TX is sufficient.

The basic UART write operation:

1. `STAT.TX_FULL` is checked,
2. if the FIFO is full, one waits,
3. if the FIFO is not full, the character is written to `TX_FIFO`.

For example, the monitoring result can be printed line by line as follows:

CYCLES=0x.....
 RETIRED=0x.....
 STALL=0x.....

A complete example dump is in Section 9.6. In the report, the raw hex value should also be kept as much as possible.

9.3.2 Vivado Waveform

The Vivado waveform should be used to investigate at cycle level the cause of anomalies seen in the monitoring results.

For example, if the result:

LOAD_USE high

has been obtained, the pipeline behavior between the load instruction and the dependent instruction that follows it can be examined on the waveform.

The waveform does not replace the performance table. It is used to understand the cause of the problem shown by the counters.

9.3.3 Computation

The monitoring unit produces the raw counters.

IPC, CPI, and the ratios are computed from these values.

$$\text{IPC} = \frac{\text{RETIRED}}{\text{CYCLES}}$$

$$\text{CPI} = \frac{\text{CYCLES}}{\text{RETIRED}}$$

$$\text{Stall Rate} = \frac{\text{STALL}}{\text{CYCLES}}$$

It must be checked that the denominator is not zero.

In particular, if:

$$\text{RETIRED} = 0$$

then CPI cannot be computed.

This situation should not be treated as a performance result; why the program or the measurement setup did not retire an instruction should be investigated.

9.4 How Is the Performance Report Prepared?

To be able to compare two different performance results, the measurement conditions must be known.

Saying only:

“Configuration A is faster.”

is not sufficient.

CPI and IPC are dimensionless performance metrics; they do not require the same clock frequency for a microarchitectural comparison. The real condition is that the measurement window is taken with the same program, the same compilation, and the same memory/ISA model.

In the CPI/IPC tests being compared, as much as possible:

- the same binary,
- the same input,
- the same compiler,
- the same optimization flag,
- the same ISA,
- the same memory behavior

should be used.

When real execution time or throughput (seconds, work/second) is compared, the frequencies need not be the same; at different frequencies the time

$$t = \text{CYCLES} / f_{\text{clk}}$$

can be computed and compared. Raw CPI by itself does not mean time.

If a microarchitectural parameter is being compared, ideally only that parameter should be changed.

9.4.1 Identity Information

Every performance report should state on which system the measurement was made.

Table 9.2: Identity information that should be kept in the performance report.

Field	Information to record
Environment	Simulation or FPGA
Binary	ELF / HEX name and input data
Compiler	Compiler version and optimization flag
ISA	RV32I / RV32IM / RV32IMC etc.
Profile	REF32 or a different configuration used
Clock	Target clock frequency in FPGA or simulation; for time $t = \text{CYCLES} / f_{\text{clk}}$
Memory	IMEM/DMEM wait-state information
Core parameters	MUL, DIV, predictor, AXI etc.
MONITOR_ENABLE	Whether monitoring is on

9.4.2 Raw Counters

Raw values should be written into the report before performance commentary.

The offsets in the table are added to the 0x40005000 base. The absolute address for STALL is 0x4000500C, and for FLUSH it is 0x40005010.

At a minimum:

Table 9.3: Recommended basic monitoring counters in the performance report.

Register	Offset	Use
CYCLES	0x04	Total measurement cycle
RETIRED	0x08	Completed instruction
STALL	0x0C	Pipeline stall cycle
FLUSH	0x10	Redirect / flush event
BUBBLE	0x14	Cycle without a valid instruction
LOAD_USE	0x1C	Load-use interlock
BRANCH	0x20	Branch count
BRANCH_TAKEN	0x24	Taken branch count
MISPREDICT	0x28	Wrong prediction
IMEM_WAIT	0x2C	Instruction memory wait
DMEM_WAIT	0x30	Data memory wait
DIV_USE	0x48	Divide/remainder use

STALL, FLUSH, and BUBBLE are not interchangeable and are not summed to compute empty cycles.

In RTL each is a separate event bit; the monitoring unit can increment them independently in the same cycle. For example, while `pipeline_advance=1`, if a valid instruction has not progressed because of a decode/execute stall, both `STALL` and `BUBBLE` can increment in the same cycle. `BUBBLE` production is suppressed during redirect/exception; in contrast the `STALL` and `FLUSH` event bits are independent and can both be 1 in the same cycle.

`CYCLES` increments every cycle while monitoring is on. Therefore the equality:

$$\text{STALL} + \text{FLUSH} + \text{BUBBLE} = \text{CYCLES}$$

is not valid. For the same reason `IMEM_WAIT` and `DMEM_WAIT` are also not summed; for overlapping cycles there is `MEM_WAIT_TOTAL`.

As needed, counters such as:

- `PREDICT_CORRECT`,
- `AXI_WAIT`,
- `BUS_ERROR`,
- `IRQ_TAKEN`,
- `LOAD_COUNT`,
- `STORE_COUNT`

can also be added to the report.

9.4.3 Derived Metrics

The following basic metrics can be computed from the raw values:

Table 9.4: Basic metrics used in the performance report.

Metric	Formula
IPC	retired / cycles
CPI	cycles / retired
Stall rate	stall / cycles
Mispredict rate	mispredict / branch
Taken rate	branch taken / branch
IMEM wait rate	imem wait / cycles
DMEM wait rate	dmem wait / cycles

If the branch count is zero, branch-based rates should not be computed.

9.4.4 How Are CPI and IPC Interpreted?

microBee is a single-issue core.

In an ideal continuous instruction stream:

$$\text{IPC} \approx 1$$

$$\text{CPI} \approx 1$$

is expected.

Pipeline stall, memory wait, branch recovery, and long-latency instructions lower IPC and raise CPI.

For this reason, for CPI in general it can be said:

a lower value is better

However, CPI by itself does not show the cause of the bottleneck.

9.5 How Are Counters Interpreted Together?

During performance analysis, counters should be examined together with one another, not one by one. For example, if:

CPI high + DMEM_WAIT high

then data memory accesses may be an important bottleneck.

If:

CPI high + MISPREDICT high

then the branch predictor or a branch-dense program flow should be examined.

If:

CPI high + DIV_USE high

then divider latency may be affecting the total run time.

If:

STALL high + LOAD_USE high

then load-use dependency and compiler scheduling can be examined.

None of these interpretations should be made by looking at only a single counter value.

9.6 Example Performance Report

The following values are given to show the report format. They should not be thought of as a real benchmark result.

It is assumed that the counters were frozen after the program completed.

9.6.1 Host Output

```
CYCLES=0x00000B18
RETIRED=0x000007E4
STALL=0x00000140
FLUSH=0x00000008
BRANCH=0x000003E8
MISPREDICT=0x00000004
IMEM_WAIT=0x00000000
DMEM_WAIT=0x00000000
LOAD_USE=0x00000000
DIV_USE=0x00000000
```

Decimal values:

Table 9.5: Decimal equivalents of the example monitoring output.

Register	Value
CYCLES	2840
RETIRED	2020
STALL	320
FLUSH	8
BRANCH	1000
MISPREDICT	4
IMEM_WAIT	0
DMEM_WAIT	0

Derived values:

$$\text{IPC} = \frac{2020}{2840} = 0.711$$

$$\text{CPI} = \frac{2840}{2020} = 1.406$$

$$\text{Stall Rate} = \frac{320}{2840} = 0.113$$

$$\text{Mispredict Rate} = \frac{4}{1000} = 0.004$$

In this example IMEM_WAIT, DMEM_WAIT, LOAD_USE, and DIV_USE are zero. That is, this dump does not support a claim of memory wait, load-use, or divide latency.

CPI being above 1 is consistent with the presence of stall and flush events; however, because counters can overlap, how much of the CPI difference comes from stall or flush cannot be separated from these numbers alone. BRANCH_STALL is not present in this example, and the loop of 1000 is not a real benchmark.

Commentary should therefore remain an example of format. A real bottleneck claim rests on a controlled comparison such as Section 9.7.

9.7 Configuration Comparison

The comparison was made on the Embench-IoT suite (19 benchmarks). Environment: RV32IMC, MONITOR_ENABLE=1, 50 MHz cycle counting. The reported CPI and IPC are weighted values over the suite.

Rule: in a run only one microarchitectural parameter changes. The binary, compiler, and memory model remain the same. Otherwise it cannot be known which change the gain came from.

The percentage difference (for example for CPI) is reported as follows:

$$\frac{\text{CPI}_{\text{yeni}} - \text{CPI}_{\text{eski}}}{\text{CPI}_{\text{eski}}} \times 100$$

9.7.1 Why Is Only One Parameter Changed?

In an intermediate run, store-related core stalling was disabled. DMEM_WAIT dropped by approximately 185 million cycles. In the same run the branch predictor remained effectively off (PREDICT_CORRECT=0, MISPREDICT=0, SPEC_REDIRECT=0). Branch-stall increased by almost the same amount.

Result: even though the memory side was corrected, weighted CPI went from approximately 2.27 to 2.29. The effects of the two changes masked each other. When the predictor was enabled again, the memory gain became visible on CPI.

For this reason the memory experiment and the predictor experiment are separate rows in the tables.

9.7.2 Experiment 1: predictor capacity (GShare off)

Keeping the same memory/LSU configuration, only BP_ENTRIES was changed. GShare is off. The sweep started at 8 entries, and the suite-level results favored the 128-entry bimodal configuration. The REF32 profile default is this choice (BP_ENTRIES=128).

Table 9.6: Embench weighted results, GShare off, only BP_ENTRIES changes.

BP_ENTRIES	CPI	IPC	Prediction accuracy	Note
8	2.240	0.446	71.9%	after index correction
16	2.232	0.448	74.6%	mispredict $\approx$ 9.4% decreased
32	2.218	0.451	76.7%	32-entry experiment baseline
64	2.203	0.454	81.6%	32→64 largest jump
128	2.198	0.455	83.3%	lowest suite CPI in the scan

Interpretation:

- As the table grows, accuracy increases and CPI decreases.
- The 32→64 gain is pronounced (accuracy +4.96 points, mispredict $\approx 21\%$, cycle $\approx 0.70\%$).
- The 64→128 gain is small (cycle $\approx 0.23\%$). Diminishing returns in capacity begin.
- `DMEM_WAIT` stayed fixed in this scan at ≈ 505 million cycles; the difference is from the predictor.
- `nsichneu` regressed on the 16-entry and 128-entry transitions. A larger PHT does not mean automatic improvement on every workload.

On the FPGA the actual choice is not made by looking at this table alone; the decision between 64 and 128 is made together with LUT, FF, and Fmax.

9.7.3 Experiment 2: GShare, BP_ENTRIES=128

Capacity was fixed at 128 and only GShare and GHR_WIDTH were changed.

Table 9.7: Embench weighted results, BP_ENTRIES=128, GShare scan.

GShare	GHR	CPI	IPC	Accuracy
off	—	2.198	0.455	83.3%
on	7	2.212	0.452	78.8%
on	3	2.204	0.454	81.0%
on	2	2.202	0.454	81.7%

Interpretation:

- In this suite the result improves as GHR shortens: GHR=7 → CPI 2.212, GHR=3 → 2.204, GHR=2 → 2.202 (at each step CPI decreases, accuracy increases).
- GHR=2 is the best of the pure GShare configurations tried.
- Even so it cannot surpass 128-entry bimodal: CPI is approximately 0.20% higher, accuracy 1.62 points lower.
- `nettle-sha256` and `edn` benefit from global history; `slre`, `nbody`, `minver` are better with bimodal.

Over the suite, the choice of this scan is **128-entry, GShare off**; this is the REF32 profile default. If the FPGA LUT/FF/Fmax report makes a different product decision between 64 and 128, that value is documented separately. Even if GShare wins for a single workload, the report is written by looking at suite-weighted CPI.

Table 9.8: Short summary of the predictor experiments.

Result	Why?
BP_ENTRIES 8→128 lowers CPI	A larger PHT reduces aliasing; accuracy increases.
The 64→128 gain is small	Diminishing returns in capacity; area/Fmax cost increases.
GShare (in this suite) cannot surpass bimodal	Even a short GHR does not improve suite-weighted CPI.

9.7.4 Summary of the Experiment Result

The valid summary of this campaign is as follows:

With the same Embench suite and the same memory model, BP_ENTRIES was raised from 8 to 128; weighted CPI went from 2.240 to 2.198, predictor accuracy went from 71.9% to 83.3%. After 64 the gain decreased. On the same 128-entry table, GShare (GHR=2, 3, 7) did not improve suite CPI. The store-stall run in which two parameters changed at the same time was not taken into the comparison because it masked the DMEM gain.

9.8 Common Mistakes

9.8.1 MONITOR_ENABLE = 0

When `MONITOR_ENABLE=0`, the monitoring block is not synthesized. The request router does not select the `0x40005000` window as a valid target; the access results in an error response / access fault.

In this case the counters neither increment nor are read. An access fault may be seen on the core side, but it cannot be said that the `BUS_ERROR` monitoring counter incremented: that register does not exist in the first place.

A valid CPI/IPC report is formed only when `MONITOR_ENABLE=1` and monitoring is enabled through `MON_CTRL`.

9.8.2 Counter Saturation

REF32 monitoring counters saturate at the value:

`0xFFFF_FFFF`

A counter that has reached this value no longer increments.

In this case the measurement window is too long and the true event count is no longer known.

As a solution:

- the measurement duration can be shortened,
- an intermediate snapshot can be taken,
- the benchmark can be split into parts.

9.8.3 Not Using Freeze or Snapshot

If the core continues to run while live counters are being read, the counter values may belong to different points in time.

For example, if first `CYCLES` is read, and a few instructions later `RETIRED` is read, the two values were not taken at the same time.

For this reason, at the end of the measurement:

- `FREEZE` should be applied

or:

- `SNAPSHOT` should be taken and the shadow registers should be read.

9.8.4 Including UART Output in the Measurement

Printing monitoring results to the terminal also executes instructions.

In particular, when the UART FIFO fills, many cycles can be spent because of polling.

If the counters are left on, these cycles appear as if they were part of the measured benchmark.

Therefore the order:

`Benchmark -> FREEZE/SNAPSHOT -> UART Print`

should be used.

9.8.5 Mistaking Different Binaries for a Core Comparison

The same C code can turn into a different instruction stream with:

`-O0`

and:

-02

In this case **RETIRED**, the branch count, and memory accesses can change.

Therefore a test made using two different binaries is not directly a microarchitectural comparison.

9.8.6 Taking the Simulation Cycle to Be Real Time

The monitoring counter counts cycles.

If 50 MHz is used on the FPGA, the theoretical execution time can be computed as:

$$t = \frac{\text{CYCLES}}{50\,000\,000}$$

However, in how many seconds the simulation completed on the computer is different from this.

The wall-clock time of the simulation and the simulated run time of the core must not be mixed.

9.8.7 Looking at a Single Metric

A CPI or IPC value alone does not show the bottleneck.

For example, the branch count may be high, but if:

MISPREDICT = 0

then predictor performance is not the fundamental cause of the problem.

Likewise **DIV_USE** may be low, but because divider latency is high, a small number of **DIV** instructions can increase total time in a noticeable way.

Therefore the raw counters should be interpreted together.

9.8.8 Mixing Live and Shadow Counters

After a snapshot, the consistent measurement is in the shadow register region.

For example, while:

CYCLES live = 0x004

it is read from the address:

CYCLES shadow = 0x104

Taking a snapshot and then later reading the live register does not give the value at the snapshot instant.

9.9 Minimum Valid Performance Report

A performance result should contain at least the following information:

- the test environment,
- the binary used,
- compiler and flag information,
- the core configuration,
- raw **CYCLES** and **RETIRED**,
- the necessary monitoring counters,
- CPI and IPC,
- a short performance commentary based on the counters.

Without this information, a result given only as:

“CPI = 1.4”

cannot be compared in a sound way with another test.

Chapter 10

Debug Ports

Chapter 8 and Chapter 9 described the counters and the preparation of the performance report. This chapter covers the observation ports that the core presents to the outside. The goal is not to define JTAG or an external debug bridge. The goal is to observe, in simulation and on the FPGA waveform, which PC the pipeline is processing at which stage, what is written to the register file, and the instantaneous values of the trap CSRs.

These ports are defined on `tmcu_core`. `tmcu_soc` and `tmcu_top` bring the ports to the upper level. When `DEBUG_ENABLE=0`, the outputs are pulled to 0 and the ports can be hidden in the Vivado IP. In Vivado Block Design simulation they are seen on the ports of the `tmcu_soc_0` instance; in Verilator `tmcu_top` simulation they are seen through the core hierarchy (when `DEBUG_ENABLE` is on).

10.1 Structure of the Debug Ports

There are two separate observation families in the core. They must not be mixed.

- **dbg_* ports.** They are enabled by the `DEBUG_ENABLE` parameter (default 1). They are for observation only; they do not change core behavior. When off, the outputs are constant (0).
- **trace_* ports.** They are always produced on `tmcu_core` and `tmcu_top`. In the SoC wrapper they are exposed with the `TRACE_ENABLE` parameter. In the `tmcu_soc` default this parameter is 0. When off, the SoC outputs are constant (`trace_valid_o = 0`).

The main subject of this chapter is the `dbg_*` family. The `trace_*` family is an optional commit/trace observation interface; it is summarized briefly in Section 10.1.3.

10.1.1 Pipeline PC observation

The five PC outputs reflect the fetch address in that cycle and the four pipeline register PCs. `dbg_fetch_pc_o` is not a pipeline stage register; it is the IMEM fetch PC (`fetch_pc_q`). The other four come from the related pipeline registers. The port names are abbreviations of the stage names: `if` IF/ID (decode), `ex` ID/EX (execute), `mem` EX/MEM, `wb` MEM/WB.

Table 10.1: Debug PC ports and RTL source.

Port	Width	Source / stage
<code>dbg_fetch_pc_o</code>	32	<code>fetch_pc_q</code> (fetch)
<code>dbg_if_pc_o</code>	32	<code>if_id_pc_q</code> (decode)
<code>dbg_ex_pc_o</code>	32	<code>id_ex_pc_q</code> (execute)
<code>dbg_mem_pc_o</code>	32	<code>ex_mem_pc_q</code> (memory)
<code>dbg_wb_pc_o</code>	32	<code>mem_wb_pc_q</code> (writeback)

The validity bits answer the question “is there an instruction in this stage?”. There is no separate valid port for fetch; `dbg_fetch_pc_o` is published every cycle.

Table 10.2: Debug valid bits.

Port	Source	Meaning
dbg_if_valid_o	if_id_valid_q	There is an instruction in the decode stage.
dbg_ex_valid_o	id_ex_valid_q	There is an instruction in the execute stage.
dbg_mem_valid_o	ex_mem_valid_q	There is an instruction in the memory stage.
dbg_wb_valid_o	mem_wb_valid_q	There is an instruction in the writeback stage.

dbg_fetch_pc_o is at the same time the IMEM request address (`imem_req_addr_o = fetch_pc_q`). Because IMEM fetches a word (32-bit), when the request is accepted (`imem_req_valid & imem_req_ready`) `fetch_pc_q` always increments by +4. This does not mean that the architectural instruction PC always advances by +4; for compressed instructions the architectural next PC (+2/+4) is kept separately on the decode/pipeline path.

On a redirect the fetch address is written to the word-aligned base (`{target[31:2], 2'b00}`). If the target is 2-byte aligned (`target[1]=1`), the low half-word of the first IMEM response is skipped (`fq_fill_skip_lo`); thus 2-byte targets compatible with the C extension are supported. If there is no request or memory is not ready, `fetch_pc_q` freezes. When looking at the fetch row, “did the PC change?” and the next stage’s `valid` bit are looked at together. If a stage’s `valid` bit is 0, the PC on that row may be carrying an old value.

10.1.2 Register file write port

The core presents the register file’s *write port* to the outside. There is no scan path that reads the contents of all 31 registers. They are the same signals as Table 3.2.

Table 10.3: Register file debug write port.

Port	Source	Meaning
dbg_rf_we_o	rf_write_enable	<code>mem_wb_valid & mem_wb_rd_write & ~mem_wb_exception_valid</code>
dbg_rf_waddr_o	mem_wb_rd_addr_q	Destination register
dbg_rf_wdata_o	mem_wb_result_q	The 32-bit value written

`mem_wb_result_q` comes from a load result, an ALU result, the next PC, or a CSR read result. During an exception, write is off.

An x0 destination is ignored in the register file (`rd_write & (rd_addr != 0)`). The debug port write-enable does not additionally mask for x0. Therefore even if `we=1` and `waddr=0` are seen on the waveform, there may not be a real RF write; nothing was written to the file. The trace port (`trace_rd_write`), on the other hand, masks x0.

If the shadow bank is on (`SHADOW_BANK_ENABLE=1`; in the REF32 SoC/FPGA profile the default is 0), hardware selects the shadow bank on trap entry. The debug port still shows the address and data *written in that cycle*; which bank is selected is not a separate `dbg_` signal.

10.1.3 CSR snapshot

The CSR block (`tmcu_csr_trap`) brings selected CSR values (for example `mstatus`, `mepc`, `mcause`) out to the debug ports. This is not presenting the entire architectural register file to the outside; nor is it looking at the CSR address space via MMIO. The ports are not separate shadow registers, but instantaneous observation values of the related CSR state.

Table 10.4: Debug CSR ports.

Port	Width	Source
dbg_csr_mstatus_o	32	mstatus_q (held fields: MIE bit3, MPIE bit7, MPP bit[12:11]=11)
dbg_csr_mie_o	32	mie_q
dbg_csr_mtvec_o	32	mtvec_q
dbg_csr_mscratch_o	32	mscratch_q
dbg_csr_mepc_o	32	mepc_q
dbg_csr_mcause_o	32	mcause_q (bit31 interrupt, [30:0] code)
dbg_csr_mtval_o	32	mtval_q
dbg_csr_mip_o	32	mip_value
dbg_csr_mcycle_o	32	mcycle_q[31:0]
dbg_csr_mcycleh_o	32	mcycle_q[63:32]
dbg_csr_minstret_o	32	minstret_q[31:0]
dbg_csr_minstreth_o	32	minstret_q[63:32]

After reset `mstatus` = 0x00001800 (MPP=M), `mtvec` = 0x80000000 (REF32 MTVEC_RESET). On an `mtvec` write the MODE bits are cleared; the trap vector is always the address {`mtvec`[31:2], 2'b00} in direct mode. These CSR copies are not the same registers as the CYCLES and RETIRED MMIO counters in the monitoring unit (Appendix A).

10.1.4 Optional trace (TRACE_ENABLE)

`tmcu_core` produces the following trace record every cycle:

```
trace_valid = interrupt_trace_pending_q | mem_wb_valid
```

A normal WB commit trace is produced with `mem_wb_valid`. An interrupt trace, on the other hand, is recorded delayed by one cycle (`interrupt_trace_pending_q <= interrupt_trap_valid`); thus it appears in the same event stream as interrupt accept, but shifted by one cycle. The record fields contain PC, the instruction word, an `rd` write (`x0` excluded), memory access, and trap information.

In the SoC default (`TRACE_ENABLE=0`) these ports are not connected outside. Enabling the trace on the FPGA requires LUT/FF, routing, and if desired ILA capacity; the signals do not necessarily have to be brought out to external FPGA pins. In course measurements the main path is monitoring MMIO; the trace is for a short wave-dump or a special trace collector.

10.2 Use of the Debug Ports

10.2.1 When to Use dbg_* and When to Use Monitoring

If there is a short question, a wrong instruction, or an unexpected branch, `dbg_*` is looked at first. If the CPI of a million-cycle benchmark is sought, the monitoring counters are used. The two do not replace each other: the debug port provides instantaneous observation; monitoring produces accumulated counter values.

10.2.2 Vivado Block Design simulation

In the Arty A7 BD wrapper the core is the `tmcu_soc_0` instance. The prepared wave script:

```
scripts/wave_arty_bd_hello.tcl
```

The top module must be `tb_tmcu_arty_bd_hello`. The script adds three debug groups:

- 09_Debug_PC — five PCs and four valids.
- 10_Debug_RF — `we`, `waddr`, `wdata`.
- 11_Debug_CSR — twelve CSR copies (`mstatus ... minstreth`).

The signal path is first searched from the wrapper ports:

```
/tb_tmcu_arty_bd_hello/dut/tmcu_arty_bd_i/tmcu_soc_0/dbg_*
```

If the port is absent, one descends in the hierarchy to the related core instance (`inst`).

10.2.3 Verilator / `tmcu_top`

In a simulation without the SoC wrapper the ports are on `tmcu_core` and `tmcu_top` transfers them to the external interface. In the wave viewer (GTKWave, Surfer) add the same names. After reset is released (`rst_ni=1`), `dbg_fetch_pc_o` should show the reset vector. In REF32, `RESET_VECTOR = 0x80000000` (IMEM base).

10.2.4 Reading order

When reading a cycle the order is as follows:

1. Reset and clock: while `rst_ni=0` the PCs are meaningless; when `rst_ni=1`, reset has been released.
2. Fetch PC: which address is the core requesting?
3. IF valid and PC: what entered decode?
4. EX, MEM, WB: at which stage is the instruction?
5. `dbg_rf_we_o`: was a register updated in this cycle?
6. CSR: if there was a trap, did `mcause` and `mepc` change?

10.3 Examples of the Debug Ports

The following examples are behaviors to be read from a simulation waveform. The addresses are of the REF32 map (Appendix A).

10.3.1 First fetch after reset

When reset is released (`rst_ni=1`), `dbg_fetch_pc_o` becomes `0x80000000`. IMEM gives the first instruction from this address. Depending on IMEM latency and the fetch-queue fill time, the same PC later appears on `dbg_if_pc_o`; `dbg_if_valid_o` goes to 1. This shows that the pipeline has filled. Because `fetch_pc_q` fetches an IMEM word, it advances by +4 on every accepted request. The architectural next instruction address is +2 on a compressed instruction, +4 on a 32-bit instruction (or the predicted branch target); this value is held on the `pc_next` path inside the pipeline.

10.3.2 An ALU write

When `addi x3, x0, 1` commits:

- `dbg_wb_valid_o = 1`,
- `dbg_wb_pc_o` = that instruction's PC,
- `dbg_rf_we_o = 1`,
- `dbg_rf_waddr_o = 3`,
- `dbg_rf_wdata_o = 1`.

Because there is no trap in this example, `mcause` does not change. If there is no write, either the instruction is not yet in WB, or an exception has cut off the write. `dbg_rf_wdata_o` also carries a value while `we=0`; for interpretation it is looked at together with `we`.

10.3.3 Load

For `lw`, the `wdata` in WB is the load coming from memory (or from MMIO). The address path is not on the `dbg_*` port; from which address it was read is in the EX/MEM internal signals or in the `trace_mem_*` fields of the trace port. If the load is extended with `memory_stall`, WB valid is delayed; in the cycles while the stall continues, the instruction appears on `dbg_mem_pc_o/dbg_mem_valid_o`.

10.3.4 Branch

On a taken branch, the fetch PC jumps to the target with a prediction or a correction. When a mispredict is resolved in EX, the IF/ID and ID/EX valid bits that are younger than the branch are cleared; the branch instruction itself can advance onto the MEM/WB path. Older instructions in MEM/WB are not affected by this recovery flush. For flush event counting there is the monitoring FLUSH counter ; the debug port, on the other hand, shows which stage PCs were invalidated in that cycle.

10.3.5 Trap

A synchronous exception (for example `ecall`) and an interrupt use the same trap CSR ports, but the `mepc` semantics differ:

- `dbg_csr_mcause_o` goes to the related code (Appendix A, Table A.11 and Table A.12),
- On an exception, `dbg_csr_mepc_o` holds the faulting instruction PC; on an interrupt it holds the next architectural PC (`fetch_pc_q`) (`mepc[0]=0`),
- Inside `dbg_csr_mstatus_o`, MIE is cleared, MPIE stores the previous MIE, MPP remains 11,
- The fetch PC goes to the `mtvec` base (`{mtvec[31:2], 2'b00}`).

After `mret`, the return to `mepc`, `MIE ← MPIE`, and `MPIE=1` are read from the same CSR ports. If the shadow bank is on, the shadow bank is entered on a trap and the normal bank is returned to on `mret`; this selection is not a separate wire on `dbg_*`.

10.3.6 A character written to UART

When software does an `sw` to address `0x60000004` (AXI UART Lite `TX_FIFO`), the store instruction does not do an RF write; the debug RF port does not show a write for this instruction. The peripheral write is on the memory/MMIO path. To confirm that the character went, AXI UART Lite `STAT` bit 3 (`TX_FIFO full=0`) and/or the TX queue of the simulation UART model is looked at. The debug port answers the question “which instruction was the core on?”; it does not answer the question “what did UART send?”.

10.4 Limitations of the Debug Ports

1. **It is not the RISC-V Debug Specification.** There is no JTAG, halt, abstract command, program buffer, or external debugger. The core does not stop running.
2. **Not all registers are visible.** There is only a write port. You cannot read the current value of `x5` at an arbitrary instant through the debug ports; you can only observe the next write made to `x5`, or software can dump the value to memory/UART.
3. **There is no fetch valid.** `dbg_fetch_pc_o` is present every cycle ; a new fetch and a stalled fetch PC are not distinguished from this port.
4. **mcycle is a CSR, not CYCLES MMIO.** `mcycle/minstret` on the debug port are architectural counters. Monitoring `CYCLES/RETIRED` are separate registers; freeze and clear affect only the MMIO side.
5. **dbg_csr_mstatus_o is not the full mstatus.** The port reflects only the implemented fields: the writable fields are MIE and MPIE; MPP is always forced to 11. The other bits of the 32 bits read are zero (reset value `0x00001800`).
6. **The trap vector is Direct mode only.** Even if `mtvec[1:0]` is written, it is clipped to 00. There is no `mtvec + 4*cause` behavior.
7. **TRACE_ENABLE is off by default in the SoC.** Before looking at the FPGA BD trace ports, verify that the parameter is 1. In `tmcu_top` simulation the trace is always present.
8. **DEBUG_ENABLE disables the debug outputs when set to 0.** The default is 1. When made 0, the `dbg_*` outputs become 0 and in Vivado Customize IP the ports are hidden. Keeping it on for waveform observation is recommended; if observation ports are not needed it can be disabled.
9. **FPGA resource and routing cost.** Bringing all `dbg_*` signals out to the board is not practical. The place of use is the simulation waveform or a few markers selected for the ILA. This repository does not add an ILA core to the default Vivado flow.
10. **Time resolution is the cycle; it is not a total.** Translating a long Embench run from the debug port into CPI is not practical; monitoring should be used for this purpose (Chapter 9).
11. **Observation is not control.** It is not possible to write to these signals. For interrupt, stall, or freeze, the related MMIO and pins are used.

A short distinction:

`dbg_*`: instantaneous pipeline / CSR observation; `monitoring`: accumulated counters; `trace_*`: optional commit/trace interface.

DRAFT

Chapter 11

rv32 Emulator and Assembly Development Environment

This chapter is intended to help students writing assembly programs for EHB326 install the rv32 tool, run their programs, and explain the observed results. The order followed is preparation of the source code, the assembly operation, register and memory observation, debugging, and production of the memory image.

Tool behavior has been verified through the local rv32 1.1.0 implementation and the source code examined. The reference used for the main book is *EHB326_Book_MCU.pdf*, Rev. P1.0. The chapter number in this text is a merge suggestion; it can be changed using the main book's chapter counters.

Document Use Rules

Chapters 3–10 and Appendix A of the book define microBee hardware. The screens, shortcuts, and example addresses here, on the other hand, belong to the rv32 software environment. Support for an ISA operation does not imply that the peripheral register map or cycle timing is identical.

Table 11.1: Information classes used in this chapter

Class	Meaning
Verified in the tool	Behavior observed in the examined rv32 version or verified with source code.
Configurable	A value selected with the rv32 interface or supported board.toml keys.
REF32 information	Hardware information belonging to the main book's microBee-REF32 profile.
To be selected in the course scope	A condition the instructor will determine, such as submission files, board, and target tool flow.

Learning outcome. The student should be able to show which instruction of a program changed which register or memory address; and should be able to distinguish the result observed in rv32 from the timing measured in hardware.

Scope of the examples. The programs in this package have been prepared for rv32. Versions to be run on REF32 must additionally be adapted with respect to address placement, start address, MMIO access, and halt method. The comparison is given in Section 11.10.

11.1 Introduction

11.1.1 What Is It?

rv32 is an emulator, assembler, and debugger application that supports the RV32IMC `_Zicsr_Zifencei` instruction set. Source code is written in the same window and converted to machine code. Registers, data memory, and instruction encoding can be examined while the program is advanced. The name of the desktop application is `rv32-gui`; the name of the terminal application is `rv32`.

11.1.2 Purpose

It makes the effect of an assembly line visible. The student observes when the arithmetic result is written to the register, to which address the store operation writes, and under which condition the branch is taken. Small assembly experiments can be done without using a separate RISC-V compiler chain.

11.1.3 Usage

1. Select the machine to work with and the example source file.
2. Edit and save the source; assemble with F5.
3. Step with F8 or run with F9.
4. Check the expected register, memory, and Console values.
5. If necessary, correct the source; assemble again.
6. Export the memory images the course asks for at the appropriate sizes.

11.1.4 Limitations

rv32 is not an RTL simulator. FPGA pins, bus handshakes, memory waits, timing closure, and the physical clock are not verified here. The tool's Pipeline view is for teaching purposes; it does not contain a hazard, stall, and forwarding model. There is no ELF loading, floating-point instructions, or user/supervisor mode support.

When simulation environments are introduced in Section 2.4 of the book, this distinction can be used: rv32 serves to follow the functional effect of the program; RTL simulation, on the other hand, serves to verify microBee's hardware signals and clock-based behavior. For the hardware performance measurements in Chapters 8–9, the related RTL/FPGA environment should be used.

11.2 Installation and First Launch

11.2.1 Windows

1. Extract the rv32-windows-x64.zip package provided for the course to a folder for which you have write permission.
2. Run the rv32-gui.exe file. Keep the DLL files and the platforms folder next to the application.
3. Open your source file with Ctrl+O. Use Ctrl+S to save.

11.2.2 Linux

The installed application is opened from the menu or with the `rv32-gui` command. If the AppImage and the `install.sh` file provided with the project are being used, write the real file paths in the command below:

```
bash /path/to/install.sh ~/Downloads/rv32-x86_64.AppImage
```

The application is installed under the user's home directory. The installer extracts the AppImage contents and thereby bypasses the FUSE requirement. On systems that have FUSE support, the AppImage can also be opened directly by giving it execute permission. A debug build created from source is run from the repository root with `./build/debug/rv32-gui`.

11.2.3 Layout of the Working Files

```
lab01/  
  first_steps.s  
  board.toml  
  exports/
```

`.s` is the editable assembly source. `.toml` is the machine configuration. `.mem` is the memory image. After the memory image is produced, the source file should be kept. The `examples/rv32` directory at the book root contains four starter programs and an example board file; these can be used by copying them into the `lab01` working folder.

11.2.4 Troubleshooting Startup Problems

Table 11.2: Situations frequently seen during installation

Situation	Check
The Windows application does not open	Check that the entire ZIP was extracted; that the DLL and platforms folder were not left behind.
The Linux command is not found	Use the real path of the application or check the PATH setting of the user installation.
The AppImage does not open	Try installation with the provided installer; when running directly, check permission and FUSE support.
The file is not found	Select the real .s file with Ctrl+O; check the working directory in the terminal.

11.3 Interface and Run Controls

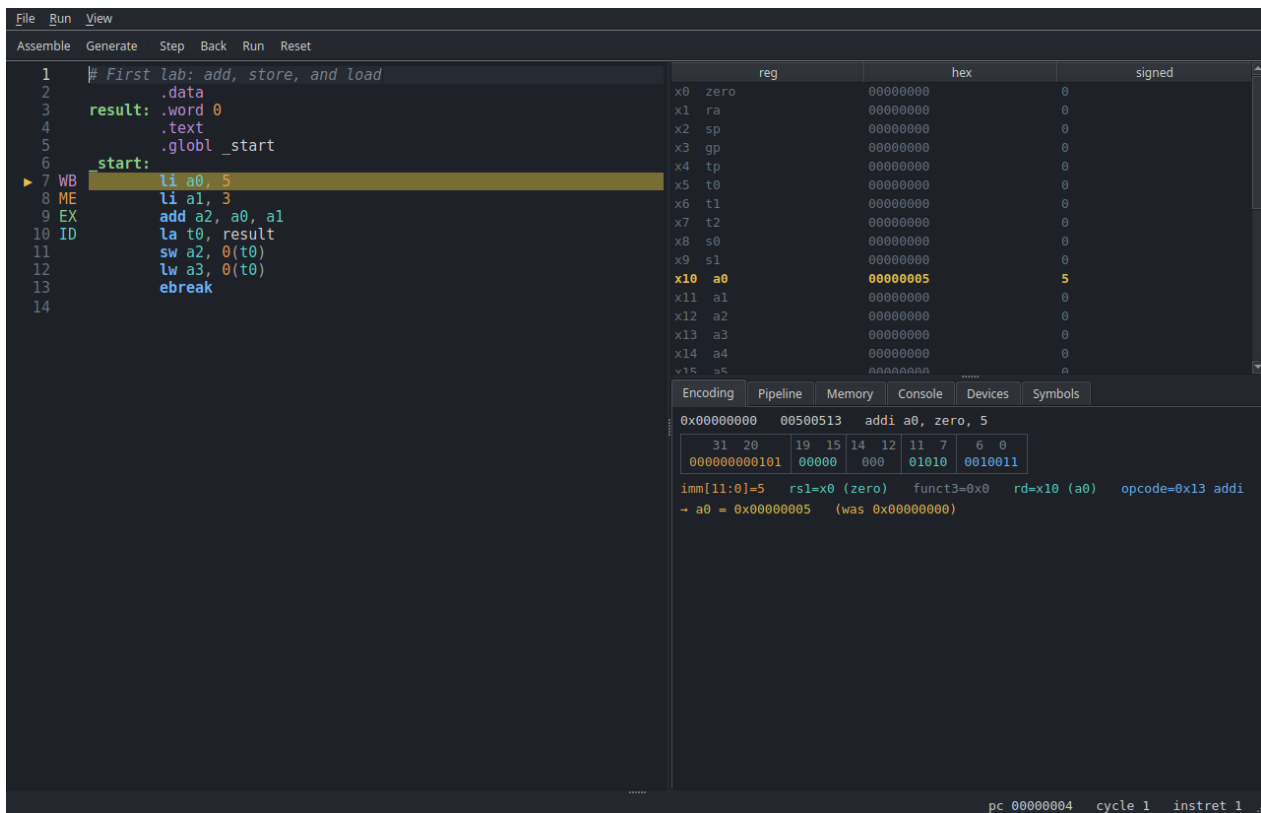


Figure 11.1: When the first instruction reaches the write-back stage, register a0 becomes 5. The Encoding tab shows the operation that occurred.

Table 11.3: Basic shortcuts

Shortcut	Function
F5 — Assemble	Assembles the source in the editor and loads it.
F8 — Step	Advances one pipeline cycle.
F6 — Back	Undoes one pipeline cycle, together with the commit effect in that cycle.
F9 — Run	Runs; pressing again while running stops execution.
F4 — Reset	Restarts the loaded program; keeps breakpoints.
Ctrl+E — Generate	Produces memory images of the most recently assembled program.
Ctrl+N / Ctrl+O / Ctrl+S	New file / open file / save.

Saving and assembly are different operations. Before running the changed source or producing an image, F5 must succeed. Reset does not reassemble the edited text. A failed assembly keeps the previously loaded program.

11.4 First Program: Arithmetic, Store, and Load

11.4.1 Usage Example

Open the file `examples/rv32/first_steps.s`. The program adds 5 and 3, writes the result to DMEM, and loads it again into a register.

```
# First lab: add, store, and load
.data
result: .word 0
.text
.globl _start
_start:
    li a0, 5
    li a1, 3
    add a2, a0, a1
    la t0, result
    sw a2, 0(t0)
    lw a3, 0(t0)
    ebreak
```

11.4.2 How to Run It

1. Press F5. The note explaining that two instructions were produced for `la` is not an error.
2. From the newly loaded state, press F8 five times. When the first instruction reaches WB, `a0 = 5` is observed.
3. Continue advancing. `a1 = 3` and `a2 = 8`. The `la` line contains two machine instructions.
4. When `sw` completes, cell +0 of address 0 on the Memory tab becomes 00000008. When `lw` completes, `a3 = 8`.
5. Finish the program with F9. `ebreak` in this example is a deliberate halt point.

Table 11.4: Expected values at the end of the program

Observation	Value
<code>a0 / x10</code>	5
<code>a1 / x11</code>	3
<code>a2 / x12</code> and <code>a3 / x13</code>	8 = 0x00000008
<code>t0 / x5</code>	0: DMEM address of the result symbol
DMEM[0]	00000008

11.4.3 Student Exercise

Change the first constant to 12, save the file, then press F5 and F9 again. The expected result is 15. Explain the difference between using only Reset after the change and reassembling.

11.5 Register, Memory, and Instruction Observation

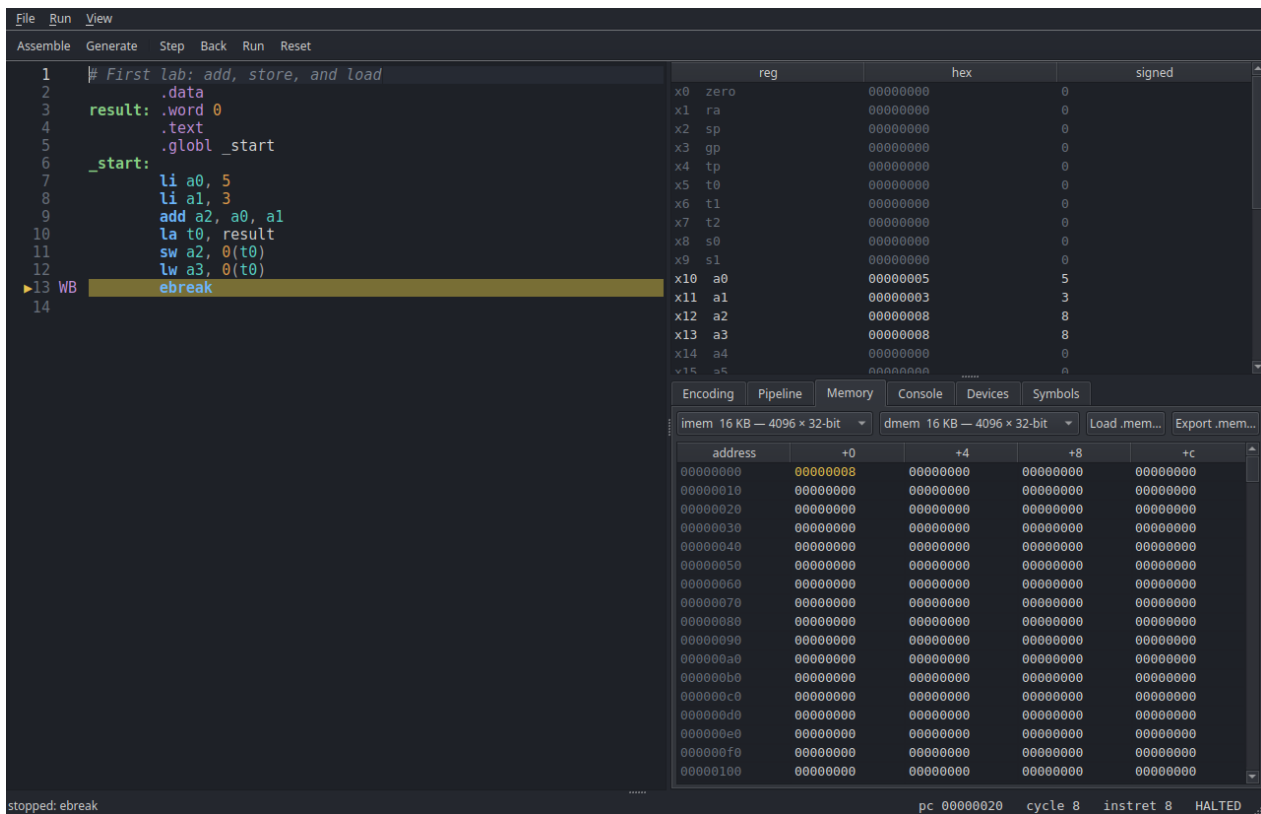


Figure 11.2: After the first program completes, result = 8 is observed in data memory. The row address and the column offset are read together.

11.5.1 Reading the Register File

The Registers view shows both architectural and ABI names, such as x10 and a0. The Hex and signed columns are different interpretations of the same 32-bit pattern. For example FFFFFFFF is -1 in the signed display. The last written register is highlighted. x0 does not store a writable general-purpose value; zero is always read.

11.5.2 Computing the DMEM Address

The cell's byte address is the sum of the row address and the column offset. Column +4 of row 00000010 is address 0x14. Each cell shows a 32-bit word. For lw/sw the address must be a multiple of 4; for lh/sh, of 2. Memory is little-endian: the contents of the word 0x12345678 at increasing byte addresses are 78, 56, 34, 12.

11.5.3 Encoding and Symbols

Encoding shows the instruction bit fields and the effect of the operation; during stepping it follows WB or the line selected in the code. For a compressed instruction the expanded base instruction is also visible. Symbols shows that the label is in the Text or Data region. In rv32 both memories start from 0; the same numeric address does not mean the same physical place.

11.6 Pipeline and Debugging

11.6.1 Reading the Pipeline

IF, ID, EX, MEM, and WB show the stages of the instruction. The rows of the lower diagram represent instructions, the columns cycles. The yellow source line shows WB. A resolved branch is shown green when taken; red when not taken. Ctrl+Shift+P toggles pipeline markers, Ctrl+Shift+B toggles branch colors.

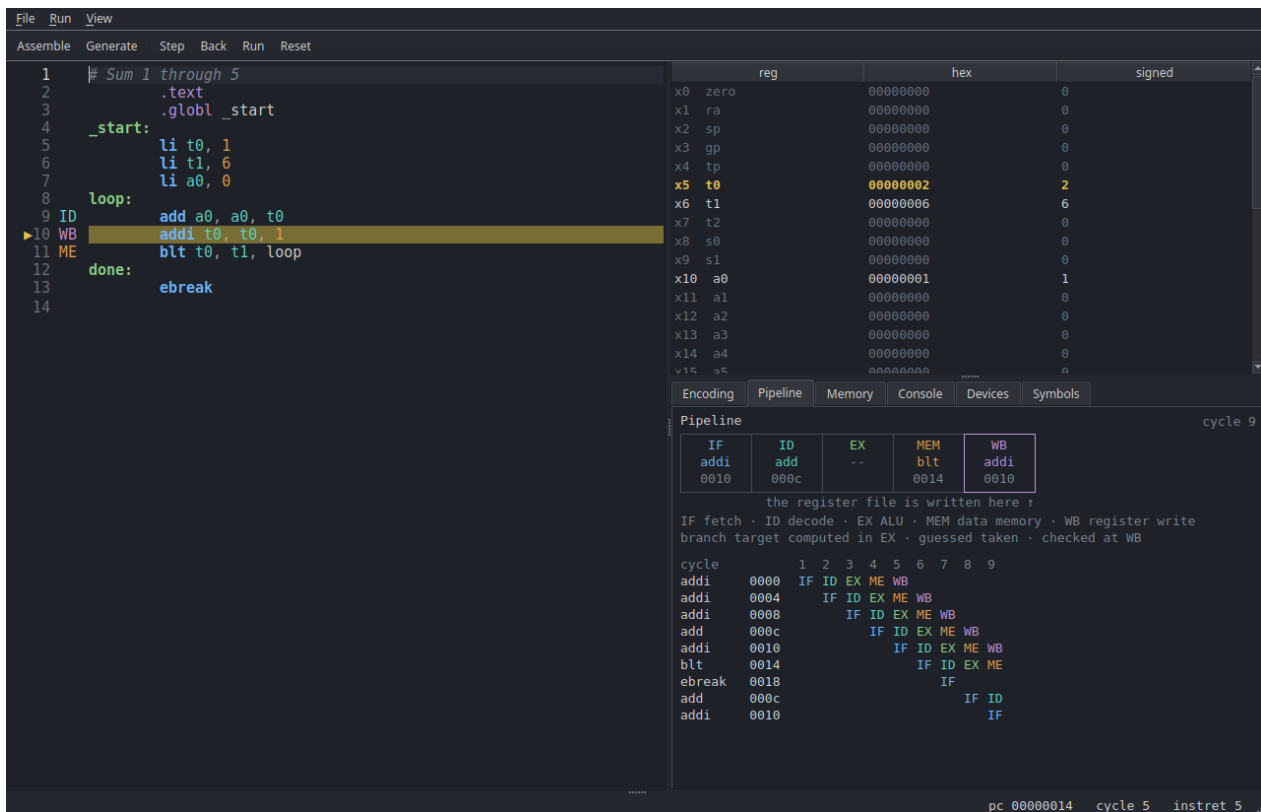


Figure 11.3: Pipeline view in the sum_loop.s program. The last cycles can be followed on an instruction basis.

11.6.2 Meaning of One Step

F8 does not execute a single source line; it advances one display cycle. The emulator's architectural state change is applied when the instruction arrives at WB. The interface's cycle count is not the same measurement as the cycle/mcycle/time counters that software reads. If the source line is a pseudo-instruction, it can correspond to more than one instruction.

11.6.3 Difference from the Hardware Pipeline

The tool uses an always-taken prediction policy; the branch result is finalized in WB and on a mispredict the view is cleared. The microBee predictor, stall, and hazard features in Chapters 3 and 7 of the book are not modeled on this screen. microBee CPI, forwarding, or branch penalty is not measured through this screen.

11.7 Breakpoint, Step Back, and Diagnostic Messages

11.7.1 Examining a Loop

The file sum_loop.s adds the numbers from 1 to 5 in a0. The initial values are t0 = 1, t1 = 6, and a0 = 0; the body consists of add, addi, and blt instructions.

1. Load with F5. Click the left edge of the add a0, a0, t0 line and set a breakpoint.
2. Come to the breakpoint with F9. Before advancing, read the counter and the sum.
3. Advance with F8 until add completes. Go back with F6 and watch the register return to its old value.
4. Remove the breakpoint by clicking the same edge and press F9. a0 = 15 and t0 = 6 are expected.
5. Restart with F4. To step over a function in the GUI, set a breakpoint on the executable line after the call and run.

11.7.2 Using Code Completion

When two letters of the mnemonic are typed, suggestions open; they can be requested explicitly with Ctrl+Space. Tab/Shift+Tab advances among selections, Enter accepts the selection, Esc closes. Gray operand hints are not part of the source text. Commas, registers, and immediate values are entered by the user.

Table 11.5: Common errors and recommended fixes

Symptom	Path to follow
Undefined symbol	Check the spelling of the label. Click the diagnostic message to go to the related line.
Immediate out of range	The addi range is -2048...2047; load a large constant with li.
Calling a data label as code	A Text label for a code target; use la for a data address.
Misaligned load/store	Check the effective address and the data placement.
Red memory overflow mark	Increase the allowed memory size or shrink the program; assemble again.
The program shows old behavior	Verify that F5 completed successfully.
Console empty	Check that the program actually wrote to UART and that the device is connected.
Infinite loop	Stop with F9; examine the branch operands and the counter update.

Correction method: determine the expected value, stop before the instruction that produces the change, find the first difference, correct the source, and try again from the same initial condition.

11.8 Peripheral and Interrupt Studies

11.8.1 UART Usage Example

uart.s sends ASCII 65 and 10 to the rv32 default UART device. A and a line ending are expected on the Console. The output services of other ecall-based simulators are not used here.

```
li t0, 0xffff0000
li a0, 65
sb a0, 0(t0)
li a0, 10
sb a0, 0(t0)
ebreak
```

Table 11.6: rv32 default peripheral addresses

Address	Role
0xFFFF0000	UART; +0 byte transmit, +4 status.
0xFFFF0010	LED output register.
0xFFFF0020	Switch input register.
0xFFFF0030	mtime low/high words.
0xFFFF0040	mtimecmp deadline value.
0xFFFF0050 / 0xFFFF0054	Software / external interrupt request registers.

11.8.2 Loading the Configuration

examples/rv32/board.toml is loaded with File → Load peripherals... This operation changes the device set; it does not mean adding to the existing device. The file can also determine memory sizes. switches_leds.s reads the switch once and writes to the LEDs. Because the provided board's initial input is 0b1010, a0 = 10 is expected. Change the input and resample with Reset and Run. The switch input is preserved throughout reset.

11.8.3 What Should Be Considered When Handling Interrupts?

First the mtvec handler address must be set up, and the related source inside mie and mstatus.MIE must be enabled. The handler must clear the source or advance the timer deadline. On interrupt return mepc is not

advanced; on an ecall exception, on the other hand, the return address is arranged in a suitable way to prevent repetition of the same instruction. For a start, the timer example provided in the course repository can be used; in this book package there is not yet a timer.s under `examples/rv32/`.

rv32 does not implement the book's PLIC claim/complete mechanism and the shadow register bank. Do not run the book's Chapter 5 examples directly with this default device map. Address and behavior differences are given together in Section 11.10.

11.9 Producing a Memory Image

11.9.1 Using Generate

1. Load the related board file and determine the IMEM/DMEM sizes on the Memory tab.
2. Save the source; do a successful assembly and overflow check with F5.
3. Select Generate, Ctrl+E, or File → Generate .mem. . .
4. Fill in the Folder and Base name fields. The name lab01 produces lab01_imem.mem and lab01_dmem.mem. The default name is rv32.
5. Mark the desired memories. If a plain file is wanted, keep the annotation selection off; press Export.

Table 11.7: Scope of the export operations

Operation	Data produced
Generate / Ctrl+E	The most recently assembled instruction image and the initial .data contents. Completed to memory depth.
Memory → Export .mem. . .	Current DMEM after run and manual edits.
Devices → block → Export .mem. . .	The selected RAM/ROM block's own current image.

Even if result = 8 after the first program has run, Generate produces the initial .word 0 value. To keep the current value, the export operation of the Memory tab is used. Generate does not automatically assemble a source change made afterwards.

11.9.2 Reading the File

Each line is a 32-bit hexadecimal word. If the index of the first line is taken as 0, the byte offset of line number n is $4 \times n$. 16 KiB memory corresponds to a depth of 4096×32 -bit. Compressed instructions are packed into the same word stream; a line is not always a single instruction.

```
rv32 asm first_steps.s --pad -o lab01_imem.mem --dmem lab01_dmem.mem
rv32 asm first_steps.s --pad --devices board.toml
```

If `--pad` is not given in the terminal, only the used portion is exported. For special memory sizes, give the same board file. If a special hardware memory that is word-addressed and does not contain byte enable is being used, sub-word access warnings can be obtained with `--strict-word-mem`.

11.9.3 Requirements for Using the Output in Vivado

This output is a word array in \$readmemh form. The form and depth requested by the laboratory's simulation or IP initialization flow must be verified. Changing the file name is not a format conversion. For REF32, only growing the depth is not sufficient; the address and device adaptation in Section 11.10 is also required.

11.10 Use Together with microBee-REF32

This comparison has been extracted from Chapters 3, 5–9 and Appendix A of the main book and from the examined rv32 sources. For the functional result to be comparable, the same initial data, algorithm, and observation point must be defined.

Table 11.8: Differences affecting porting between REF32 and rv32

Topic	REF32 in the book	Examined rv32
IMEM	0x80000000; 64 KiB	Separate Text region, starts from 0; default 16 KiB.
DMEM	0x20000000; 64 KiB	Separate Data region, starts from 0; default 16 KiB.
MMIO	Native devices around 0x40000000; AXI window 0x60000000	0xFFFF0000–0xFFFF03FF slot window.
GPIO / UART	GPIO register bank, native UART and AXI UART Lite	LED/switch and simplified UART devices; register behaviors are not equivalent.
Interrupt	CLINT, PLIC sources; related hardware options	Software, timer, external lines; no PLIC claim/complete or shadow bank.
Pipeline	Hardware hazard/stall and predictor configuration	Teaching view; no hazard/stall/forwarding model.
Time / measurement	Clock-based mcycle and monitoring registers	Instruction-based counters and a separate GUI cycle count.
Timer	Hardware timer/CLINT register model	Simplified mtime/mtimecmp; mtimecmp 32-bit.
Halt	Depends on the target debug/trap/testbench flow	ebreak by default stops at the debugger.

11.10.1 Does a Board File Make rv32 REF32-Compatible?

No. The supported board keys select memory sizes and rv32 devices; they do not create REF32's entire bus, GPIO, UART, PLIC, or monitoring model. Selecting 64 KiB does not make the IMEM start 0x80000000 or the DMEM start 0x20000000.

11.10.2 Preparing the Port

1. Test the algorithm and the expected result in rv32; write the environment into the result record.
2. In the target assembly/link placement, set up Text, Data, and entry addresses according to REF32. Re-evaluate the absolute data addresses formed with `la` and all MMIO accesses.
3. Adapt the device drivers, access widths, interrupt setup, and program-end method to the target.
4. Verify the image produced for the target in RTL simulation. Do the performance measurement in the book's monitoring/FPGA flow.

11.11 Command Line and Repeatable Experiment

11.11.1 Basic Commands

Apply the following commands in the working folder where the examples are. If rv32 is not on PATH, write the path of the executable file. In Windows PowerShell the program in the current folder is invoked in the form `.\rv32.exe`.

```
rv32 run first_steps.s
rv32 run first_steps.s --trace
rv32 run sum_loop.s --limit 1000
rv32 dis lab01_imem.mem
rv32 run switches_leds.s --devices board.toml
rv32 dbg sum_loop.s
```

Trace shows the machine instructions and their effects. A pseudo-instruction produces more than one line. The default run limit is 1,000,000 instructions. Reaching the limit does not prove that the program finished correctly. IMEM disassembly that contains padding can also show unused space after the program.

11.11.2 Using the Debugger

```
(rv32) break done
```

```
(rv32) run
(rv32) info registers
(rv32) print a0
(rv32) backi
(rv32) si
(rv32) continue
(rv32) quit
```

Table 11.9: Terminal debugger commands

Command	Function
step / si	One source line / one machine instruction.
next / finish	Step over a call / continue until the current function returns.
back / backi	Undoing one source line / one instruction.
run / continue	Running by resetting from the start / continuing from the current state.
watch result	Watching the change of the related data word in a program where result is defined.
x/4xw 0	Reading four words in hex from address 0.
help	Showing the command list and its usage.

GUI F8/F6 is in cycle units; terminal step/back is in source-line units, si/backi in instruction units. In the terminal an empty line repeats the previous command.

11.12 Student Exercises and Result Record

11.12.1 Exercise Order

Table 11.10: Starter exercises

Exercise	Change / observation	Expected
Arithmetic	first_steps.s: 12 instead of 5	a2 = a3 = 15; DMEM[0] = 0000000f.
Loop	sum_loop.s: upper bound 11 instead of 6	Sum of 1...10; a0 = 55.
UART	uart.s: 66 instead of 65	B and a line ending on the Console.
Input/output	Change the switch value, reset, run	a0 and the LEDs carry the sampled value.
Encoding	Compare addi a0, a0, 1 with c.addi a0, 1	Same arithmetic effect; 32/16-bit encoding.
Finding an error	Temporarily make the loop branch target loop	Undefined symbol diagnosis; successful assembly after correction.

11.12.2 Reporting the Result

Table 11.11: Functional experiment record template

Field	Information to record
Identity	Name/number, date, experiment name; in the form the course asks for.
Environment	rv32 version; board file; IMEM/DMEM sizes.
Program and input	Source file name, change, input constants and switch values.
Expected	Register/memory/Console value at the checkpoint.
Observed	The actual value at the same checkpoint and a screenshot if needed.
Result	Agreement with the expectation; explanation of the first difference and the correction.
Outputs	Names of the requested .s, board, and .mem files; initial/current distinction.

This record is for a functional experiment. The clock, monitoring counters, and CPI/IPC measurements required for the hardware performance report in Chapter 9 of the main book are not derived from the rv32 screen. Submission scope and assessment rules are determined by the instructor.

11.12.3 Review Questions

- Why might a0 not change immediately on the first F8 press?
- Why can the la line produce more than one instruction?
- Why might result differ between Generate and Memory export?
- Why might two memory images that have the same byte offset not run directly on REF32?
- Does a branch result being correct show how many clocks it will take in hardware?

11.13 Sources and Relation Across Chapters

The sources of this chapter are given below. Page numbers in the main-book citations are the book’s printed page numbers rather than the PDF viewer’s sequential numbers.

Table 11.12: Sources used

Code	Source and use
K1	EHB326_Book_MCU.pdf, Rev. P1.0: §2.4, p. 2; Chapter 3, p. 3–17; Chapter 5, p. 36–49; Chapters 6–7, p. 51–66; Chapters 8–10, p. 67–94; Appendix A, p. 97–104.
K2	rv32 README.md and docs/building.md: application opening and package use.
K3	docs/keybindings.md; src/gui/main_window.cpp: shortcuts, panels, export scope, and stepping.
K4	docs/memory-map.md; src/core/device_config.cpp: emulator address spaces and board scope.
K5	docs/isa-support.md; src/dbg/pipeline.cpp: ISA and display limits.
K6	src/main.cpp and src/cmd/commands.cpp: CLI options and debugger commands.
K7	The <code>examples/rv32/</code> and <code>validation.txt</code> files in the book: executed starter examples and their validated results.

The narrative order established in the main book with “What Is It?”, “Purpose”, “Usage”, a usage example, and debug explanations has been preserved here. The new content completes tool use without changing the main book’s hardware definition.

The suggested place for merging is after Chapter 10 and before the bibliography/appendices. A short introduction can be added to §2.4 with a pointer to this chapter. In an alternative placement the chapter number must be updated with the main document’s counter; figures, tables, and cross-references must be regenerated.

Bibliography

- [1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Official online document.
- [2] RISC-V International, *RV32I Base Integer Instruction Set, Version 2.1*. Official RV32I reference.
- [3] RISC-V International, *M Extension for Integer Multiplication and Division, Version 2.0*. Official M extension reference.
- [4] RISC-V International, *C Extension for Compressed Instructions, Version 2.0*. Official C extension reference.
- [5] RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. Official privileged ISA document.
- [6] Arm Limited, *Armv8-A Instruction Set Architecture*, Non-Confidential, Issue 1.1, Document ID: ARM062-948681440-3280, 2019–2020.

DRAFT

Appendix A

Address Map and System Capacity

In this chapter all address decode windows used in the REF32 profile and the defined register addresses are given in increasing address order.

The window selected during address decode may be larger than the register space that the related peripheral actually uses. Therefore an address being inside the related peripheral window does not mean that that address necessarily corresponds to a valid register.

An access made to an undefined offset or to an address at an unsupported alignment produces an error response.

The absolute address of a register is computed as:

$$\text{absolute address} = \text{base address} + \text{offset}$$

A.1 REF32 Capacity Summary

The REF32 profile denotes microBee's reference SoC configuration. The capacities of the basic memory, peripheral, and microarchitectural resources used inside this profile are given in the table below.

Table A.1: REF32 system capacity.

Resource	Capacity	Note
IMEM BRAM	16384×32 bit, 64 KiB	Instruction fetch, start address 0x80000000
DMEM BRAM	16384×32 bit, 64 KiB	Load/store, start address 0x20000000
Reset / mtvec	0x80000000	RESET_VECTOR
MMIO windows	6×4 KiB	CLINT, PLIC, GPIO, TIMER, UART, and MONITOR
GPIO	32 bit, 1 bank	GPIO_WIDTH=32
PLIC	16 interrupt sources, 3-bit priority	ID 1 AXI UART, ID 2 Timer, ID 3 Native UART
Native UART FIFO	16 entries	8N1 UART
Timer	64-bit counter, 2 compare channels	PLIC ID 2
Monitoring	32-bit saturating counters	Offset +0x100 for shadow registers
Branch predictor	128 entries, bimodal	BP_ENTRIES=128
AXI window	256 MiB	Start address 0x60000000
FPGA clock / baud	50 MHz / 115200	AXI UART Lite

In the REF32 profile the basic memory regions that can be used by software are IMEM, DMEM, and the AXI window.

IMEM holds the program's instructions and is used by the Fetch unit. DMEM is the data memory accessed by load/store instructions.

The AXI window can be used only when:

$$\text{AXI_ENABLE}=1$$

In MMIO regions, on the other hand, only the register addresses defined for the related peripheral are valid. Accesses made to the remainder of the decode window produce an error response.

A.2 Global Memory Map

Inside the microBee request router, address regions are checked according to a definite priority order.

Router priority is:

CLINT → PLIC → GPIO → TIMER → UART → MONITOR → DMEM → AXI → Error

The REF32 global memory map is as follows.

Table A.2: REF32 memory windows, in address order.

Start	End	Region	Access
0x00000000	0x1FFFFFFF	Empty	Error
0x20000000	0x2000FFFF	DMEM BRAM	RW, 64 KiB
0x20010000	0x3FFFFFFF	Empty	Error
0x40000000	0x40000FFF	CLINT	Register map
0x40001000	0x40001FFF	PLIC	Register map
0x40002000	0x40002FFF	GPIO	Register map
0x40003000	0x40003FFF	TIMER	Register map
0x40004000	0x40004FFF	Native UART	Register map
0x40005000	0x40005FFF	MONITOR	Register map
0x40006000	0x40006FFF	Reserved mailbox	Error in REF32
0x40007000	0x5FFFFFFF	Reserved	Error
0x60000000	0x6FFFFFFF	AXI4-Lite	AXI UART Lite @ 0x60000000
0x70000000	0x7FFFFFFF	Empty	Error
0x80000000	0x8000FFFF	IMEM BRAM	Instruction fetch, data alias off
0x80010000	0xFFFFFFFF	Empty	Error

If a peripheral's enable parameter is 0, the related peripheral is removed from the system address map.

For example, when:

MONITOR_ENABLE=0

accesses made to the MONITOR window are not served by a valid peripheral and an error response occurs.

A.3 DMEM — 0x20000000

DMEM is the 64 KiB RW data memory that the program uses for load and store operations.

The start address is defined as:

0x20000000

In word, half-word, and byte store operations, which byte lanes will be written is determined by the `wstrb` signal.

In software examples the stack pointer is generally started around:

0x2000FF00

Test firmware may use some DMEM addresses as a software mailbox. For example:

- 0x20000000: test signature,
- 0x20000100: test phase.

Using these addresses as a mailbox is not a separate hardware decode mechanism. The addresses are part of the normal DMEM region.

A.4 CLINT — 0x40000000

CLINT is the peripheral used in producing the machine software interrupt and the machine timer interrupt. The base address is defined as:

0x40000000

Table A.3: CLINT register map.

Offset	Absolute address	Register	Access
0x000	0x40000000	MSIP	RW, bit 0
0x400	0x40000400	MTIMECMP_LO	RW
0x404	0x40000404	MTIMECMP_HI	RW
0xFF8	0x40000FF8	MTIME_LO	RW
0xFFC	0x40000FFC	MTIME_HI	RW

mtime and mtimecmp are 64-bit values, but in an RV32 system they are accessed in two parts through 32-bit registers.

A.5 PLIC — 0x40001000

PLIC is used in managing the priority, pending, enable, and trigger mode information of external interrupt sources.

In the REF32 profile there are 16 external interrupt sources.

The PLIC base address is defined as:

0x40001000

Table A.4: PLIC register map.

Offset	Absolute address	Register	Access
0x000	0x40001000	CAPABILITY	RO
0x004+4(<i>i</i> − 1)	starting at 0x40001004	PRIORITY[<i>i</i>]	RW, <i>i</i> = 1..16
0x100	0x40001100	PENDING	RO
0x104	0x40001104	SOURCE_MODE	RW
0x108	0x40001108	INPUT_STATE	RO
0x10C	0x4000110C	RESET_LEVEL	RO
0x180	0x40001180	ENABLE	RW
0x300	0x40001300	THRESHOLD	RW
0x304	0x40001304	CLAIM	RO, read has a side effect
0x308	0x40001308	COMPLETE	WO

A.6 GPIO — 0x40002000

The GPIO peripheral provides software control of general-purpose digital input and output pins.

The base address is defined as:

0x40002000

Table A.5: GPIO register map.

Offset	Absolute address	Register	Access
0x00	0x40002000	IN	RO, write no-op
0x04	0x40002004	OUT	RW
0x08	0x40002008	OUT_SET	WO, read 0
0x0C	0x4000200C	OUT_CLR	WO, read 0
0x10	0x40002010	OUT_TGL	WO, read 0
0x14	0x40002014	OE	RW
0x18	0x40002018	VALID	RO
0x1C	0x4000201C	—	Invalid, error response

A.7 TIMER — 0x40003000

Native Timer is the 64-bit timer peripheral used for general-purpose timing and compare operations.

The base address is defined as:

0x40003000

Table A.6: Native Timer register map.

Offset	Absolute address	Register	Access
0x00	0x40003000	CTRL	RW
0x04	0x40003004	PRESCALE	RW
0x08	0x40003008	COUNT_LO	RW
0x0C	0x4000300C	COUNT_HI	RW
0x10	0x40003010	CMPO_LO	RW
0x14	0x40003014	CMPO_HI	RW
0x18	0x40003018	CMP1_LO	RW
0x1C	0x4000301C	CMP1_HI	RW
0x20	0x40003020	PERIOD_LO	RW
0x24	0x40003024	PERIOD_HI	RW
0x28	0x40003028	STATUS	RW1C
0x2C	0x4000302C	CAPABILITY	RO

The timer interrupt output is connected to PLIC ID 2 in the REF32 profile.

A.8 Native UART — 0x40004000

Native UART is the UART peripheral found inside microBee RTL.

The base address is defined as:

0x40004000

Table A.7: Native UART register map.

Offset	Absolute address	Register	Access
0x00	0x40004000	DATA	RW, write for TX / read for RX
0x04	0x40004004	STATUS	RO
0x08	0x40004008	CTRL	RW
0x0C	0x4000400C	BAUDDIV	RW, minimum 8
0x10	0x40004010	IRQ_STATUS	RW1C
0x14	0x40004014	FIFO_LEVEL	RO
0x18	0x40004018	CAPABILITY	RO

The Native UART interrupt output is connected to PLIC ID 3 in the REF32 profile.

A.9 MONITOR — 0x40005000

The monitoring unit provides observation of microarchitectural performance events through hardware counters. The base address is defined as:

0x40005000

The MON_CTRL register is RW; the performance counters are RO.

The offset of snapshot shadow registers is found by adding:

+0x100

to the related live counter offset.

Table A.8: Monitoring register map.

Offset	Absolute address	Register	Access
0x000	0x40005000	MON_CTRL	RW
0x004	0x40005004	CYCLES	RO
0x008	0x40005008	RETIRED	RO
0x00C	0x4000500C	STALL	RO
0x010	0x40005010	FLUSH	RO
0x014	0x40005014	BUBBLE	RO
0x018	0x40005018	HAZARD	RO
0x01C	0x4000501C	LOAD_USE	RO
0x020	0x40005020	BRANCH	RO
0x024	0x40005024	BRANCH_TAKEN	RO
0x028	0x40005028	MISPREDICT	RO
0x02C	0x4000502C	IMEM_WAIT	RO
0x030	0x40005030	DMEM_WAIT	RO
0x034	0x40005034	AXI_WAIT	RO
0x038	0x40005038	BUS_ERROR	RO
0x03C	0x4000503C	IRQ_TAKEN	RO
0x040	0x40005040	ALU_USE	RO
0x044	0x40005044	MUL_USE	RO
0x048	0x40005048	DIV_USE	RO
0x04C	0x4000504C	CAPABILITY	RO
0x050	0x40005050	LOAD_COUNT	RO
0x054	0x40005054	STORE_COUNT	RO
0x058	0x40005058	BRANCH_STALL	RO
0x05C	0x4000505C	MEM_WAIT_TOTAL	RO
0x060	0x40005060	PREDICT_CORRECT	RO
0x064	0x40005064	SPEC_REDIRECT	RO
0x104	0x40005104	CYCLES shadow	RO
+0x100	live offset + 0x100	Other shadow counters	RO

A.10 AXI UART Lite — 0x60000000

AXI UART Lite is different from the Native UART found inside microBee RTL.

This unit is added to the system as a Vivado catalog IP in the FPGA design and is connected to microBee over AXI4-Lite.

In the REF32 FPGA configuration the base address is defined as:

0x60000000

Table A.9: AXI UART Lite register map.

Offset	Absolute address	Register	Access
0x00	0x60000000	RX_FIFO	RO
0x04	0x60000004	TX_FIFO	WO
0x08	0x60000008	STAT	RO
0x0C	0x6000000C	CTRL	RW

A.11 IMEM — 0x80000000

IMEM is the 64 KiB program memory in which instructions are held.

The start address is defined as:

0x80000000

The reset vector and the default `mtvec` address are in this region.

IMEM is used only on the instruction fetch side. Because the data-bus alias is off, IMEM access is not made through normal load/store instructions.

A.11.1 Alignment

MMIO accesses must be word aligned.

Valid alignment is:

`address[1:0] = 00`

On accesses at an unsupported alignment, `rsp_err` is produced.

A.12 CSR Numbers

CSR addresses must not be mixed with normal memory addresses.

A CSR is a 12-bit register number and is not a load/store address.

In `CSRW`, `CSRRS`, `CSRRC`, and their immediate forms, the CSR number is found in the instruction field:

`instruction[31:20]`

microBee implements only machine mode.

The supported basic CSR set is given in the table below.

Table A.10: microBee machine-mode CSR set, in number order [5].

Number	CSR	Access	Function
0x300	<code>mstatus</code>	MRW/WARL	MIE, MPIE, and MPP
0x304	<code>mie</code>	MRW/WARL	bit3 MSIE, bit7 MTIE, bit11 MEIE
0x305	<code>mtvec</code>	MRW/WARL	Handler base address, [1:0]=00, Direct mode
0x340	<code>mscratch</code>	MRW	Working space that can be used by software during a trap
0x341	<code>mepc</code>	MRW/WARL	Trap return PC, [0]=0
0x342	<code>mcause</code>	MRW	bit31 interrupt information and cause code
0x343	<code>mtval</code>	MRW	Value associated with the trap
0x344	<code>mip</code>	MRW/WARL	MSIP, MTIP, and MEIP pending information
0xB00	<code>mcycle</code>	MRW	Cycle counter [31:0]
0xB02	<code>minstret</code>	MRW	Retired instruction counter [31:0]
0xB80	<code>mcycleh</code>	MRW	Cycle counter [63:32]
0xB82	<code>minstreth</code>	MRW	Retired instruction counter [63:32]

After reset, the basic CSR values are:

- `mstatus` = 0x00001800,
- `mie` = 0,
- software `mip` bits = 0,
- `mtvec` = 0x80000000,
- `mcycle` = 0,
- `minstret` = 0.

`mcycle` increments on every clock cycle.

`minstret`, on the other hand, increments with architecturally completed instructions.

These CSR counters are not the same registers as the memory-mapped `CYCLES` and `RETIRED` counters in the monitoring unit.

If in a `CSRRS` or `CSRRC` instruction the source register is `x0`, only a CSR read operation is performed.

`CSRWR`, on the other hand, performs a write operation on the CSR.

A.12.1 `mcause` Codes

`mcause` indicates the trap cause.

Bit 31 indicates whether the trap is an interrupt:

- 0 = synchronous exception,
- 1 = interrupt

Table A.11: Exception cause codes, bit31=0.

mcause	Event
0	Instruction address misaligned
1	Instruction access fault
2	Illegal instruction
3	Breakpoint (<code>ebreak</code>)
4	Load address misaligned
5	Load access fault
6	Store address misaligned
7	Store access fault
11	Environment call, M-mode <code>ecall</code>

Table A.12: Interrupt cause codes, bit31=1.

mcause	Source
0x80000003	Machine Software Interrupt, CLINT <code>MSIP</code>
0x80000007	Machine Timer Interrupt, CLINT <code>mtime</code> / <code>mtimecmp</code>
0x8000000B	Machine External Interrupt, PLIC

When more than one interrupt is in an acceptable state at the same time inside microBee, interrupt priority from high to low is:

External → Software → Timer